Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e Statistica

Laurea Triennale

Dipartimento di Informatica

Sistemi Operativi

Appunti a cura di
Lorenzo Ferraro

11 novembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Sistemi Operativi*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale `github.com/TonnoArcobaleno/uni-notes`, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@proton.me**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1 Introduzione **1**
1.1 Architettura dell'Hardware . . . 1
1.1.1 Von Neumann Architecture . . . 1
1.1.2 Ciclo delle istruzioni . . . 3
1.1.3 CPU . . . 4
1.1.4 CPU Registers . . . 4
1.1.5 Instruction Pipelining . . . 5
1.1.6 CPU Performance . . . 7
1.1.7 I/O . . . 11
1.2 Introduzione ai sistemi operativi . . . 13
1.2.1 Cos'è il sistema operativo? . . . 13
1.2.2 Astrazioni . . . 15
1.2.3 Dual-Mode . . . 17
1.2.4 Chiamate di sistema . . . 18
1.2.5 Interruzioni, eccezioni e trappole . . . 19
1.2.6 Protezione della memoria . . . 21
1.3 Introduzione a C . . . 22
1.3.1 I tipi in C . . . 24
1.3.2 Array . . . 26
1.3.3 Stringhe in C . . . 29
1.3.4 Puntatori . . . 31
1.3.5 Errori nella gestione della memoria . . . 34
1.3.6 Stack . . . 36
1.3.7 Heap . . . 37
1.3.8 Altri errori nella gestione della memoria . . . 39
1.3.9 Fasi della compilazione . . . 40
1.3.10 Linking statico e dinamico . . . 45

2 Virtualizzazione **47**
2.1 Processi e API . . . 47
2.1.1 Gestione dei processi in UNIX . . . 49
2.1.2 Altre funzioni per gestire i processi in UNIX . . . 55
2.1.3 Shell . . . 56
2.2 Scheduling . . . 56
2.2.1 Policy di scheduling . . . 58
2.2.2 Incorporare operazioni I/O . . . 62
2.2.3 Implementazioni delle diverse policy in C . . . 62

2.3 Policy di scheduling più complesse . . . 63
2.3.1 MLFQ . . . 63
2.3.2 Proportional Share Scheduling . . . 65
2.3.3 Linux passa a CFS . . . 67
2.4 Virtualizzazione della memoria . . . 68
2.4.1 Perché virtualizzare la memoria? . . . 68
2.4.2 Passare da memoria virtuale a fisica . . . 70
2.4.3 Swapping . . . 80
2.4.4 Thrashing . . . 83

3 Concorrenza **85**
3.1 Thread . . . 85
3.1.1 **`pthread`** API . . . 86
3.1.2 Problemi del multithreading . . . 87
3.2 Implementazione del **`lock`** . . . 90
3.2.1 Implementazioni software-only . . . 90
3.2.2 Implementazioni hardware . . . 92
3.2.3 Tabella riepilogativa . . . 100
3.3 Condition variable . . . 101
3.3.1 Fare aspettare un thread . . . 101
3.3.2 Struttura interna . . . 102
3.3.3 Problema del produttore e consumatore . . . 103
3.3.4 Due modi per implementare il signal . . . 105
3.3.5 Quando il **`signal`** non è più sufficiente . . . 106
3.3.6 Thread Pool . . . 106
3.4 Semaforo . . . 107
3.4.1 Unire lock e CV . . . 107
3.4.2 Cos'è un semaforo? . . . 108
3.4.3 **`S = 0`** . . . 108
3.4.4 Produttore e consumatore con i semafori . . . 109
3.4.5 Filosofi a cena . . . 110
3.4.6 Implementare il semaforo . . . 111

Capitolo 1

Introduzione

1.1 Architettura dell'Hardware

1.1.1 Von Neumann Architecture

Prima di entrare nel vivo del corso, è bene fare un'introduzione riguardo all'architettura dell'hardware. Questo perché i sistemi operativi si pongono come **ponte** tra hardware e software.

Definizione 1.1 – Von Neumann Architecture

L'**Architettura Von Neumann** è il fondamento teorico di tutte le architetture hardware dei computer moderni. Si basa su diversi principi:

- Le istruzioni e i dati **si trovano nello stesso spazio in memoria**
- Le istruzioni sono **sequenziali**: vengono lette una ad una dalla memoria per poi essere eseguite dalla CPU

Le componenti principali che la compongono sono:

- **Processing Unit**, dove si trovano la **ALU** (un circuito digitale dove vengono svolti i calcoli algebrici più semplici) e i **registri** (piccolissime e velocissime unità di memoria volatile)
- **Control Unit**
- **Memoria RAM** dove si trovano i dati e le istruzioni
- **Mass Storage / External I/O**
- **System Bus**, il quale collega tutti gli altri componenti e li fa lavorare insieme

La Control Unit e la Processing Unit si trovano nella CPU, e comunicano con la RAM e con i dispositivi I/O tramite il System Bus.

Osservazione 1.1 – Von Neumann Bottleneck

Un classico problema dell'Architettura Von Neumann si verifica quando la RAM e la CPU comunicano: essendo la CPU molto più veloce, quando queste lavorano la RAM non riesce a tenere il passo, e dato che la CPU ha bisogno dei dati che si trovano in RAM, si ritrova spesso ad **aspettare inutilmente**, rendendo questa architettura non sempre efficiente.

Per risolvere questo problema è importante cercare di non accedere alla memoria se non strettamente necessario, sfruttando la cache ad esempio.

Definizione 1.2 – Harvard Architecture

Esiste un'altra scuola di pensiero riguardo all'architettura hardware: l'**Architettura Harvard**. La differenza più importante rispetto all'Architettura Von Neumann è che qui i dati e le istruzioni **non si trovano nello stesso spazio in memoria**.

Definizione 1.3 – System Bus

Il **System bus** (o Internal Bus) è l'insieme di cavi che permette alla CPU, alla memoria e all'I/O di comunicare fra loro. Questo si divide in 3 componenti principali:

- **Address Bus**, che contiene l'indirizzo di memoria del dato richiesto dalla CPU
- **Data Bus**, che trasporta effettivamente i dati
- **Control Bus**, che trasporta il comando da eseguire sul dato

L'**Address Bus** è **unidirezionale**, dato che viaggia sempre dalla CPU verso la memoria e mai il contrario. La **Bus width** è il numero di linee fisiche che compongono il bus. Con **N** linee si riesce ad ottenere un numero massimo di $\mathbf{2}^{N}$ locazioni uniche. Fino a qualche anno fa erano comuni i sistemi a 32 bit, ma essendo diventati troppo limitanti si è passati ai modelli a 64 bit. Questi, in realtà, sono troppo anche per gli hardware moderni, e dunque per risparmiare si usano in pratica indirizzi virtuali a 48 bit, molto più che sufficienti.

Il **Data Bus** è **bidirezionale**, dato che il dato può sia arrivare alla CPU (se lo si vuole leggere), sia partire da essa (se lo si vuole scrivere). La **Bus width** indica quanti bit si possono trasferire in un singolo clock cycle. La **Word Size** invece è la quantità di bit con cui la CPU elabora i dati (da cui si dividono i processori a 32 bit o a 64 bit).

Il **Control Bus** può trasportare diversi messaggi, tra cui i più comuni sono:

- **MEMR/MEMW** (Memory Read/Memory Write), che permettono di leggere/scrivere dati in memoria
- **IOR/IOW** ((I/O Read)/(I/O Write)), che permettono di comunicare con i dispositivi I/O
- **Bus Request/Bus Grant**, che permettono al DMA controller di usare il Bus

- **Interrupt Signals**, messaggi mandati dai componenti hardware per notificare un evento alla CPU.

1.1.2 Ciclo delle istruzioni

Definizione 1.4 – Ciclo delle istruzioni

Il **ciclo delle istruzioni** è alla base di ogni operazione di un computer, che lo esegue fin dall'avvio. Si divide in tre passaggi:

1° **Fetch**

2° **Decode**

3° **Execute**

Fetch

Durante la fase di Fetch, la CPU copia il valore del **PC** (Program Counter), che contiene l'indirizzo di memoria dove si trova la prossima istruzione da eseguire, nel **MAR** (Memory Address Register) e lo inserisce anche nell'address bus. A questo punto la Control Unit manda un comando di Memory Read al control bus. Si legge dunque l'indirizzo di memoria dell'istruzione e l'istruzione viene inserita nel data bus. La CPU la legge e la inserisce nell'**IR** (Instruction Register). Infine, il PC viene incrementato così da contenere al suo interno l'indirizzo della prossima istruzione.

Decode

Durante la fase di decodifica, la Control Unit legge i bit che si trovano nell'Instruction Register. Identifica l'**Opcode** (Operation Code), ossia l'operazione da eseguire che può essere ad esempio `ADD` o `LOAD`, e gli **Operands**, ossia i registri, gli indirizzi di memoria o i valori semplici su cui eseguire tale operazione. A questo punto sempre la Control Unit prepara la ALU e i registri per eseguire tale operazione.

Execute

Durante la fase di esecuzione, in base al tipo di istruzione vi sono tre possibili scenari:

1) Se l'istruzione è aritmetica, come ad esempio l'istruzione `add %eax, %ebx` (vedremo ogni tanto del codice assembly, non è essenziale ma può aiutare a capire di cosa si sta parlando. In questo caso, stiamo facendo una somma tra i valori contenuti all'interno di due registri, `eax` e `ebx`), parliamo di **ALU operations**, dato che è la ALU che se ne occupa, facendo l'operazione e scrivendo il risultato finale.

2) Se l'istruzione è una `load/store`, come ad esempio l'istruzione `mov (%rax), %rcx` (carichiamo in `%rcx` il valore che si trova in memoria all'indirizzo contenuto in `%rax`, performando dunque un'operazione di `load`), parliamo di **Memory operations**, dove la CPU legge/scrive dati nella RAM usando MAR o MDR.
3) Se l'istruzione è un `jump/branch`, come ad esempio l'istruzione `jmp 0x4010a0` (stiamo saltando all'indirizzo `0x4010a0`), parliamo di **Control flow**, dove il Program Counter viene aggiornato direttamente all'indirizzo indicato, modificando il flow di esecuzione.

1.1.3 CPU

Definizione 1.5 – CPU

La **CPU** è il cervello del computer, che esegue le operazioni in memoria. Questa è divisa in tre blocchi principali:

- **Datapath**, che contiene i registri e la ALU
- **Control Unit**, che prende le istruzioni e le decodifica, e in base a queste gestisce il datapath
- **Internal Bus**, che collega i registri, la ALU e la CU (Control Unit)

Il **Datapath** performa le operazioni aritmetiche (somma, sottrazione, moltiplicazione e divisione) e **bitwise** (`AND`,`OR`,`XOR`,`NOT` e gli shift), grazie alla ALU, e ha al suo interno i **GPRs** (General-Purpose Registers), che sono delle celle di memoria interne alla CPU per mantenere dati così da non dover sempre accedere alla memoria, risparmiando tempo.

La **Control Unit** gestisce il datapath e gli dice **come** e **quando** agire. Prende in input l'Instruction Register, dei segnali di clock o dei flags, e in base a questi manda in output segnali diversi alla ALU, ai registri, o ai sistemi di I/O. È il coordinatore della CPU.

1.1.4 CPU Registers

Abbiamo visto nel datapath i registri, che sono piccolissime e velocissime unità di memoria nella CPU. Si dividono in due macrogruppi:

- **User-visible registers** (ossia i **General-Purpose**), che sono richiamabili direttamente tramite assembly (come i registri `%rax` e `%rbx` visti prima)
- **Control & Status registers** (ossia i **Special-Purpose**), che sono usati dalla Control Unit per modificare il flusso di esecuzione

General-Purpose Registers

In base al tipo di CPU questi registri sono salvati con nomi differenti:

- Per le CPU con architettura `x86-64` vi sono 16 registri chiamati `%rax, %rbx, %rcx, %rdx, %rsi, %rdi, %rbp, %rsp` e poi da `%r8` fino a `%r15`
- Per le CPU con architettura `ARM64` vi sono 31 registri che vanno da `x0` a `x30`

Special-Purpose Registers

Vi sono molti Special-Purpose Registers, come ad esempio il Program Counter, l'Instruction Register, lo Stack Pointer, il MAR e l'MDR e il PSR. Vediamoli più nello specifico:

- Il **Program Counter**, come già detto, contiene l'indirizzo di memoria per la prossima istruzione da eseguire.
- L'**Instruction Register** contiene l'istruzione da eseguire.
- Lo **Stack Pointer** mantiene l'indirizzo di memoria della cima della call stack
- Il **MAR** (Memory Address Register) contiene l'indirizzo che sta venendo letto o scritto sulla RAM. È strettamente collegato all'address bus.
- L'**MDR** (Memory Data Register, o anche detto MBR, ossia Memory Buffer Register) contiene i dati letti dalla RAM o che devono esser scritti sulla RAM. È strettamente collegato al data bus.
- Il **PSR** (Program Status Register) contiene i bit flag che indicano lo stato della CPU e l'ultima operazione effettuata dall'ALU. Tra le flag più note ci sono:
  - **Zero Flag** (Z), che diventa 1 se l'ultima operazione dell'ALU ha restituito 0 come valore
  - **Sign/Negative Flag** (S/N), che diventa 1 se l'ultimo risultato era negativo

  Queste flag sono fondamentali per eseguire if e cicli: infatti, per controllare ad esempio se due valori `x` e `y` siano uguali, il computer effettua una sottrazione e verifica se il risultato sia zero grazie proprio alla Zero Flag.

1.1.5 Instruction Pipelining

Definizione 1.6 – Pipelining

Nei processori più semplici, ogni istruzione deve essere presa, decodificata ed eseguita prima di poter passare alla prossima. Questo però non è molto efficiente, e per questo oggi si utilizza una tecnica nota come **pipelining** dove più istruzioni vengono eseguite contemporaneamente.

Proprietà 1.1 – RISC 5-Stage Pipeline

Una pipeline standard si divide in **5 stage**:

1° **IF** (Instruction Fetch): si prende l'istruzione

2° **ID** (Instruction Decode): si decodifica l'istruzione

3° **EX** (Execute): si esegue l'istruzione

4° **MEM** (Memory Access): si leggono o si scrivono i dati dell'operazione (eseguita solo se necessaria)

5° **WB** (Writeback): scrive il risultato nel registro di destinazione

Proprietà 1.2 – Pipeline Hazards

Tramite il Pipelining l'output ideale sarebbe di eseguire un'istruzione per clock cycle, ma questo è spesso quasi impossibile perché vi sono diversi **hazards** che rallentano la pipeline. Si dividono in:

- **Structural Hazards**, quando due istruzioni diverse hanno bisogno dello stesso hardware fisico e dunque non possono essere eseguite contemporaneamente.
- **Data Hazards**, quando un'istruzione dipende dal risultato di un'altra e dunque deve aspettare che questa finisca.
- **Control Hazards**, quando il PC viene modificato da un `jump/branch`, modificando il flusso di esecuzione e rendendo le istruzioni prese inutili. Per risolvere tali problemi molte CPU moderne utilizzano i **branch predictors**, che provano a indovinare il risultato di un branch. Possono essere molto utili, ma quando questi dovessero sbagliare si perderebbe ancora più tempo del normale.

Definizione 1.7 – Instruction Set Architectures

La **ISA** (Instruction Set Architecture) definisce l'interfaccia tra hardware e software. Vi sono due possibili scelte molto diverse:

- **CISC** (Complex Instruction Set Computer), che ha una lunga lista di istruzioni già pronte, come ad esempio gli Intel x86/x86-64
- **RISC** (Reduced Instruction Set Computer), che ha invece un set di istruzioni più semplici, come ad esempio gli ARM (Apple M-series)

1.1.6 CPU Performance

Definizione 1.8 – Processori multicore

Per superare i limiti di potenza, si è pensato di **usare più processori** (**core**) su un singolo dispositivo.
Ogni core è una CPU indipendente che può far partire il proprio ciclo di istruzioni indipendentemente dalle altre.

Definizione 1.9 – Clock

Ogni CPU lavora in base al proprio **System Clock**. Esistono il **Clock period**, che è la durata di un singolo clock cycle (come ad esempio 0.25 ns), e la **Clock Frequency**, ossia il numero di cicli di Clock al secondo, misurato in Hz e che si calcola banalmente prendendo il reciproco del periodo.

Le istruzioni più semplici impiegano anche un solo ciclo, ma quelle più complesse possono impiegarne anche molti. Esiste quindi il **CPI**, ossia il numero di cicli medio per istruzione, e anche l'**IPC**, ossia il numero di istruzioni per ciclo (che non è altro che l'inverso del CPI). Avendo più core, i processori moderni riescono nelle condizioni ideali anche a superare il limite di un'istruzione per ciclo, ottenendo quindi un valore di IPC > 1.

Osservazione 1.2 – Memory wall

Negli anni, i processori sono migliorati in modo esponenziale, al contrario della DRAM, che ha avuto un miglioramento molto più lineare. Questo gap ha portato alla creazione di un **memory wall**: la CPU si ritrova spesso ad aspettare la RAM che è troppo lenta.
Abbiamo già visto in breve questo problema quando abbiamo parlato del Von Neumann Bottleneck.

Per risolvere questo problema, si è creata una **struttura gerarchica delle memorie**:

1° **Registri** (tempo: $< 1ns$, capacità: $<$ 1KiB)
2° **L1 Cache** (tempo: $1ns$, capacità: 32/64KiB)
3° **L2 Cache** (tempo: $3ns$, capacità: 256/512KiB)
4° **L3 Cache** (tempo: $10ns$, capacità: 8/64MiB)
5° **RAM** o Memoria Principale (tempo: $60ns$, capacità: 8/64GiB)
6° **SSD** (tempo: $10\mu s$, capacità: 256GiB/2TiB)
7° **HDD** (tempo: $10ms$, capacità: 1/10TiB)

È evidente che i registri sono molto più veloci di un SSD o della memoria RAM, ma sono anche meno capienti, e più costosi. Infatti, il costo di tutte queste memorie diminuisce scendendo giù nella gerarchia, rendendo l'HDD il meno costoso e i registri i più costosi.

Principio di località

Proprietà 1.3 – Principio di località

Quando si accede a un indirizzo in memoria, è molto probabile che bisognerà accedervi anche nel breve futuro. Per questo, si salvano queste informazioni in memoria cache per potervi accedere più velocemente senza dover tornare nella RAM. Questa viene detta **località temporale**.
Allo stesso modo, se si accede a un indirizzo in memoria, è molto probabile che bisognerà accedere anche agli indirizzi a esso vicini. Per questo, si salvano anche questi nella cache. Questa viene detta **località spaziale**.

Prendiamo questo codice in C come esempio:

```
int sum = 0;
for (int i = 0; i < 1000; i++){
    sum += array[i];
}
```

Per la località temporale, le variabili `sum` e `i` sono salvate nei registri o nella L1 cache, così da non dover riaccedere alla DRAM. Allo stesso modo, per la località spaziale, è molto probabile che anche i valori `array[1], array[2]...` saranno richiesti in un breve futuro, dunque la CPU salva il maggior numero possibile di questi valori riempiendo una linea di L1 cache, di 64 byte. Dato che ogni `int` in C occupa 4 byte, si salveranno in totale 16 valori dell'array, da `array[0]` a `array[15]`. In questo modo accediamo una sola volta alla DRAM al posto di 16.

RAM

Definizione 1.10 – RAM

La memoria **RAM** si divide in due sottocategorie:

1) **SRAM** (Static RAM), veloce ma costosa. Usata per le cache delle CPU
2) **DRAM** (Dynamic RAM), lenta ma più economica. Usata per la memoria principale (quando si parla di RAM si intende quasi sempre la DRAM)

Fisicamente, la RAM è un **vettore lineare di locazioni**. Nella maggior parte delle architetture l'indirizzamento è al byte: a ogni indirizzo univoco corrisponde esattamente un singolo byte (8 bit). La CPU può comunque richiedere accessi a parole di 4 o 8 byte, accedendo a una sequenza di indirizzi contigui.

Cache

Definizione 1.11 – Linee di cache

Le cache non operano sui singoli byte, ma su strutture chiamate **linee di cache**. Queste sono blocchi (di solito di 64 byte), che contengono:

- Un **valid bit**, che indica se i dati salvati sono validi o meno
- I **tag bit**, utili per identificare la singola linea
- I **data block**, ossia gli effettivi 64 byte copiati dalla RAM

Quando la CPU richiede un accesso in memoria, ci sono due possibilità:

- **Cache Hit**: la CPU trova l'indirizzo nel giro di 1/2 cicli.
- **Cache Miss**: la CPU non trova l'indirizzo nella cache, dunque è costretta a cercarlo nella DRAM, arrivando a impiegare fino a 100 cicli di clock.

Cerchiamo di calcolare la AMAT (Average Memory Access Time), ossia il tempo medio per accedere a un indirizzo di memoria.

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

Più il Miss Rate aumenta, più questo valore aumenta, rendendo l'intero processo più lento.

Osservazione 1.3 – Cache Coherence Problem

Nei sistemi multicore, ogni core ha una sua L1 cache. Questo però crea un **problema di coerenza**: se il core 0 e il core 1 hanno lo stesso valore X, ma il core 1 lo aggiorna modificandone il valore, il core 0 non avrà il valore aggiornato.

Per risolvere questo problema vi sono i **coherence protocols**, che invalidano o aggiornano le copie nelle altre cache ogni volta che si presenta questo problema.

Proprietà 1.4 – Cache Write Policies

Quando la CPU scrive sulla cache, vi sono due modi per aggiornare anche la RAM con il nuovo valore:

- **Write-Through Policy**: la CPU scrive in contemporanea sia sulla Cache che sulla RAM. Dunque la RAM è sempre aggiornata, ma ogni operazione di scrittura diventa più lenta dovendo ogni volta accedere alla RAM.
- **Write-Back Policy**: la CPU aggiorna solo la Cache line, che viene marcata come *Dirty*, e la RAM viene aggiornata solo quando la cache va eliminata. Dunque le operazioni di memoria sono veloci, ma la RAM contiene valori vecchi.

Cache Line Mapping Policies

Vi sono diversi modi per mappare un blocco RAM a una specifica sezione della cache:

1) **Direct Mapped**
2) **Fully Associative**
3) **Set Associative**

Il **Direct Mapped** è l'approccio più semplice: l'indice della cache viene mappato usando l'aritmetica modulare:

$$\text{Cache Slot} = \text{Block Address} \ \% \ \text{Numero di slot della cache}$$

Se il numero di slot per la cache è 4, avremo quindi che i blocchi $0, 4, 8, \ldots$ andranno allo slot 0, quelli $1, 5, 9, \ldots$ allo slot 1 e così via. In questo caso vi è un problema chiamato **cache thrashing**: se vi sono due blocchi che devono andare allo stesso slot, è possibile che questi ogni volta si rubino il posto a vicenda. Se ad esempio inizialmente serve il blocco 0, questo andrà nello slot cache 0. Quando serve il blocco 4, anche questo deve andare nello slot 0, dunque viene tolto il blocco 0 per far spazio al blocco 4. Se poi servirà nuovamente il blocco 0, bisognerà togliere il blocco 4 per fargli spazio e così via.

Il **Fully Associative** è invece l'approccio più libero: ogni blocco può andare dove vuole. Non vi è alcun cache thrashing, ma richiede ricerche in parallelo per controllare ogni singola linea di cache finché non si trova quella che serve. Questo è molto costoso e lento, rendendo questa tecnica utile solo per strutture molto piccole.

Il **Set Associative** è un compromesso tra il Direct Mapped e il Fully Associative. La cache viene organizzata in N insiemi, che contengono un valore di blocchi prestabilito (ad esempio, in un 4-way Set Associative ogni insieme ha 4 blocchi). Un blocco viene poi associato a uno di questi insiemi tramite l'aritmetica modulare:

$$\text{Set Index} = \text{Block Address} \ \% \ \text{Numero di insiemi}$$

Questo riduce notevolmente il cache thrashing (dato che vi è spazio per più di un blocco allo stesso indice), e allo stesso tempo mantiene velocità accettabili.

Memoria Secondaria

Vi sono due tipi di memoria secondaria:

- **HDD** (Hard Disk Drive), ossia il disco meccanico. Questo è molto lento perché è necessario che un braccio meccanico fisico si muova sopra la traccia, che il disco ruoti, e che venga letto/scritto qualcosa sulla superficie del disco. Il sistema operativo in questo caso deve fare il massimo per limitare i movimenti meccanici del disco.
- **SSD** (Solid State Drive), che invece non ha parti meccaniche. I dati qui sono strutturati in **pagine** che vengono riunite in **blocchi** di, ad esempio, 128 pagine. Si può leggere tranquillamente ogni singola pagina, ma per riscriverla bisogna cancellare l'intero blocco a cui appartiene, il che rende tutto molto più lento dato che bisognerà poi ricopiare l'intero blocco. Ogni blocco, poi, può supportare solo un numero limitato di cancellazioni prima che si degradi. Il sistema operativo deve in questo caso mandare dei comandi TRIM per aiutare il controller dell'SSD a gestire i blocchi vuoti.

1.1.7 I/O

Un computer ha senso solo se comunica con strumenti esterni. Vi sono ad esempio i **dispositivi di storage**, come ad esempio gli SSD o gli HDD, o le **interfacce utente**, come la tastiera o il mouse. Indipendentemente dal tipo, ogni dispositivo di I/O è gestito da due layer:

- **Device Controller** (Hardware), un microchip che espone dei nuovi registri alla CPU da poter controllare
- **Device Driver** (Software), un modulo nel sistema operativo che dice alla CPU come comunicare con tale dispositivo.

La CPU, per comunicare con il Device Controller, ha due possibilità:

- **PMIO** (Port-Mapped I/O), delle istruzioni della CPU dedicate all'I/O. Ad esempio, nelle CPU con architettura x86, vi sono i comandi `in` e `out` per riferirsi in modo chiaro ai dispositivi I/O.
- **MMIO** (Memory-Mapped I/O), dove i registri del device di I/O vengono mappati direttamente nella memoria fisica, potendo così utilizzare dei normali puntatori per accedervi.

Ad oggi, l'MMIO è lo standard.

Proprietà 1.5 – Registri del Device Controller

Nel Device Controller vi sono 3 tipi di registri:

- **Status Register**: read-only dato che riporta lo status del dispositivo
- **Command Register**: write-only dato che contiene le istruzioni per il dispositivo
- **Data Register**: read/write dato che contiene i dati mandati o ricevuti

I/O Transfer Protocols

La CPU ha tre diversi protocolli per comunicare con un dispositivo I/O:

- **Programmed I/O** (**Polling**)
- **Interrupt-Driven I/O**
- **DMA** (Direct Memory Access)

Il **Programmed I/O**, più comunemente chiamato **Polling**, è il metodo più semplice: semplicemente, se la CPU vuole ad esempio leggere qualcosa su un dispositivo I/O, legge lo Status Register del suo Device Controller, e se lo stato è "Busy" continua a chiederlo, finché non lo è più e può finalmente leggere i dati dal Data Register. Dunque semplicemente la CPU continua a chiedere al dispositivo i dati finché questo non è libero e può darglieli. Questo processo prende il nome di *busy-waiting*. Il problema più grande di questo protocollo è che la CPU è impegnata al 100%.

L'**Interrupt-Driven I/O** risolve il problema del busy-waiting: semplicemente, sono direttamente i dispositivi di I/O ad avvertire la CPU di esser pronti. Dunque la CPU manda la richiesta al device, e mentre aspetta esegue altre istruzioni. Quando il dispositivo manda l'ok, inviando un segnale su una **Interrupt Line**, la CPU mette in pausa l'esecuzione, e passa all'**IVT** (Interrupt Vector Table), un array di puntatori che le indica dove trovare la corretta **ISR** (Interrupt Service Routine), che è una speciale funzione che salva tutti i flags e i registers e li ripristina alla fine dell'esecuzione dell'istruzione. Vi sono due problemi con questo protocollo:

- salvare/ripristinare i registri e i flags richiede tempo
- se si ha un dispositivo molto veloce, è possibile che la CPU sia sempre interrotta perché le operazioni di I/O finiscono subito e le interruzioni arrivano di continuo, finendo così per utilizzarne quasi il 100% solo per gestirle.

Il **DMA** (Direct Memory Access) consiste nel delegare il trasferimento ad un controller hardware a parte, chiamato appunto **DMA controller**. Se per esempio bisogna spostare 4 KiB dal Disco alla RAM, la CPU può delegare tale operazione al DMA controller che lo farà senza dover coinvolgere la CPU. Alla fine, il DMA controller crea un singolo segnale che invia alla CPU

per avvisare della fine del trasferimento. Il più grande problema del DMA è che condivide il bus di sistema con la CPU, dunque se la CPU deve accedere alla RAM spesso finisce per dover aspettare il DMA controller.

La soluzione dunque è quella di sfruttare le potenzialità di tutti e 3 i protocolli, usando uno o l'altro in base alla situazione. Ad esempio, per operazioni veloci e semplici il polling è la migliore opzione, mentre per dispositivi lenti, come la tastiera, l'Interrupt-Driven I/O ha più senso.

1.2 Introduzione ai sistemi operativi

1.2.1 Cos'è il sistema operativo?

Definizione 1.12 – Sistema Operativo

Un **sistema operativo** (OS) è una collezione di sistemi software con l'obiettivo di gestire le risorse hardware e offrire dei servizi alle applicazioni.

Si può considerare il sistema operativo come il primo software che il computer avvia e che viene eseguito in una modalità privilegiata che prende il nome di **modalità kernel**, che rivedremo più avanti.

I vantaggi del sistema operativo sono molteplici:

- Senza di esso, dovremmo scrivere applicazioni che si occupano ad esempio anche di scrivere e leggere i dati direttamente sul disco o di gestire direttamente gli indirizzi della RAM. Con esso invece viene creata un'astrazione più semplice e pulita che tutte le app possono usare:
  - non ci sono più settori numerati sulla memoria secondaria ma **file** e **cartelle**
  - non c'è più memoria RAM da gestire direttamente ma ci sono **spazi di indirizzamento virtuali**
  - non ci sono più le interfacce network ma ci sono i **socket**
- La gestione delle risorse del computer è lasciata a lui. Decide quali programmi possono usare la CPU e per quanto tempo, non lascia nessuna applicazione senza risorse per troppo tempo e ottimizza questo utilizzo per minimizzare gli sprechi.
- Gestisce, inoltre, tutti i programmi in esecuzione. Isola le diverse applicazioni, in modo tale che due applicazioni non si ostacolino a vicenda, impedisce che queste modifichino codice o impostazioni di sistema e, se un'applicazione crasha, evita di far crashare tutto il sistema.

Definizione 1.13 – Kernel

Il **kernel** (nucleo) è la componente centrale del sistema operativo. È il primo software caricato dal computer ed è sempre attivo.

Proprietà 1.6 – Obiettivi sistema operativo

Gli obiettivi del sistema operativo sono molti:

- **Virtualizzazione**, ossia la pratica di far apparire le risorse fisiche come delle risorse logiche
- **Concorrenza**, ossia il saper gestire contemporaneamente più processi senza problemi mantenendo i dati consistenti
- **Persistenza**, ossia l'abilità di salvare i dati su dei sistemi progettati per il lungo termine (dato che la RAM è volatile, bisogna salvare spesso i dati e in modo veloce sulla memoria secondaria)
- **Sicurezza e isolazione**, ossia l'abilità di gestire la sicurezza dei file e gli spazi di memoria dei diversi processi
- **Efficienza**, ossia l'abilità di dare priorità ad aspetti diversi in base alla circostanza

Esistono due tipi di virtualizzazione:

- **CPU Virtualization**, dove l'OS trasforma una singola CPU fisica in molte CPU virtuali (dette processi o *thread*, che vedremo meglio tra poco)
- **Memory Virtualization**, dove l'OS dà a ogni processo il suo personale spazio di memoria virtuale da poter usare

Per quanto riguarda invece la sicurezza, come già accennato prima è necessario che ogni programma abbia il suo spazio di memoria virtuale a cui gli altri non possono accedere, e se un singolo programma crasha deve crashare solo lui e non tutto il sistema. Inoltre, in un sistema multi-user, per i file protetti è necessario controllare che l'utente che vuole accedervi abbia l'autorizzazione necessaria per poterlo fare.

Infine, per quanto riguarda l'efficienza, è importante capire che il SO fa sempre dei "*trade-off*", ossia deve scegliere se dare priorità a una cosa piuttosto che a un'altra. La domanda più classica è: "Velocità o sicurezza?" Purtroppo è necessario scegliere: se infatti si vuole la sicurezza, allora bisognerà aumentare i controlli e dunque la velocità diminuirà, altrimenti se si vuole aumentare quest'ultima il nostro sistema diventerà più vulnerabile.

Storia del sistema operativo

I primi computer negli anni '40 e '50 non avevano neppure un sistema operativo. Si usavano delle schede perforate dove veniva scritto il codice binario, si runnava il codice e lo si debuggava sul momento. La potenza della CPU era praticamente inespressa: doveva infatti rimanere ad aspettare un sacco di tempo l'input umano.

Alla fine degli anni '50 si vede un **primo prototipo di sistema operativo**: infatti i programmatori potevano caricare più programmi da eseguire insieme, e il sistema operativo si sarebbe occupato di gestirne il flusso: caricava il primo programma, lo eseguiva, e poi caricava il secondo e così via. La CPU, però, rimaneva assai limitata dai sempre troppo lenti dispositivi di I/O.

Per eliminare questo spreco di tempo per la CPU, si è pensato negli anni '60 di far lavorare la CPU anche mentre aspetta il completamento di un'operazione di I/O. Ad esempio, se il programma 1 aveva bisogno di un input, la CPU non avrebbe più aspettato inutilmente ma avrebbe iniziato a eseguire il programma 2. Questo aumentava molto l'efficienza, ma anche il carico sul sistema operativo, che ora doveva isolare la memoria di più programmi e doveva gestire lo switch tra un programma e l'altro della CPU.

Negli anni '70 si diffondono i sistemi di **timesharing** (come UNIX, nati già negli anni '60 con CTSS e Multics): la CPU, tramite un clock interno, cicla tra i diversi utenti attivi che possono interagire con la macchina tramite un proprio terminale. Dato che lo switch tra i processi dei vari utenti avviene in millisecondi, all'utente sembra di avere la macchina tutta per sé, potendo quindi usare una sola CPU in contemporanea tra più persone senza alcun delay notabile. Questo crea ancora più pressioni sul sistema operativo che deve ora affrontare molti nuovi problemi.

Dagli anni '80 ad oggi abbiamo visto sempre più sistemi: negli anni '80 arrivano i PC (Personal Computers), con però diversi problemi di sicurezza, come MS-DOS, mentre dagli anni '90 ad oggi sono arrivati i sistemi operativi che tutti conosciamo, come Windows o macOS. Per i telefoni vi sono invece iOS e Android, che prioritizzano l'efficienza elettrica e la sicurezza al resto.

1.2.2 Astrazioni

Definizione 1.14 – Astrazione

Un'**astrazione** è una vista semplificata di un sistema complesso che espone solo alcuni dettagli mentre nasconde le meccaniche più complesse.

Definizione 1.15 – Processo

L'astrazione più importante del sistema operativo è il **processo**, che è un'**istanza di un programma**.

Per capire bene cos'è il processo, basti pensare che il programma è il codice statico che si trova sulla memoria secondaria che noi scarichiamo, mentre un processo rappresenta il programma in esecuzione in un determinato momento.

Definizione 1.16 – Thread

Dentro un processo possono esserci dei **thread**, che sono la più piccola unità di esecuzione gestibile in modo autonomo dal SO.

Tutti i thread di uno stesso processo hanno lo stesso spazio di memoria e gli stessi file, ma ogni thread ha un proprio *PC* (dato che ogni thread può eseguire azioni indipendentemente dagli altri, ha bisogno di un proprio Program Counter che gli dice quali istruzioni eseguire), stack e un registro di stato.

Astrazione della memoria principale

La nostra RAM fisica è una griglia di celle: per gestire meglio questo spazio, il sistema operativo la astrae in uno **spazio di indirizzamento**. Ogni processo ha un range di indirizzi che va da 0 a $2^N - 1$ in base al sistema (nei sistemi a 64 bit si arriva fino a 2^{64} indirizzi diversi). Ogni indirizzo virtuale viene poi mappato ad un indirizzo fisico unico. Due processi A e B possono avere quindi lo stesso indirizzo di memoria virtuale 0x1000 (nel proprio spazio di indirizzamento) mappato a due indirizzi fisici differenti (rispettivamente 0x7fff000 e 0x8aaa000, ad esempio).

Questa gestione della memoria è tutta opera del sistema operativo, e viene fatta per tre motivi:

- **Facilità d'uso**: è molto più facile assumere che il primo indirizzo in memoria sia 0 e non un indirizzo fisico reale
- **Isolazione**: in questo modo se un programma sbaglia qualche reference e crasha, crasha solo lui e non tutto il sistema
- **Efficienza**: il sistema operativo carica solo le parti attualmente utili del processo, così da risparmiare risorse

Astrazione della memoria secondaria

Le memorie secondarie operano su blocchi fisici e settori. Per rendere questi blocchi di dati più comodi il sistema operativo li astrae in **file**, ossia degli array di byte a cui viene dato un nome e che possono essere letti o scritti. Per organizzare questi file vi sono poi le **cartelle**, ossia dei file speciali che contengono file o altre cartelle. Si crea dunque un *albero gerarchico delle cartelle*, dove se una cartella ne contiene un'altra significa che si trova nella gerarchia a un livello più in alto. Sarà poi il sistema operativo a trasformare i percorsi come `/home/Utente/file.txt` nei blocchi fisici del disco dove si trovano i dati del file.

1.2.3 Dual-Mode

Il sistema operativo, essendo solo software, ha bisogno di un supporto hardware per evitare che l'utente possa toccare sezioni del computer che non lo riguardano. Per questo motivo, le CPU moderne hanno due diverse modalità operative:

- **modalità utente** (non privilegiata), usata per eseguire le applicazioni dell'utente e che non ha accesso a tutto
- **modalità kernel** (privilegiata), usata per eseguire codice del sistema operativo e che ha accesso completo a tutte le istruzioni della CPU

Nelle architetture x86 in realtà oltre alla modalità utente e alla modalità kernel vi sono due modalità intermedie per i driver dei dispositivi. Questi quattro livelli sono chiamati **anelli**, ma nella pratica i SO moderni utilizzano solo l'anello 0 (modalità kernel) e l'anello 3 (modalità utente), spostando i driver o in uno o nell'altro in base alla situazione. Questo rende i sistemi operativi più **semplici e chiari**, e soprattutto li rende più **accessibili** a diverse CPU, che ad oggi supportano solo l'utilizzo di due livelli.
Per sapere in che modalità ci troviamo, la CPU usa un **bit modalità** che si trova in un registro di stato, e che vale **0** per la **modalità kernel** e **1** per la **modalità utente**. Quando si vuole eseguire un'istruzione, la CPU controlla prima se nella modalità attuale questa istruzione è eseguibile o meno.

Definizione 1.17 – Istruzioni

Le **istruzioni privilegiate** sono le istruzioni che la CPU può eseguire solo se si trova nella modalità kernel. Alcune istruzioni privilegiate sono ad esempio lo spegnimento della CPU, oppure modificare il puntatore della MMU o mandare segnali a dei registri hardware.
Le **istruzioni non privilegiate** sono invece le istruzioni che la CPU può eseguire sempre, anche in modalità utente. Alcune istruzioni non privilegiate sono ad esempio le istruzioni aritmetiche (`add,sub,mult,div`) o le istruzioni logiche (`and,or,xor`).

Se ad esempio la CPU in modalità utente decodifica un'istruzione privilegiata, la CPU non la esegue e l'hardware triggera un **GPF** (General Protection Fault) o un'eccezione. La CPU switcha automaticamente alla modalità kernel e passa al gestore delle eccezioni del kernel, che termina il processo.

Per cambiare dalla modalità utente a quella kernel ci sono solo 3 modi:

- **Chiamate di sistema / traps**: l'utente chiede al sistema operativo di eseguire per lui un'istruzione privilegiata
- **Eccezioni**: il programma dell'utente fa qualcosa di illegale

- **Interruzioni hardware**: un dispositivo esterno manda un segnale di interruzione

Per cambiare da quella kernel a quella utente invece il kernel deve prima ripristinare lo stato salvato del processo, per poi eseguire un'istruzione speciale (`iret/sysret`) che compie in modo *atomico* (ossia tutte insieme, senza poter essere interrotte a metà) le seguenti operazioni:

1° Ripristina il PC
2° Imposta il bit di modalità a 1
3° Riprende le istruzioni del programma

1.2.4 Chiamate di sistema

Un programma per funzionare ha però bisogno di eseguire delle istruzioni privilegiate (altrimenti non potrebbe salvare nulla su disco o leggere dalla tastiera, ad esempio). Per questo nascono le **chiamate di sistema**.

Definizione 1.18 – Chiamate di sistema

Le **chiamate di sistema** (**system calls**, syscalls o traps) sono delle richieste che le applicazioni fanno al kernel per eseguire istruzioni privilegiate.

A differenza delle chiamate di libreria, che si possono eseguire interamente in modalità utente e sono molto veloci, le chiamate di sistema sono significativamente più lente perché coinvolgono anche il kernel. Esistono, però, delle chiamate di libreria dette **wrappers**, le quali al loro interno eseguono una chiamata di sistema. Spesso nel codice non si utilizzano le system calls in modo diretto ma dei wrappers, così da non dover scrivere in assembly (linguaggio delle chiamate di sistema).

Quando ad esempio un'applicazione chiama una funzione `read()`, succedono diverse cose:

1° Il wrapper copia gli argomenti negli specifici registri della CPU
2° Il numero identificativo della chiamata di sistema viene inserito in `%rax`
3° Il wrapper esegue la syscall
4° La CPU cambia la modalità in modalità kernel e va all'indirizzo del gestore centrale delle syscall, ossia il punto di ingresso unico per tutte le syscall, che usa il numero in `%rax` come indice nella tabella delle syscall per chiamare la funzione giusta
5° Il kernel valida gli argomenti, esegue la syscall, mette il valore di ritorno in `%rax` ed esegue `sysret`, che permette di ritornare rapidamente alla modalità utente

Il wrapper ha tre metodi per poter passare i parametri al kernel:

- **Registri**

- **Puntatore stack**: se i registri sono pieni si passa a questi solo un puntatore che punta ad un blocco di memoria
- **Stack utente**: gli argomenti sono passati allo stack utente

La regola numero uno riguardo alle chiamate di sistema è quella di **non fidarsi mai dell'input dell'utente**. Infatti, prima di far partire una syscall il kernel deve sempre validare i parametri della chiamata. Deve controllare se i puntatori puntano a spazi di memoria validi o se il processo ha i permessi necessari per fare quella determinata azione ad esempio.

Perché le chiamate di sistema sono così costose per il computer?

Abbiamo visto come le chiamate di sistema siano decisamente più lente delle chiamate di libreria. Questo succede per diversi motivi:

- Bisogna passare dalla modalità utente alla modalità kernel e nuovamente alla modalità utente
- Bisogna salvare tutti i parametri della chiamata nei registri o nei vari stack, e se vi sono puntatori, il kernel deve validarli e copiare i dati tra spazio utente e spazio kernel; se poi la memoria puntata è stata swappata su disco, si aggiunge anche un page fault (vedremo queste operazioni di swapping più avanti, nel capitolo della virtualizzazione), rallentando di molto l'operazione
- Bisogna pulire la cache e i registri ogni volta così da averli pronti per le prossime chiamate

Per questo si cerca di limitare la frequenza delle system calls tramite un processo che prende il nome di **buffering**. In poche parole, si alloca una zona di memoria chiamata **buffer** dove si accumulano i dati destinati a una syscall. Quando il buffer è pieno, si fa una singola chiamata di sistema con tutti i dati. Ad esempio, se si vuole eseguire l'istruzione `printf` 100 volte per mostrare un carattere alla volta, il computer non fa 100 syscall ma una sola con tutti e 100 i caratteri, riducendo così di 100 volte il numero di syscall, e quindi il loro overhead.

1.2.5 Interruzioni, eccezioni e trappole

Durante l'esecuzione di un programma possono sbucare degli eventi improvvisi che richiedano immediata attenzione. Un utente che clicca un tasto sulla tastiera, un programma che prova a dividere per 0, o un'applicazione che richiede di leggere informazioni sul disco sono eventi di questo tipo. Questi eventi si dividono in tre tipi:

1) **Traps**, provenienti da un software, intenzionali e sincrone, ossia generate dall'istruzione che la CPU stava effettuando (come la richiesta di una lettura sul disco)

2) **Eccezioni**, condizioni anomale rilevate dalla CPU durante l'esecuzione di un'istruzione (non sempre errori veri e propri, come nel caso di un page fault), non intenzionali e sincrone (come una divisione per 0)
3) **Interruzioni**, provenienti dall'hardware, non intenzionali e non sincrone (come un utente che clicca un pulsante della tastiera)

Per gestirli, il sistema operativo usa un meccanismo chiamato **Processor Interrupts**, che gestisce ogni tipo di evento in modo diverso:

1) Le **traps** sarebbero le syscalls che abbiamo già visto: la CPU completa l'istruzione, passa alla modalità kernel, esegue la syscall, torna alla modalità utente e passa alla prossima istruzione.
2) Le **eccezioni** sono generate dalla CPU quando trova un errore, e si dividono in due tipi:
   - **faults**, che sono potenzialmente correggibili
   - **aborts**, che sono irrecuperabili e fanno crashare il processo
3) Le **interruzioni** sono segnali mandati alla CPU da dispositivi hardware esterni. La CPU finisce l'istruzione corrente, gestisce l'interruzione, e passa alla prossima istruzione.
   N.B. Dato che i dispositivi esterni non possono comunicare direttamente con la CPU, devono far richiesta a un controller chiamato APIC, che fa da intermediario tra la CPU e l'hardware.

Ogni evento è identificato da un **vector number** che va da 0 a 255, che fa da indice per l'evento che viene inserito in una tabella, di cui esistono due diversi tipi:

- **IVT** (Interrupt Vector Table)
- **IDT** (Interrupt Descriptor Table), dove ogni voce non è un puntatore semplice ma un gate descriptor che include anche dei metadati. Ogni gate descriptor ha un *gate type*, di cui esistono due tipi:
  - *Interrupt Gate*, che forza `IF = 0` bloccando quindi altre interruzioni esterne, dando priorità all'evento corrente
  - *Trap Gate*, che invece lascia invariato il flag `IF`

  In questa tabella, i vettori 0-31 sono riservati alle eccezioni della CPU, mentre i vettori 32-255 sono per le traps o per le interruzioni.

Cosa succede quando arriva un'interruzione?

Le interruzioni sono più particolari rispetto alle traps e alle eccezioni. Queste arrivano da input esterni e dunque sono imprevedibili, per questo esistono delle azioni automatiche che la CPU esegue quando arriva un'interruzione per gestirla:

1° **Finisce** l'istruzione corrente

2° **Salva** i registri correnti, il PC e lo Stack Pointer
3° **Passa** alla modalità kernel
4° **Prende** il vettore dell'interruzione tramite l'Interrupt Controller
5° **Esegue** l'ISR, che inizia un'altra serie di passaggi:
    1° **Salva** tutti i registri che la CPU non aveva già salvato
    2° **Gestisce** l'interruzione
    3° **Invia** il segnale EOI (End of Interrupt) all'APIC per confermare che l'interruzione è stata gestita
    4° **Ripristina** i registri
    5° **Esegue** l'istruzione assembly `iret` per tornare al programma che stava eseguendo.

A volte, però, la CPU si ritrova a eseguire codice che non deve assolutamente essere interrotto. In questo caso può usare delle istruzioni speciali come l'istruzione **`CLI`**, che **blocca** tutte le interruzioni esterne, per poi eseguire l'istruzione **`STI`** che invece fa di nuovo **passare** le interruzioni. Però non tutte le interruzioni sono ignorabili, e per questo ne esistono due tipi:

- **Interruzioni mascherabili**, che sono interruzioni standard (come il cliccare un tasto di una tastiera), che possono essere ignorate con il comando `CLI`
- **Interruzioni non mascherabili** (**NMI**, Non Maskable Interrupts), che sono interruzioni con alta priorità (come un avviso di interruzione della corrente), che non possono essere ignorate con il comando `CLI`

Definizione 1.19 – Timer

Il **timer** è uno dei supporti hardware più importanti. Questo permette di interrompere la CPU in modo periodico, così che ogni tot di tempo il sistema operativo controlli se il processo attuale dovrebbe essere fermato o meno, evitando i loop infiniti.

1.2.6 Protezione della memoria

Per isolare i processi è necessaria una protezione a livello hardware. Il metodo più semplice è lo schema **base e limiti**. In questo schema vi sono due registri fondamentali:

- **Registro base**, che contiene l'indirizzo fisico da cui inizia in memoria lo spazio d'indirizzamento del processo
- **Registro dei limiti**, che contiene il limite massimo dell'intervallo di memoria allocato

Dunque la CPU, per controllare che un indirizzo di memoria sia nel range corretto, deve controllare che l'indirizzo virtuale sia ≥ 0 e che rientri nei limiti.

I sistemi moderni utilizzano invece la memoria virtuale con la **paginazione** (Paging) invece del semplice schema base e limiti. Entra qui in gioco un componente hardware chiamato MMU che si trova nella CPU che traduce gli indirizzi virtuali in fisici al volo usando le **tabelle delle pagine**. Ogni riga della tabella ha poi diversi flag di autorizzazione: esiste il bit Read/Write, che se impostato a 0 indica che tale segmento di memoria può solo essere letto. C'è poi il bit User/Supervisor, che se impostato a 0 indica che solo in modalità kernel si può accedere a queste informazioni, e vi è poi il bit No-Execute che impedisce l'esecuzione di codice da questa pagina. Tutti i problemi riguardanti la gestione della memoria sono delle **Page Fault Exception**. Quando una PFE si verifica, possono esserci due possibili risposte dal kernel:

- La richiesta è valida, ma la pagina **non è presente in RAM**. Dunque la richiesta era legittima, ma bisognerà prima caricare i dati
- La richiesta non è valida, dunque **il processo viene terminato**

DMA Controller e I/O Controller

Abbiamo già parlato nelle prime pagine del DMA Controller, un componente hardware in grado di leggere e scrivere dati sulla RAM bypassando la CPU. Questo introduce diversi rischi di sicurezza, e perciò i registri del DMA Controller sono mappati come privilegiati, così che l'utente non possa scriverci sopra.

Un altro problema del DMA Controller è che questo comunica con la memoria tramite indirizzi fisici, mentre i programmi usano indirizzi virtuali. È quindi necessario l'intervento del sistema operativo per tradurre ogni volta gli indirizzi virtuali in fisici e passarli ai registri del DMA.

Riguardo invece i controller di input e output, abbiamo già visto che in base al dispositivo dobbiamo comunicare usando o il PMIO o l'MMIO.
Nel caso del PMIO, la CPU usa semplicemente delle istruzioni privilegiate come `in` e `out`.
Nel caso dell'MMIO, i registri fisici del Device Controller sono mappati a speciali indirizzi fisici. Questi si trovano su delle pagine che sono marcate come Supervisor Only così da evitare l'intervento dell'utente se non tramite syscall.

1.3 Introduzione a C

Definizione 1.20 – C

C è il linguaggio della programmazione di sistema. Viene usato per i sistemi operativi, i sistemi embedded, i compilatori, gli interpreti e i database. È così usato perché permette di avere **controllo diretto** sulla memoria, il che lo rende potentissimo ma anche molto pericoloso.

Definizione 1.21 – Variabile

Una **variabile** è uno **spazio di memoria con un nome** che contiene un valore di un tipo specifico. Ogni variabile si divide in 4 attributi: nome, tipo, valore e indirizzo.

Noi **dichiariamo** una nuova variabile quando ne annunciamo l'esistenza definendone il nome e il tipo, ma la variabile è **definita** quando viene allocato uno spazio di memoria a lei dedicato. Spesso queste operazioni avvengono in contemporanea, ma dato che non è sempre così è importante sapere che sono due eventi differenti.

```
extern int variable; // Variabile dichiarata, ma nessuna memoria allocata, tramite
    extern
int variable = 0; // Variabile definita
int variable_2; // Variabile dichiarata e definita
```

Una variabile è poi detta **inizializzata** quando definiamo il suo valore. Se una variabile locale non viene inizializzata, avrà dei valori non prevedibili provenienti da allocazioni precedenti (le variabili globali e statiche non inizializzate valgono invece 0). Per questo, è bene dichiarare e inizializzare una variabile nello stesso momento.

Proprietà 1.7 – Scope di una variabile

Lo **scope** di una variabile determina dove nel codice quella variabile è leggibile. Si dividono per questo 3 diversi tipi di variabili:

- **Variabili locali**: vivono nello stack e sono distrutte quando il loro scope finisce (ad esempio una variabile definita in una funzione esiste solo lì, è accessibile solo al suo interno e muore quando questa finisce)
- **Variabili globali**: vivono per l'intera durata del programma e sono definite fuori da qualsiasi funzione. Sono accessibili da qualsiasi funzione
- **Variabili statiche**: in questo caso dividiamo ancora in
  - **Variabili statiche locali**: sono inizializzate all'inizio del programma e vivono per tutta la durata del programma, ma sono chiamabili solo all'interno della funzione dove sono create
  - **Variabili statiche globali**: sono variabili globali accessibili solo all'interno del file .c dove sono create. Questo di norma non succede con le variabili globali, che sono accessibili da qualsiasi altro file .c del progetto

Una dichiarazione interna può comunque coprirne una esterna.

```
int x = 100;
int main(void){
```

```
    int x = 200;
}
```

In questo esempio, nella funzione `main` la variabile `x` diventa una nuova variabile che copre quella globale in quella determinata funzione.

Esistono poi due importanti keyword riguardanti le variabili che non sono molto utilizzate ma che sono importanti da conoscere:

- **`register`**: consiglia al compilatore di salvare questa variabile nei registri della CPU
- **`volatile`**: dice al compilatore di prendere la variabile non in cache ma in RAM

1.3.1 I tipi in C

C è un linguaggio **staticamente tipato**, dove quindi ogni variabile ha un tipo preciso conosciuto già a compiletime che non può cambiare. Questi tipi servono al compilatore per determinare quanti byte allocare ad ogni variabile, come leggere/scrivere quei byte e quali operazioni sono consentite.

Definizione 1.22 – `char`

`char` è il tipo con meno spazio di memoria disponibile in C: solo 1 byte. Questo rappresenta un qualsiasi carattere della tabella ASCII, che è una tabella dove ad ogni indice da 0 a 127 corrisponde un simbolo. Si può inizializzare usando direttamente il carattere che si vuole, scrivendo `char c = 'A'`, oppure scrivendo l'indice della tabella ASCII che indica il carattere che si vuole, scrivendo ad esempio `char c = 65` (65 in ASCII indica proprio il simbolo 'A').

Definizione 1.23 – Tipi interi

Il `char` è solo uno dei tanti tipi che fanno parte della **famiglia dei tipi interi**. Oltre a lui troviamo:

- **`short`**, con almeno 2 byte di memoria
- **`int`**, con almeno 2 byte di memoria (tipicamente 4 byte)
- **`long`**, con almeno 4 byte di memoria (su Linux/macOS sono 8 byte)
- **`long long`**, con almeno 8 byte di memoria

Per controllare la dimensione in byte di un tipo esiste comunque un operatore chiamato **`sizeof`** che a compiletime ritorna il numero di byte destinato ad una variabile qualsiasi di quel tipo su

quel sistema operativo. Ad esempio, `sizeof(long)` su Windows restituirà 4, mentre su Linux o macOS 8.

Proprietà 1.8 – Signed vs Unsigned

Tutti i tipi interi possono poi essere **signed** (default, tranne il `char`, il cui segno dipende dal compilatore) o **unsigned**. Utilizzando un tipo unsigned si perdono tutti i numeri negativi ma il range dei numeri positivi raddoppia. Ad esempio, un `char` signed ha un range da -128 a 127, ma un `char` unsigned ha un range da 0 a 255.
N.B. In una `printf` (funzione per mandare a schermo un valore), bisogna usare `%d` per i tipi signed e `%u` per gli unsigned.

Riguardo all'overflow, i tipi signed e unsigned si comportano in modo diverso:

- gli unsigned **ricominciano da 0**, seguendo le regole dell'aritmetica modulare
- i signed **non hanno un comportamento stabilito**, il che lo rende molto pericoloso

Definizione 1.24 – Tipi floating-point

I tipi **floating point** (a virgola mobile) sono quei tipi usati per rappresentare i numeri decimali. Seguono tutti lo standard IEEE 754 e sono tre:

- **`float`**, con 4 byte di memoria e una precisione di 7 cifre decimali
- **`double`**, con 8 byte di memoria e una precisione di 15 cifre decimali
- **`long double`**, con almeno 10 byte e fino a 16 byte e una precisione di almeno 18 cifre decimali

Definizione 1.25 – `bool`

Il **`bool`** è l'unico tipo booleano presente in C. Questo può assumere solo due valori: **true** (1) o **false** (0). Occupa perciò un solo byte di memoria (non un singolo bit, dato che l'unità minima di memoria indirizzabile per le architetture moderne è un byte).
N.B. È possibile assegnare a una variabile booleana un qualsiasi valore, che sarà sempre trasformato in 1 se diverso da 0.

Proprietà 1.9 – Type casting

In C è supportato sia il casting **implicito** che quello **esplicito**. Il casting implicito è automatico: avviene nelle espressioni, dove un valore viene convertito nel tipo più ampio (ad esempio da `short` a `int` o da `int` a `double`), ma anche nelle assegnazioni, dove può succedere il contrario con perdita di informazione (ad esempio `int i = 3.7` salva 3). Il casting esplicito invece va scritto dal programmatore e si può fare tra i tipi scalari (numeri e puntatori, ma non ad esempio tra `struct`). Se si passa da un tipo più preciso a uno meno, è possibile che perdere delle informazioni.

Ad esempio, se poniamo `int i = 300` e poi `double j = i`, stiamo effettuando un casting implicito da `int` a `double`. Invece se scriviamo `double pi = 3.14` e poi `int ipi = (int) pi`, stiamo effettuando un casting esplicito da `double` a `int`, che troncherà il valore di `pi` a 3.

Il casting esplicito a volte è fondamentale per avere risultati corretti. Ad esempio, prese due variabili `int a = 3` e `int b = 2`, se scriviamo `double c = a / b` non otterremo 1.5, bensì 1. Infatti, in C **una divisione tra interi restituisce un intero**. Dunque è necessario un casting esplicito su `a` o su `b` per trasformarli in `double` e rendere così la divisione una divisione tra `double` e `int`, che restituisce un `double`. Scriveremo quindi **`c = (double) a / b`**.

Riguardo ai tipi esiste anche una keyword per dare loro un nome alternativo (alias): il **`typedef`**. Scrivendo all'inizio del programma `typedef unsigned int uint_t` sarà possibile scrivere in tutto il programma `uint_t` al posto di `unsigned int`.

1.3.2 Array

Definizione 1.26 – Array

Un **array** è una **sequenza di elementi dello stesso tipo**.

Proprietà 1.10 – Proprietà degli array

Tutti gli array hanno le seguenti proprietà:

1) **Dimensione fissa**
2) Il primo elemento **indicizzato a 0**
3) Elementi **salvati vicini in memoria** (in un array di `int`, se il primo elemento si trova all'indirizzo di memoria x il secondo si troverà all'indirizzo x + `sizeof(int)`, tipicamente x + 4 byte)

Un array si indica scrivendo "tipo + nome + [lunghezza]". Ad esempio, `int array_interi[5]` è un array di `int` lungo 5. Per prendere un elemento dell'array bisogna scrivere `array_interi[x]`,

dove x indica l'indice dell'elemento. È importante che $0 \leq x <$ lunghezza dell'array, altrimenti avremo non un errore di compilazione ma un comportamento indefinito: il programma potrà leggere un valore sulla memoria che noi non conosciamo o anche andare in crash (segmentation fault).

Preso un array, vi sono poi diversi modi per inizializzarlo. Il modo più semplice è scrivere ad esempio `int array_interi[5] = {232,23,2,1,1}`. In questo caso, la lunghezza dell'array è anche omissibile, dato che è calcolabile dal compilatore. Se non la si omette, si può anche inizializzare una sola parte dell'array, scrivendo ad esempio `int array_interi[5] = {232,23}`. Si può poi anche inizializzare un array scegliendo i valori in corrispondenza di alcuni indici, scrivendo ad esempio `int array_interi[5] = {[0] = 232, [3] = 1}`. Se si vuole si può anche inizializzare un array con tutti gli elementi a 0, scrivendo `int array_interi[5] = {0}` o `int array_interi[5] = {}`. Infine, un array di `char` si può inizializzare velocemente scrivendo ad esempio `char array_char[] = "Ciao"`, che equivale a scrivere `char array_char[] = {'C', 'i', 'a', 'o', '\0'}` (aggiunge un elemento in più pari a \0, rendendo l'array di 1 più lungo di quanto ci aspetteremmo).

Ovviamente poi è possibile anche modificare il valore in corrispondenza di un indice x dell'array, scrivendo ad esempio `array_interi[2] = 7`.

Proprietà 1.11 – Iterare un array

Iterare un array significa scorrere l'array dal primo all'ultimo valore. Per farlo si usano i **`for`**, dei loop che creano una variabile (solitamente `i`) dove viene salvato un valore, che aumenta ad ogni giro di un quantitativo scelto fino a quando una condizione non è più rispettata.

```
int array_interi[10] = {9,8,7,6,5,4,3,2,1,0};
int sum = 0;
for (int i = 0; i < 10; i++){ // For che compie 10 iterazioni
    sum += array_interi[i];
}
printf("Sum = %d\n", sum); // %d è usato per mandare in output un numero int signed,
    mentre \n è il carattere per andare a capo
```

Definizione 1.27 – Variable-Length Array

I **Variable-Length Arrays** (VLAs) sono degli array di cui non conosciamo la dimensione a compiletime ma solo a runtime. Sono array di dimensione `n` dove `n` rappresenta un valore in input che non possiamo conoscere a priori.

```c
void print_squares(int n){
    int squares[n];
    for (int i = 0; i < n; i++){
        squares[i] = i*i;
    }
}
int main(void){
    print_squares(6);
    return 0;
}
```

In questo caso l'array `squares` è un VLA, dato che la dimensione dipende dal parametro della funzione `print_squares(int n)`.

Definizione 1.28 – Array 2D

Fino ad ora abbiamo parlato di array a una dimensione, ma esistono anche **array a due dimensioni** (array 2D), ossia array di array.

Un array 2D si indica scrivendo "tipo + nome + [righe][colonne]". Ad esempio, `int matrix[5][3]` indica un array 2D con 5 righe e 3 colonne, ossia una matrice con 5 righe e 3 colonne. Per prendere l'elemento alla riga i e alla colonna j bisogna scrivere `matrix[i-1][j-1]` (il -1 è fondamentale perché ricordiamo che gli indici partono da 0).

Questi array sono salvati in memoria in modo lineare: sono come dei lunghi array 1D, dove tutte le righe sono una di fianco all'altra. Dunque in un array di 2 righe e 3 colonne di `int`, per accedere al quinto elemento partendo con un indirizzo x in corrispondenza del primo elemento dovremo andare all'indirizzo x + 16.

Per iterare su questi array 2D si utilizzano due for annidati, in questo modo:

```c
int matrix[2][3] = {
    {1, 7, 9},
    {8, 6, 5}
};
int sum = 0;
for (int i = 0; i < 2; i++){
    for (int j = 0; j < 3; j++){
        sum += matrix[i][j];
    }
}
printf("Sum = %d\n", sum);
```

Proprietà 1.12 – Array decay

Il passaggio di un array a una funzione (**array decay**) in C non avviene copiando interamente l'array ma **passando un puntatore al primo elemento**.

Infatti, dato che gli array sono salvati in memoria in modo lineare, basta conoscere il puntatore al primo elemento per conoscere tutti gli elementi. È importante, però, passare alla funzione anche la dimensione dell'array. Si potrebbe pensare di usare l'operatore `sizeof` per trovare la lunghezza dell'array, ma questo non è possibile. Infatti, se passiamo ad una funzione il parametro `int arr[]`, abbiamo detto che stiamo passando solo il puntatore al primo elemento, dunque `sizeof(arr)` restituisce la lunghezza del puntatore, che è di 8 byte sui sistemi a 64 bit.

Ci sono tre dichiarazioni equivalenti per scrivere come parametro di una funzione un array:

```
1 void funzione(int arr[], int n);
2 void funzione(int *arr, int n);
3 void funzione(int arr[20], int n); // Il 20 viene ignorato
```

1.3.3 Stringhe in C

Definizione 1.29 – Stringa

Una **stringa** è una **sequenza ordinata di caratteri**. In C non esiste il tipo `string`, ma una stringa è piuttosto un **array di `char`** con un **null terminator** ('\0', che abbiamo già visto in precedenza). Questo null terminator è fondamentale perché fa capire al computer quando la stringa è terminata.

Un literal di una stringa (ossia il valore puro) è salvato nella memoria read-only (.rodata). Dunque se scriviamo ad esempio `char *s = "Ciao"`, avremo una stringa non modificabile. Se invece vogliamo una **stringa modificabile**, dobbiamo istanziare un vero array e non solo un puntatore, scrivendo **`char s[] = "Ciao"`**, che dunque restituirà un array lungo 5 (ricordiamo sempre il null terminator \0).

Proprietà 1.13 – Copiare una stringa

Se si vuole copiare una stringa in C, esistono due funzioni della libreria **`<string.h>`** (libreria che contiene tutte le funzioni riguardanti le stringhe) per farlo:

- **`strcpy`**, che non controlla la lunghezza delle stringhe
- **`strncpy`**, che invece controlla la lunghezza delle stringhe

Se abbiamo una stringa `str1` lunga 10 inizializzata, e una stringa `str2` lunga 3 vuota, fare `strcpy(str2, str1)` crea un buffer overflow. Invece scrivendo `strncpy(str2, str1, sizeof(str2) - 1)` non abbiamo problemi, dato che copiamo al massimo n elementi.

Proprietà 1.14 – Misurare la lunghezza di una stringa

Per misurare la lunghezza di una stringa ci sono due possibilità:

- con l'operatore **`sizeof`**, che restituisce la dimensione in byte dell'intero array (null terminator compreso), indipendentemente da quanti caratteri contenga davvero la stringa
- con la funzione **`strlen`**, che invece conta solo i caratteri che prendono il null terminator (escluso)

È importante capire che su una stringa non inizializzata `strlen` non funziona, dato che ha bisogno del null terminator per capire la lunghezza della stringa. Invece `sizeof` restituisce sempre la dimensione dell'array, anche se questo non è inizializzato (su un `char *`, però, restituisce la dimensione del puntatore).

Esiste un pattern per copiare una stringa in un'altra stando attenti però alla lunghezza delle due stringhe:

```
char str[] = "Hello";
char buf[5];
// Pattern
strncpy(buf, str, sizeof(buf) - 1); // Lasciamo l'ultimo elemento libero
buf[sizeof(buf) - 1] = '\0'; // Aggiungiamo il null terminator
```

Proprietà 1.15 – Comparare due stringhe

Per comparare due stringhe `a` e `b` non bisogna scrivere `a == b`. Infatti, in questo modo staremmo comparando i puntatori ai loro rispettivi primi elementi e non le stringhe stesse. Per farlo correttamente si deve usare la funzione **`strcmp`**, scrivendo quindi `strcmp(a,b)`, che restituisce un valore **negativo** se `a` viene prima di `b` alfabeticamente, **0** se sono uguali e un valore **positivo** se `a` viene dopo di `b`.

Proprietà 1.16 – Comporre una stringa

Per comporre una stringa possiamo usare due funzioni:

- **`sprintf`**, che non controlla la lunghezza della stringa
- **`snprintf`**, che invece controlla la lunghezza della stringa

Ricordano molto le funzioni viste in Proprietà 1.13.

Se ad esempio abbiamo 3 variabili `giorno`, `mese` e `anno`, e vogliamo unirle in una stringa formando la scritta giorno-mese-anno, possiamo usare queste due funzioni. Se però la nostra stringa dovesse essere troppo corta, il programma continuerebbe a scrivere anche fuori dalla stringa, creando un buffer overflow. Per evitare questo problema si preferisce usare `snprintf`, scrivendo `snprintf(str1, sizeof(str1), "%02d-%02d-%04d", giorno, mese, anno)`.

1.3.4 Puntatori

Definizione 1.30 – Puntatore

Un **puntatore** è una variabile che ha come valore un **indirizzo di memoria**. Se vogliamo indicare che una variabile è un puntatore dobbiamo precedere il nome di questa con un `*`. Esistono inoltre anche i *puntatori dei puntatori*, che si indicano con due `*`.

Ad esempio, `int *a` è un puntatore a un intero, `double *b` è un puntatore a un double e `int **c` è un puntatore a un puntatore a un intero. È importante capire quindi che **il tipo di un puntatore conta**: per salvare l'indirizzo di un `int` bisogna creare un puntatore `int`. Unica eccezione è il **`void *`**, che è un puntatore generico che può puntare a tutto. Il problema di questo puntatore è che non si può accedere al valore a cui punta direttamente: è necessario fare un casting esplicito.

```
1 int x = 42;
2 void *p = &x;
3 int y = *(int *)p; // Dobbiamo prima fare il casting e dire che p è un puntatore a un
      int
```

Per liberare la memoria a cui punta un puntatore esiste la funzione **`free`**. Ad esempio, se abbiamo un puntatore `int *p = malloc(sizeof(int))`, basterà scrivere `free(p)`. Attenzione: `free` si usa solo su memoria allocata nell'heap con `malloc`, `calloc` o `realloc`, che vedremo più avanti.

I puntatori permettono di:

- simulare il passaggio di variabili **by reference**, permettendo di modificare queste variabili all'interno di funzioni esterne

- lavorare con la **memoria dinamica**
- usare diverse **strutture dati**, come le linked list
- lavorare **efficientemente con gli array**, non dovendoli copiare ogni volta che li passiamo come parametri in una funzione (come già visto nel capitolo degli array)

Per prendere l'indirizzo di una variabile si usa l'operatore **&**.

```
int x = 42;
int *p;
p = &x; // p è un puntatore che contiene l'indirizzo di x
```

In questo codice, per accedere al valore che si trova all'indirizzo salvato in `p` basta scrivere `*p`. Si può, in questo modo, anche modificare il valore di x, scrivendo ad esempio `*p = 3`.

Proposizione 1.1 – Strict aliasing

Se si hanno due puntatori che puntano alla stessa area di memoria, bisogna seguire il principio dello **strict aliasing**, che dice che non è consentito accedere al valore di un oggetto tramite un puntatore di un tipo diverso dal suo. L'eccezione principale è il `char*` (e `unsigned char*`): è sempre lecito usarlo per leggere i byte di qualsiasi oggetto. Sono ammesse anche le varianti signed/unsigned e `const` dello stesso tipo. Dunque, presa `int a = 10`, non possiamo scrivere `float *b = (float *) & a` e poi accedere al valore a cui punta `b` scrivendo ad esempio `*b`, ma possiamo scrivere `char *c = (char *) &a` e accedere poi al valore a cui punta `c` senza alcun problema.

Definizione 1.31 – Puntatore `NULL`

In C esiste anche il puntatore **`NULL`**, che è un puntatore che **non punta a nulla** (punta all'indirizzo 0). Questo serve per inizializzare i puntatori che ancora non puntano a nulla.

La keyword **`const`** con i puntatori si comporta in modo particolare: infatti si può riferire al puntatore, al valore puntato, o a entrambi.

```
int x = 10;
int y = 20;
const int *p1 = &x; // p1 può cambiare, *p1 no
p1 = &y; // Valido (cambio indirizzo a cui punto)
int * const p2 = &x; // p2 non può cambiare, *p2 si
*p2 = 99; // Valido (cambio valore a cui si riferisce il mio indirizzo)
const int * const p3 = &x; // Non può cambiare nulla
```

Quando si eseguono delle somme/sottrazioni su un puntatore, questo si muove in memoria di un **multiplo della dimensione del tipo di dato a cui punta**. Ad esempio, se `p` punta a un `int` con indirizzo x, `p-1` punterà all'indirizzo x - 4, mentre `p+10` all'indirizzo x + 40.

Sottrarre invece due puntatori che puntano a elementi dello stesso tipo nello stesso array darà un risultato non in byte ma in **numero di elementi**. Se ad esempio `p1` punta al primo elemento e `p2` punta al quinto, il risultato sarà 4, dato che ci sono 4 elementi di differenza tra il primo e il quinto. Per memorizzare queste informazioni esiste un tipo chiamato **`ptrdiff_t`** definito nella libreria **`<stddef.h>`**. Oltre a sottrarre, puntatori nello stesso array possono essere anche comparati. Ad esempio, `p1 < p2` ritorna vero in questo caso perché `p1` punta al primo elemento mentre `p2` al quinto, mentre `p1 == p2` e `p1 > p2` ritornano entrambe falso.

Grazie a queste nuove informazioni, possiamo scrivere un nuovo modo per iterare su tutti gli elementi di un array:

```
1 int arr[5] = {1,2,3,4,5};
2 int *end = arr + 5;
3 for (int *cur = arr; cur < end; cur++){ // Partiamo dall'indice dell'elemento 0 e
      aumentiamo fino al quinto elemento
4     printf("%d ", *cur);
5 }
```

Notiamo alla seconda linea di questo codice che al posto di scrivere `&arr[0]` abbiamo scritto solo `arr`. Questo non è un errore: seguendo il principio visto nella Proprietà 1.12, che dice che quando gli array sono passati a una funzione decadono, e passano solo il puntatore al primo elemento, anche qui gli array passano in automatico l'indirizzo al loro primo elemento.
Un'altra cosa interessante da notare è che **`arr[i] = *(arr + i)`** sempre.

Definizione 1.32 – Puntatore a un array

Abbiamo visto che per puntare a un array si punta al suo primo elemento. Esistono però anche dei **puntatori** che puntano direttamente a tutto l'array. In questo caso si usa la & per riferirsi all'intero array.

Ad esempio, preso l'array `int arr[5] = {1,2,3,4,5}`, possiamo istanziare il puntatore `int (*p)[5] = &arr` che punta all'intero array. In questo caso, `p+1` non avanzerà quindi di 4 byte nella memoria ma di ben 20. Questi puntatori ad array interi sono particolarmente utili negli array 2D, che ricordiamo sono array di array.

I puntatori possono poi puntare anche a delle **funzioni**. Ad esempio, il puntatore `int (*p)(int, int)` è un puntatore che può puntare a una funzione che prende in input due `int` e restituisce un `int`. Capiremo l'importanza di questi puntatori solo quando introdurremo più avanti concetti più avanzati.

1.3.5 Errori nella gestione della memoria

Gli errori di gestione della memoria sono tra i bug più critici nello sviluppo software. Questi, oltre a essere particolarmente insidiosi e difficili da trovare, sono anche molto pericolosi per la sicurezza. Vediamo alcuni dei più importanti errori di gestione della memoria da evitare assolutamente:

Definizione 1.33 – Dangling pointers

I cosiddetti **dangling pointers** (o puntatori penzolanti in italiano) sono dei puntatori che puntano a una **porzione della memoria non più valida**.

Vediamo un esempio di dangling pointer:

```c
int *get_local(void){ // Questa funzione ritorna un puntatore a un intero
    int x = 42;
    return &x; // Ritorniamo un puntatore a una variabile locale!
}
int main(void){
    int *p = get_local(); // Dangling pointer, dato che x viene subito distrutta
}
```

Definizione 1.34 – Wild pointer

Un **wild pointer** è un puntatore che **non è mai stato inizializzato**.

Se ad esempio dichiarassimo `*p` senza inizializzarlo e provassimo ad accedere al valore al suo interno commetteremmo un errore, dato che in quel momento `p` sarebbe proprio un wild pointer.

Definizione 1.35 – Buffer Overflow

Un **buffer overflow** è un errore che accade quando si **scrive oltre un array**. Lo abbiamo incontrato spesso nel capitolo degli array, ed è proprio uno dei più classici errori riguardanti la gestione della memoria.

Definizione 1.36 – Off-by-One

Un **off-by-one** error è anche questo uno degli errori più classici riguardo agli array. Accade quando si prova ad **accedere a un elemento fuori dal range di un array**.

Definizione 1.37 – Variabili non inizializzate

Accedere a variabili non inizializzate è anch'esso uno degli errori più comuni della gestione della memoria. Per evitarlo, è sempre consigliato inizializzare le variabili appena create a 0.

Definizione 1.38 – Aliasing

L'**aliasing** si verifica quando due puntatori possono fare **riferimento alla stessa area di memoria**. Se non può escluderlo, il compilatore deve essere prudente e rinunciare ad alcune ottimizzazioni, rendendo il codice più lento. Per questo, se si è sicuri che due puntatori non faranno mai riferimento alla stessa area di memoria, si può usare la keyword `restrict`, che lo promette al compilatore (se poi i puntatori si sovrappongono davvero, il comportamento è indefinito).

Per trovare errori di questo tipo si utilizzano due strumenti fondamentali:

- **Valgrind**, che rileva memory leaks, tentativi invalidi di lettura/scrittura e utilizzo di variabili prima di esser state inizializzate
- **AddressSanitizer** (**ASan**), che rileva buffer overflow, accessi a memoria già liberata (use-after-free), tentativi di liberare la memoria da un puntatore usando due volte il comando `free` e memory leak (per gli altri comportamenti indefiniti esiste invece l'**UndefinedBehaviorSanitizer**, o UBSan).

Oltre a questi strumenti, ci sono poi diversi pattern da seguire per evitare tutti questi errori in partenza.

1.3.6 Stack

Definizione 1.39 – Stack

Lo **stack** è una regione di memoria dedicata a immagazzinare:

- **variabili locali**
- **parametri delle funzioni**
- **indirizzi dove tornare appena una funzione finisce** (così che il programma sappia dove riprendere appena finisce una chiamata di una funzione)
- **registri salvati**
- **valori ritornati**

Ogni chiamata di funzione crea un proprio **stack frame**, che muore quando viene eseguito il suo `return`. In ogni stack esistono poi i puntatori **RBP** e **RSP**: il primo mantiene una reference utile ai debugger per leggere lo stack frame, mentre il secondo segna la fine dello stack (ossia il punto più basso, dato che lo stack cresce **verso il basso**).

Anche le chiamate ricorsive creano ogni volta un nuovo stack frame: per questo motivo è da evitare una ricorsione molto profonda, dato che potrebbe portare a un problema di *stack overflow*. In questi casi è infatti sempre consigliata una soluzione iterativa rispetto a quella ricorsiva.

N.B. Dato che le variabili locali muoiono quando muore lo stack dove sono salvate, non bisogna mai ritornare un puntatore a una variabile locale. Se si vogliono salvare dei dati al di fuori di una funzione bisogna usare l'**heap**, che analizzeremo più nel dettaglio tra poco.

Proprietà 1.17 – Proprietà dello stack

Tra le proprietà dello stack troviamo:

- filosofia **LIFO** (Last In First Out)
- **gestione automatica**
- **limite fissato** (tipicamente a 1-8 MB per thread)
- **veloce**

```c
int add(int a, int b){
    int result = a + b;
    return result;
}

int main(void){
    int x = add(3,7);
    return 0;
}
```

In questo caso, nello stack del main è presente solo la variabile `x`, mentre nello stack dell'add sono presenti il return address e la variabile `result`. I parametri `3` e `7` non si trovano sullo stack ma sui registri: infatti, ogni funzione ha **6 registri disponibili per salvare parametri interi** (RDI, RSI, RDX, RCX, R8, R9): solo se supera questo numero di parametri interi è costretta a salvarli nello stack.

1.3.7 Heap

Definizione 1.40 – Heap

L'**heap** è una regione di memoria destinata all'**allocazione dinamica della memoria**. Serve per mantenere dati la cui dimensione e la cui durata vitale non sono conosciute a compiletime.

Abbiamo già detto che l'heap è fondamentale per far sopravvivere quelle variabili locali anche fuori dalle proprie funzioni di appartenenza, ma ha anche molti altri scopi.

Proprietà 1.18 – Proprietà dell'heap

Tra le proprietà dell'heap troviamo:

- **gestione manuale** (bisogna allocare memoria usando **`malloc`** e liberarla usando **`free`**)
- **sopravvivenza oltre le chiamate di funzione**
- **dimensione variabile**
- **più lento dello stack**
- **condiviso tra le funzioni di uno stesso processo**

È consigliato scegliere l'heap per grandi quantità di dati di dimensione dinamica che vogliamo far vivere a lungo, mentre è consigliato l'utilizzo dello stack per dati piccoli e di vita breve.

N.B. I valori salvati nell'heap possono essere il `return` di una funzione, ma è importante che chi chiami questa funzione liberi la memoria dell'heap occupata da questo valore tramite `free`.

```
int main(void){
    int n = 10;
    // Allochiamo un array di n interi nell'heap
    int *arr = malloc(n * sizeof(int));
    if (arr == NULL){ // Bisogna sempre controllare se l'allocazione ha fallito
        return 1;
```

```
    }

    for (int i = 0; i < n; i++){
        arr[i] = i*i;
        printf("%d", arr[i]);
    }
    free(arr); // Liberiamo la memoria
    arr = NULL;
    return 0;
}
```

Oltre alla funzione `malloc` esiste anche la funzione **`calloc(n, size)`** che alloca `n` elementi ognuno di grandezza `size`. Nel codice qui sopra avremmo ad esempio potuto scrivere `calloc(n, sizeof(int))` e avrebbe funzionato ugualmente. Per ridimensionare un blocco già allocato si utilizza la funzione **`realloc(ptr, new_size)`**, dove `ptr` è il puntatore che punta al blocco in memoria nell'heap che vogliamo ridimensionare con la nuova dimensione `new_size`. Ad esempio, nell'esempio precedente avremmo potuto allungare l'array di 1 scrivendo `int* tmp = realloc(arr, (n + 1) * sizeof(int))` e `arr = tmp`, dopo aver controllato che l'allocazione sia andata a buon fine. Notiamo quindi come queste funzioni siano fondamentali per creare degli **array dinamici**. Infatti, continuando a ridimensionare i blocchi salvati nell'heap possiamo modificare uno stesso array e renderlo sempre più grande. Se però si esegue questo processo troppe volte, si crea un problema chiamato **frammentazione dell'heap**.

Proprietà 1.19 – Frammentazione dell'heap

Dopo diverse chiamate di `malloc` e `free`, l'heap inizialmente contiguo si inizia a frammentare, lasciando magari tanti piccoli blocchi da pochi MB che se sommati arrivano anche ad avere una dimensione molto importante. Tuttavia, `malloc` alloca memoria contigua, dunque non può usare più blocchi in un colpo solo. Può quindi capitare di **non poter allocare qualcosa in memoria perché non vi è un blocco contiguo abbastanza grande**, anche se in memoria lo spazio totale è superiore allo spazio richiesto.

Osservazione 1.4 – Assenza del garbage collector

In C **non è presente alcun garbage collector**. Per questo motivo, è fondamentale seguire diverse regole per mantenere ordine nel proprio programma:

- La funzione che chiama `malloc` per allocare memoria sull'heap è **proprietaria** di quella memoria, a meno che non ne trasferisca la proprietà ritornando il puntatore a quella memoria
- Se una funzione ritorna un valore allocato in memoria heap, bisogna **documentare chiaramente** che la funzione chiamante dovrà chiamare `free` su quel puntatore
- Per evitare confusione è preferibile che **chi alloca memoria la liberi anche**
- Se una funzione passa un puntatore che punta a un blocco in memoria heap a un'altra, deve essere chiaro se ne sta **passando anche la proprietà o meno**, così da decidere chiaramente chi debba chiamare `free`
- Dopo aver chiamato `free` su un puntatore, questo va **subito impostato su `NULL`**, così da evitare il bug del double-free

1.3.8 Altri errori nella gestione della memoria

Ora che abbiamo introdotto lo stack e l'heap, vediamo altri errori comuni nella gestione della memoria.

Definizione 1.41 – Memory leaks

Un **memory leak** è un classico errore nella gestione della memoria che accade quando **non si libera mai la memoria allocata nell'heap**, ossia quando non si chiama la funzione `free`. Valgrind è in grado di trovare questo errore, puntando alla linea dove l'allocazione avviene.

Definizione 1.42 – Bug riguardanti la funzione `free`

Abbiamo già nominato il bug del **double-free**, che consiste nel chiamare due volte la funzione `free` sullo stesso puntatore. Un altro bug comune riguardo alla funzione `free` è quello di usare un puntatore dopo la chiamata di `free`.

Definizione 1.43 – Heap Buffer Overflow

Oltre allo stack buffer overflow esiste anche l'**heap buffer overflow**, che consiste nello scrivere oltre il limite massimo di memoria heap allocata. Questo provoca un **comportamento indefinito** (UB, Undefined Behavior) da parte della macchina: lo standard C smette di imporre qualsiasi regola su cosa debba accadere e il compilatore, assumendo che tale situazione non si verifichi, non inserisce alcun controllo: il programma corrompe silenziosamente i dati o provoca crash casuali, anche molto lontani da dove si è verificato l'errore.

A volte si prova col codice a evitare questi UB, ma questo non sempre è sufficiente.

```c
int check_overflow(int x){
    if (x+1 < x){ // Non funziona!!
        return 1;
    }
    return 0;
}
int check_overflow2(int x){
    if (x == INT_MAX){ // Funziona!!
        return 1;
    }
    return 0;
}
```

Ad esempio, qui abbiamo due funzioni che provano a controllare l'overflow di un `signed int`. Solo la seconda funzione, però, è corretta. Questo perché la prima fa un controllo che per il compilatore è inutile: infatti, il compilatore **dà per scontato che gli UB non accadano mai**, e quindi dà per scontato che `x+1 < x` sia sempre falso, non controllandolo nemmeno. Dunque il ramo if viene cancellato direttamente, e così la prima funzione restituisce sempre 0.

1.3.9 Fasi della compilazione

La compilazione di un file `.c` si divide in 4 fasi precise:

1° **Pre-processore**
2° **Compilatore**
3° **Assemblatore**
4° **Linker**

Ogni file `.c` in uno stesso progetto è compilato come una **translation unit** indipendente. Ognuno di questi è pre-processato e compilato in modo indipendente dagli altri, e non può quindi leggere

il codice interno degli altri `.c`. In questo modo, se dobbiamo modificare solo un file `.c`, non è necessario pre-processare e compilare di nuovo tutti i file ma solo quel singolo `.c`.

Pre-processore

Definizione 1.44 – Pre-processore

Il **pre-processore** è la fase antecedente alla vera compilazione. Si occupa di gestire tutte le righe che iniziano con #, prendendo in input il file con estensione `.c` o `.h` e restituendo un file con lo stesso nome ed estensione `.i`.

N.B. I file `.h` sono dei file con delle **dichiarazioni** condivise tra tutti i file del progetto. È importante non mettere mai delle definizioni in questi file, che causano diversi errori a linktime (durante la 4° fase, ossia la fase del Linker).

Tra le diverse righe che iniziano con # ci sono:

- **`#include`**, che serve per includere contenuti (come funzioni o variabili globali) da altri file. Il pre-processore prende questi contenuti e li copia nel nostro file.
- **`#define`**, che definisce una macro che sostituisce un valore (ad esempio, se c'è la linea `#define PI 3.14`, ogni volta che il pre-processore legge `PI` lo sostituisce con il valore `3.14`). Può definire anche una macro con parametri (ad esempio, se c'è la linea `#define MAX(a,b) ((a) > (b) ? (a) : (b))`, il pre-processore ogni volta che legge `MAX(x,y)` lo sostituisce con il testo `((x) > (y) ? (x) : (y))`, e il confronto verrà poi eseguito a runtime). È importante usare le () in questo caso perché queste *macro* definite con `#define` non rispettano l'ordine degli operatori. Ad esempio, `#define SQUARE(x) x * x` non è scritto correttamente, dato che passando come argomento 2 + 3 il risultato non sarebbe 25 ma 11 (2 + 3 * 2 + 3).
- **`#ifdef, #ifndef, #endif`**, che servono per includere del codice solo se alcune condizioni sono rispettate.

```
#define DEBUG_MODE // Se si vuole disattivare bisogna commentare questa riga
#ifdef DEBUG_MODE
    #define DBG(x) printf("%d", x)
#else
    #define DBG(x) // Non espande nulla
#endif
```

In questo caso quando chiameremo nel codice `DBG(x)`, funzionerà solo se la `DEBUG_MODE` è attiva. `ifndef` funziona come `ifdef` ma al contrario: se il nome nella condizione non esiste allora passa, altrimenti no. Queste condizioni sono fondamentali per creare gli **include guards**, che sono utili per evitare di includere due volte gli stessi contenuti.

```c
#ifndef MODULO
    #define MODULO
    void funzione(int x);
#endif
```

Creiamo quindi un blocco `#ifndef` che viene eseguito una volta e crea la macro `MODULO`. Avendola creata, il blocco `#ifndef` non sarà più eseguito, evitando così l'errore del double-include.

Oltre a tutte queste macro che abbiamo visto, esistono altre macro che il compilatore ha di default:

- **`__FILE__`**: nome del file sorgente
- **`__LINE__`**: linea di codice corrente
- **`__DATE__`**: data segnata a compiletime
- **`__TIME__`**: ora segnata a compiletime
- **`__func__`**: nome della funzione corrente (in realtà non è una macro, ma un identificatore che il compilatore definisce in automatico in ogni funzione)

A volte può essere utile leggere l'output del pre-processore per effettuare debugging. Per avviare un file `hello.c` solo nel pre-processore bisogna eseguire il comando `gcc -E hello.c -o hello.i`. Esistono poi tanti comandi per conoscere informazioni differenti: se vogliamo ad esempio contare il numero di righe dopo l'esecuzione del pre-processore basta eseguire il comando `gcc -E hello.c | wc -l`.

Compilatore

Definizione 1.45 – Compilatore

Il **compilatore** (**`cc1`**) si occupa di tradurre il codice C in assembly, prendendo in input il file `.i` e restituendo un file con lo stesso nome e con estensione `.s`, con assembly puro specifico per l'architettura del computer (x86-64, ARM64...).

Per generare codice assembly dal file `hello.c` usiamo il comando `gcc -S hello.c -O2 -o hello.s`. Per leggere questi file, è ovviamente necessaria una conoscenza minima delle istruzioni assembly. Tra le più importanti troviamo:

- **`mov dst, src`**, che copia un valore da `src` a `dst`
- **`push reg`**, che decrementa il valore del puntatore alla fine dello stack `RSP` e aggiunge in cima il valore salvato nel registro `reg`

- **`pop reg`**, che estrae il valore in cima allo stack e lo salva nel registro `reg` e incrementa il valore del puntatore alla fine dello stack `RSP`
- **`add dst, src`**, che somma il valore di `dst` con `src` e lo sovrascrive in `dst`
- **`sub dst, src`**, che sottrae il valore di `src` a `dst` e lo sovrascrive in `dst`
- **`cmp a,b`**, che calcola la differenza aritmetica tra `a` e `b` senza però modificare `a` o `b`
- **`je / jne / jl`**, che esegue un salto se due valori sono uguali / diversi / se il primo è minore
- **`call label`**, che salva l'indirizzo di ritorno sullo stack e salta all'esecuzione di un'altra funzione specificata in `label`
- **`ret`**, che prende l'indirizzo di ritorno dallo stack e riprende il flusso di esecuzione dopo la fine di una funzione
- **`lea dst [indirizzo]`**, che calcola l'indirizzo di memoria tra parentesi quadre e lo memorizza in `dst`

Ad esempio, una semplice funzione come questa

```
int max(int a, int b){
    if (a > b){
        return a;
    }
    return b;
}
```

in assembly diventa

```
max:
    cmp edi, esi # Compara a (edi) e b (esi)
    jle .L2 # Se a <= b ritorna b (salta all'etichetta .L2 definita dopo)
    mov eax, edi # eax = a
    ret
.L2:
    mov eax, esi # eax = b
    ret
```

Definizione 1.46 – Rappresentazione intermedia

I compilatori moderni utilizzano una **rappresentazione intermedia** (IR, Intermediate Representation).

Ad esempio, per i computer che utilizzano GCC (GNU Compiler Collection), si passa prima in GIMPLE, ossia una rappresentazione ad albero semplificata, poi in RTL, un linguaggio più vicino all'hardware, e solo dopo l'RTL si arriva all'assembly.

Questa rappresentazione intermedia effettua diverse operazioni di ottimizzazione. Ad esempio elimina codice morto mai eseguito, oppure rimpiazza una chiamata di una funzione con il suo codice, oppure può anche rimpiazzare delle operazioni costose con altre più semplici (ad esempio, dato che $x * 2 = x << 1$, rimpiazza tutte le moltiplicazioni con uno shift a sinistra). Esistono poi diversi livelli di ottimizzazione. In GCC abbiamo livelli da 0 a 3 e poi il livello s, che ottimizza la dimensione del file.

Assemblatore

Definizione 1.47 – Assemblatore

L'**assemblatore** (`as`) converte il codice assembly in codice macchina binario. Prende in input un file `.s` e restituisce un file con lo stesso nome e l'estensione `.o`.

Un file `.o` (Object File) è tipicamente in formato ELF (Executable and Linkable Format) ed è diviso in diverse sezioni:

- **ELF Header**, ossia il frontespizio del file che contiene i metadati fondamentali, l'architettura della CPU e la posizione delle altre sezioni
- **`.text` section**, che contiene il codice macchina vero e proprio
- **`.data` section**, che memorizza le variabili globali e statiche inizializzate a un valore diverso da 0
- **`.bss` section**, che memorizza le variabili globali e statiche non inizializzate o inizializzate a 0
- **`.rodata` section**, che contiene i dati in sola lettura, come le costanti o le stringhe letterali
- **Symbol table**, ossia la tabella dei simboli che elenca tutti gli identificatori (nomi delle variabili, delle funzioni...) definiti nel file oppure utilizzati qui ma definiti altrove
- **Relocation table**, ossia la tabella che contiene tutte le istruzioni per il linker su dove e come aggiustare gli indirizzi di memoria

Linking

Definizione 1.48 – Linker

Il **linker** (`ld`) combina tutti i file `.o` in un unico eseguibile.

Il linker crea una symbol table globale e associa ogni chiamata o utilizzo di un simbolo al punto esatto in cui quel simbolo è implementato. Dunque, se creiamo una funzione `add_numbers` nel file `utils.o` e chiamiamo questa funzione nel file `main.o` sarà il linker a collegare i due file, risolvendo tutti i riferimenti lasciati in sospeso nelle fasi precedenti. Se poi esiste un simbolo referenziato ma mai definito, avviene un *linker error*. Ricordiamo che una variabile può essere dichiarata ma non definita tramite la keyword `extern`.

Il linker inoltre risolve il problema lasciato in sospeso dall'assemblatore, ossia la **rilocazione**. Abbiamo visto come nei file `.o` esista una relocation table che contiene delle istruzioni su come aggiustare gli indirizzi di memoria. Questa esiste perché nella fase gestita dall'assemblatore non si sa ancora come e dove verranno collocate in memoria le diverse funzioni o variabili. Il linker quindi prende questi segnaposto temporanei lasciati dall'assemblatore e li sostituisce con i veri indirizzi di memoria ora conosciuti.

Nei file eseguibili ELF creati dal linker esistono le **sezioni** e i **segmenti**. È importante capire la differenza tra questi due concetti:

- Le **sezioni** servono al compilatore, al linker e al debugger e suddividono il codice in blocchi logici specifici, e sono descritte nella Section Header Table, la cui posizione è indicata nell'ELF Header
- I **segmenti** servono invece al sistema operativo per mandare in esecuzione il programma e suddividono il codice in base ai permessi di memoria necessari, e sono descritti nella Program Header Table, la cui posizione è indicata nell'ELF Header

1.3.10 Linking statico e dinamico

Definizione 1.49 – Libreria statica

Una **libreria statica** è un file **`.a`** che contiene un archivio di file oggetto `.o`.

Per creare una libreria si utilizza lo strumento **`ar`** (archiver) che raggruppa i file oggetto in un unico archivio. Ad esempio, se abbiamo due file oggetto `math.o` e `stats.o`, per raggrupparli in una libreria statica usiamo il comando `ar rcs libmymath.a math.o stats.o`, che li inserisce nell'archivio `libmymath.a`. La dicitura `rcs` indica che i file devono essere rimpiazzati (`r`) se sono già presenti nell'archivio, che bisogna creare l'archivio se ancora non esiste (`c`), e che bisogna generare un indice dei simboli all'interno dell'archivio per permettere al linker di trovare velocemente funzioni e variabili (`s`, symbol index). Se si vuole poi aprire la libreria, si può usare il comando `ar -t libmymath.a` per stampare la lista dei file oggetto o usare il comando `nm` per stampare la tabella dei simboli dell'archivio, dove sono presenti tutti i nomi delle funzioni e delle variabili referenziate al suo interno. Per collegare poi una libreria a un file `.c` si usa il comando `gcc main.c -L. -lmymath -o program`. `-L.` aggiunge la directory corrente ai percorsi di ricerca del linker dove cercare i file di libreria, mentre `-lmymath` dice al linker di cercare e

collegare la libreria `libmymath.a` (o `libmymath.so` che indica una libreria dinamica, che vedremo ora cos'è).

Definizione 1.50 – Libreria dinamica

Una **libreria dinamica** (o **condivisa**) è un file **`.so`** contenente un archivio di file oggetto `.o` caricato a runtime e condiviso tra i diversi processi. Queste sono condivise tramite il **dynamic linker**.

Quindi se sia il programma A che B hanno bisogno di accedere a questa libreria, questa verrà mappata una sola volta in memoria RAM e sarà condivisa tra il processo A e il processo B. Il processo di creazione di una libreria dinamica è molto simile a quello di una libreria statica.

È possibile anche caricare in memoria una libreria condivisa mentre è già in esecuzione, senza doverla collegare in fase di compilazione. Per farlo, è necessario includere l'header `<dlfcn.h>` ed è necessario aprire la libreria tramite la funzione `dlopen(percorso, RTLD_LAZY)`.

Bisogna usare librerie statiche per il deployment e per i sistemi embedded, mentre bisogna usare le librerie dinamiche per librerie condivise di grandi dimensioni.

Capitolo 2

Virtualizzazione

N.B. Per questo e per i prossimi due capitoli seguiremo la struttura **OSTEP**, che organizza il sistema operativo in 3 blocchi principali:

- **Virtualizzazione**
- **Concorrenza**
- **Persistenza**

Definizione 2.1 – Virtualizzazione

La **virtualizzazione** è una tecnica che crea una versione virtuale di una risorsa fisica astraendola dall'hardware e permettendo così di far girare più ambienti isolati sulla stessa macchina.

2.1 Processi e API

Abbiamo già visto nella Definizione 1.15 cos'è un processo (o job), ossia un'**istanza di un programma**. Se vogliamo però essere più precisi, un processo è l'**astrazione** che il sistema operativo crea di un programma in esecuzione, che include:

- Il codice che sta venendo eseguito
- Il valore corrente del Program Counter
- Il contenuto dei registri della CPU
- Uno spazio di indirizzi virtuale privato
- Un insieme di descrittori di file aperti (ossia una tabella di numeri dove ognuno è collegato a un file)
- Un univoco Process ID (PID)

Dunque i programmi sono file scritti in modo permanente nel disco, mentre i processi sono entità temporanee che vivono nella RAM.

Quando facciamo partire un programma, il sistema operativo esegue tre passi:

1° Legge il file binario ELF sul disco

2° Carica il codice e i dati statici nella RAM

3° Alloca lo stack e l'heap, prepara i descrittori dei file e salta al punto di start del programma, ossia alla funzione `main()`

Proprietà 2.1 – Caricare file in RAM

Ci sono poi due modalità in cui l'OS può caricare un programma in memoria RAM:

- **Eager Loading** (OS più vecchi), dove l'OS carica l'intero file binario nella RAM prima di eseguire il programma
- **Lazy Loading** (OS più nuovi), dove l'OS carica solo il codice necessario. Quando la CPU prova ad accedere a una pagina non ancora caricata, parte un *page fault* che fa partire il caricamento di tale pagina in RAM (riparleremo di page fault e paging più avanti quando parleremo della gestione della memoria)

Con la tecnica del lazy loading è molto probabile che gran parte del codice binario non venga mai caricato in RAM perché non necessario. Inoltre, ogni processo crede di avere l'intero spazio di memoria tutto per sé, nonostante questo ovviamente non sia vero. Questo fenomeno è chiamato l'**illusione della virtualizzazione della memoria**.

Proprietà 2.2 – Stati di un processo

Un processo si trova sempre in uno di questi 3 stati:

- **READY**, in attesa che la CPU lo esegua: i suoi contenuti sono salvati nel **PCB** (che vedremo tra poco nel dettaglio cos'è) e il processo è inserito nella **run queue** in attesa di essere schedulato
- **RUNNING**, durante il quale il processo monopolizza la CPU usandone le risorse e i registri
- **BLOCKED**, in attesa di un I/O o di un evento

N.B. In una CPU con 4 core, al massimo 4 processi possono veramente essere in modalità RUNNING in contemporanea.

A causa del fenomeno del time-sharing, un processo non sta mai a lungo in modalità RUNNING, ma piuttosto si muove molto tra questa modalità e la modalità READY. Inoltre, un processo BLOCKED non passa mai subito a RUNNING, ma ritorna ad essere READY: sarà poi lo scheduler a decidere quando mandarlo in RUNNING.

Definizione 2.2 – PCB

Il **PCB** (Process Control Block) è la struttura dati che il sistema operativo istanzia per ogni processo per identificarne, salvarne e tracciarne lo stato di esecuzione, la memoria e le risorse. Questo si trova nella *memoria kernel*, una speciale zona di memoria RAM destinata a tutti i componenti più importanti del kernel.

Ogni PCB salva diverse informazioni quindi, tra cui un identificativo (`pid`), lo stato del processo, un puntatore alla pagina di memoria riservata al processo e una struttura dati chiamata `trapframe` dove vengono salvati tutti i registri della CPU quando il processo viene interrotto. Il PCB è quindi fondamentale per poter effettuare il time-sharing senza perdere informazioni riguardo ai singoli processi. Il sistema operativo ha poi una struttura dati chiamata **process table** dove mantiene tutti i PCB. È qui che lo scheduler cerca il primo processo READY per mandarlo in esecuzione.

Definizione 2.3 – Gerarchia dei processi

Su UNIX i processi formano un **albero gerarchico** che parte dalla radice PID 1. In questo albero ogni processo ha un genitore, e se questo genitore muore prima dei figli, questi sono adottati direttamente dalla radice PID 1. L'unico modo per creare un nuovo processo è usare la funzione `fork()`, che vedremo meglio più avanti.

Definizione 2.4 – PID 1

PID 1 è il primo processo dello spazio utente eseguito dal kernel dopo l'accensione del computer. Sui Linux più moderni questo processo prende il nome di `systemd`, ed è considerabile l'antenato di tutti i processi. Questo esegue un loop che chiama la funzione `wait()` per trovare tutti gli eventuali processi orfani.
N.B. Non è possibile uccidere questo processo tramite comandi normali.

È grazie al PID 1 se nessun processo rimane veramente orfano.

2.1.1 Gestione dei processi in UNIX

La gestione dei processi in UNIX si basa su 5 syscall fondamentali facenti parte della stessa API:

- **`fork()`**, che crea un processo figlio clonando il chiamante
- la famiglia **`exec()`**, che rimpiazza l'immagine (ossia l'insieme di dati e codice che costituiscono il programma caricato in memoria) del processo corrente con un nuovo programma

(serve per avviare un nuovo programma partendo dal duplicato di un processo di un altro programma)

- **`wait()`/`waitpid()`**, che aspetta che un processo figlio termini
- **`exit()`**, che termina un processo
- **`getpid()`/`getppid()`**, che restituiscono rispettivamente l'identificativo univoco del processo chiamante e del processo padre che ha creato il processo chiamante

Per poter utilizzare queste funzioni sono necessari gli header **`<unistd.h>`**, **`<sys/wait.h>`** e **`<sys/types.h>`**.

Uno si potrebbe domandare a cosa serve fare tutto questo casino per creare un nuovo processo. Perché non fare come in Windows dove si creano nuovi processi senza doverne duplicare di vecchi come si fa in UNIX tramite `fork()`? Semplicemente perché, duplicando un processo già esistente, si evita di dover settare molti dei parametri che già sono stati gestiti dal processo padre. Infatti, in questo modo noi dobbiamo semplicemente cambiare alcune cose per rendere il processo adatto alle nostre esigenze per poi eseguire il programma tramite il comando `exec()`.

`fork()`

Definizione 2.5 – `fork()`

La syscall **`fork()`** crea un nuovo processo figlio **duplicando un processo** chiamante detto padre. Una volta terminata la chiamata a questa funzione avremo quindi due processi in esecuzione sullo stesso punto del codice (infatti, copiando il PCB stiamo copiando anche il Program Counter), che condividono lo stesso codice ma con copie differenti di stack e heap. L'unica vera differenza tra le due copie è il valore ritornato dalla funzione `fork()`: infatti, questa, nonostante venga chiamata una volta, se ha successo ritorna due valori, uno per il padre e uno per il figlio: ritorna il PID del figlio (un valore positivo) al padre e 0 al figlio. Se invece c'è stato un errore, il figlio non viene creato e la funzione ritorna -1 una sola volta, al padre.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    printf("Before fork: PID=%d\n", getpid()); // pid del processo corrente (padre)
    pid_t pid = fork();
    if (pid < 0) { // Errore
        perror("fork"); exit(1);
    } else if (pid == 0) { // Figlio
        printf("Figlio:  PID=%d, PPID=%d\n", getpid(), getppid());
```

```
    } else { // Padre
        printf("Padre: PID=%d, child PID=%d\n", getpid(), pid);
    }
    printf("Sia il padre che il figlio printano questa linea.");
    return 0;
}
```

L'ordine dell'else if e dell'else è casuale: non sappiamo infatti se verrà ritornato prima il valore del padre o prima quello del figlio, ma se non ci sono errori sicuramente entrambi saranno mostrati a schermo. Infatti, quando viene chiamata la funzione `fork()`, il processo si divide letteralmente in due processi paralleli: uno con il valore di ritorno per il padre e uno per il figlio.

Il figlio riceve sempre una copia dello spazio di indirizzamento virtuale del padre, ma come già detto avrà copie indipendenti di stack e heap, che se modificate nel codice non influenzeranno le copie del padre (e viceversa). Stesso discorso vale ad esempio anche per le variabili globali: il processo figlio ottiene una copia indipendente dalle altre.

Proposizione 2.1 – Chi va prima?

Come già accennato, non è possibile capire quale processo debba essere eseguito per primo. Infatti, è lo scheduler che fa questa scelta basandosi su molti fattori, che la rendono una scelta **non-deterministica**. Per questo motivo è importante scrivere codice che non dipenda dall'ordine di esecuzione dei processi padre/figlio.

Osservazione 2.1

Ogni `fork()` **raddoppia il numero di processi**. Infatti, da un processo A viene prima creato un processo B con una chiamata, ma se poi avviene un'altra chiamata sia il processo A che il processo B creano un processo a testa, arrivando a 4, e così via.

Definizione 2.6 – Fork Bomb

Una **fork bomb** è un attacco *denial-of-service* che clona processi all'infinito per riempire la tabella dei processi e bloccare il computer.

In bash una fork bomb si presenta con questa linea di comando semplicissima: `:() :|: &;:`. Mentre in C è un loop infinito che continua a chiamare la funzione `fork()`. In questo modo il numero di processi cresce in modo esponenziale fino a saturare la CPU.

Proprietà 2.3 – Copy-on-Write

Per come è stata descritta una chiamata `fork()` dovrebbe copiare l'intero spazio di indirizzamento ogni volta che un nuovo processo viene creato. Dato che questo spazio può occupare anche diversi giga, molti sistemi operativi evitano queste operazioni costose usando una tecnica chiamata **Copy-on-Write** (COW). In sostanza, i due processi padre e figlio inizialmente condividono le stesse pagine di memoria fisica, e se uno dei due processi prova a modificare quest'area si crea una nuova pagina dove viene copiato il contenuto della pagina condivisa, che sarà puntata dal processo che ha provato a modificare l'area condivisa.

Non tutte le risorse sono copiate con una `fork()`. Tra quelle non copiate troviamo:

- I timer
- I segnali pendenti (ossia in attesa di consegna)
- Ovviamente i valori PID e PPID

Definizione 2.7 – Segnale

Un **segnale** è un'interruzione software mandata a un processo. Un processo può catturare tutti i segnali che gli vengono inviati (tranne `SIGKILL` e `SIGSTOP`) ed eseguire codice apposito per gestire questi segnali.

`wait()` e `waitpid()`

Definizione 2.8 – Zombie

Un processo figlio, quando termina, non scompare immediatamente: diventa uno **zombie**, ossia un processo terminato di cui resta solo la voce nella tabella dei processi (con il PID e il valore di uscita), finché il genitore non chiama la funzione **`wait()`** per raccoglierlo. Se questa chiamata non arriva mai, lo zombie continua a occupare inutilmente una voce della tabella e un PID.

Quando viene chiamata la funzione **`wait()`**, questa rimane in attesa fino a quando un processo figlio non termina, e ritorna il PID del processo terminato. Se il chiamante di `wait()` non ha figli, questa funzione ritorna -1. Ovviamente, un padre con N figli dovrebbe aspettarli tutti: non ha senso fare `wait()` solo su un figlio se poi tutti gli altri diventano zombie.
Questa funzione risolve il problema dell'ordine non-deterministico visto prima tra padre e figlio. Infatti, con questa funzione possiamo dire al padre di aspettare finché il processo figlio non muoia. La **shell** è l'esempio perfetto in questo caso: possiamo infatti considerare la shell come il

programma padre, e tutti i suoi figli come i comandi che scriviamo. La shell (padre) riprende a funzionare solo dopo che il comando corrente (figlio) è terminato.

Oltre alla funzione `wait()` esiste anche la funzione **`waitpid()`**, che permette di specificare quale dei diversi figli di un processo si voglia aspettare. In base al valore del PID specificato, questa funzione si comporta in modo diverso:

- PID > 0: aspetta quel figlio esatto
- PID = -1: aspetta un qualsiasi figlio (come `wait()`)
- PID = 0: aspetta un qualsiasi figlio nello stesso process group
- PID < -1: aspetta un qualsiasi figlio nel process group con ID uguale a |PID|

Se tra gli argomenti di `waitpid()` aggiungiamo anche l'opzione **`WNOHANG`**, questa non si bloccherà e ritornerà subito 0 se nessun figlio ha ancora terminato (utile per il polling), oppure -1 se il processo chiamante non ha alcun figlio.
È possibile aggiungere tra gli argomenti una variabile **`status`** che fa da contenitore: alla fine del `waitpid()` (o della `wait()`), questa conterrà al suo interno informazioni su come il figlio è terminato o ha cambiato stato. Esistono poi delle macro da usare per studiare queste informazioni:

- `WIFEXITED(status)`, che ritorna True se il figlio è terminato normalmente con una chiamata `exit()`
- `WEXITSTATUS(status)`, che ritorna l'exit code (da 0 a 255) se il figlio è terminato con `exit()`
- `WIFSIGNALED(status)`, che ritorna True se il figlio è terminato a causa di un segnale
- `WTERMSIG(status)`, che ritorna il numero del segnale che ha terminato il figlio se questo è terminato a causa di un segnale
- `WIFSTOPPED(status)`, che ritorna True se il figlio è stato stoppato

`exec()`

Definizione 2.9 – `exec()`

La syscall **`exec()`** rimpiazza l'immagine in memoria del processo corrente con quella di un nuovo programma. È fondamentale per creare un processo di nuovo programma partendo dal duplicato di un processo di un altro programma. Rimpiazza totalmente il codice, i dati, lo stack e l'heap. La tabella dei descrittori dei file invece rimane identica, e questo permette di ridirigere l'I/O dei programmi nella shell usando sempre lo stesso codice.

È necessario separare le funzioni `fork()` e `exec()` perché tra le due chiamate un processo figlio può fare diverse cose, tra cui ad esempio cambiare alcune variabili d'ambiente (ossia coppie

chiave-valore globali gestite dal sistema operativo che definiscono il contesto di configurazione e il comportamento dei programmi in esecuzione), ridirezionare l'output da un file a un altro o cambiare la cartella di lavoro.

Esistono 6 varianti del comando `exec()`, che insieme formano la famiglia `exec()`:

- `execl(path,arg0, ... , NULL)`, che richiede il percorso esatto del file, riceve gli argomenti come una lista separata da virgole, e che eredita le variabili d'ambiente
- `execv(path, argv[])`, che richiede il percorso esatto del file, riceve gli argomenti come un array, e che eredita le variabili d'ambiente
- `execle(path, arg0, ..., NULL, envp[])`, che richiede il percorso esatto del file, riceve gli argomenti come una lista separata da virgole, e che riceve le variabili d'ambiente tramite un array
- `execve(path, argv[], envp[])`, che richiede il percorso esatto del file, riceve gli argomenti come un array, e che riceve le variabili d'ambiente tramite un array
- `execlp(file, arg0, ..., NULL)`, che riceve gli argomenti come una lista separata da virgole, che cerca l'eseguibile automaticamente nelle cartelle cercando il nome `file`, e che eredita le variabili d'ambiente
- **`execvp(file, argv[])`**, che riceve gli argomenti come un array, che cerca l'eseguibile automaticamente nelle cartelle cercando il nome `file`, e che eredita le variabili d'ambiente

L'ultima funzione è quella più utilizzata. In questa, il primo degli argomenti nell'array `argv[]` in realtà non è un argomento ma è il nome del processo. Dal secondo fino al penultimo sono i veri argomenti, mentre l'ultimo è sempre `NULL`, necessario per far capire quando smettere di cercare gli argomenti.

```
execvp("ls", (char *[]){"ls", "-l", "/tmp", NULL});
perror("execvp failed"); // Se arrivo qui allora non è riuscito a saltare al nuovo
    codice
_exit(EXIT_FAILURE);
```

Se `exec()` fallisce, come vediamo qui sopra il codice del programma semplicemente continua a essere eseguito. Se invece è eseguito correttamente, questa funzione non ritorna nulla. Questo ha molto senso: stiamo andando a eseguire del nuovo codice modificando interamente i dati, l'heap e lo stack, quindi l'indirizzo al quale l'`exec()` dovrebbe ritornare un valore semplicemente non esiste più, perché è stato sovrascritto.

```
pid_t pid = fork();
if (pid < 0){ // Errore nella fork
    perror("fork");
    exit(1);
}
```

```
else if (pid == 0){ // Figlio
    char *args[] = {"wc", "-l", "file.txt", NULL};
    execvp("wc", args);
    perror("exec"); // Se arriviamo qui l'exec è fallito
    _exit(1);
}
else{ // Padre
    int status;
    waitpid(pid, &status, 0);
    printf("wc finito con codice d'uscita %d\n", WEXITSTATUS(status));
}
```

Questo codice mostra come usare nel modo corretto le funzioni `fork()`, `wait()`, `waitpid()` e `exec()`.

2.1.2 Altre funzioni per gestire i processi in UNIX

Oltre a tutte le funzioni che abbiamo già visto, ve ne sono altre fondamentali.

Definizione 2.10 – `exit()` e `_exit()`

Abbiamo già visto la funzione **`exit()`**, che **termina un processo**, ma non abbiamo parlato della sua alternativa **`_exit()`**. Questa chiamata permette di fare un'uscita "grezza", ossia termina un processo senza flush dei buffer, senza handler e cose del genere.
N.B. Nel processo figlio, se termina senza aver eseguito un `exec()` (o dopo un `exec()` fallito), è consigliato usare `_exit()` invece di `exit()`. Questo perché il figlio eredita una copia dei buffer di output del padre non ancora svuotati: una `exit()` normale li svuoterebbe una seconda volta, duplicando l'output.

Definizione 2.11 – `nice()`

Per gestire la priorità di un processo lo scheduler del sistema operativo usa un valore di "**niceness**" che va da -20 a 19. Per gestire questo valore esiste la funzione **`nice()`**, che prende un intero in input che viene aggiunto al valore di niceness del processo corrente. Più questo valore è alto, meno è importante per lo scheduler. Un processo normale può solo aumentare il suo valore di niceness ma non diminuirlo (per farlo servono i privilegi di root).

Definizione 2.12 – `pipe()`

La funzione **`pipe()`** crea un **canale di comunicazione unidirezionale** tra due processi, fondamentale per scambiare dati tra un processo padre e un processo figlio. Si compone di un array di dimensione 2, dove al valore 0 corrisponde l'estremità da cui si leggono i dati usando la funzione `read()`, mentre al valore 1 corrisponde l'estremità su cui si scrivono i dati usando la funzione `write()`.

Definizione 2.13 – `dup2()`

La funzione **`dup2()`** prende in ingresso due file descriptor e fa in modo che il secondo punti dove punta il primo.

2.1.3 Shell

Un esempio classico per capire tutte queste funzioni è la **shell**.

Definizione 2.14 – Shell

Una **shell** è una **REPL** (Read-Eval-Print-Loop), ossia un ciclo infinito che stampa il prompt (ad esempio $), legge un comando e lo esegue, all'infinito.

Ogni comando in una shell crea un nuovo processo, fatta eccezione per i comandi built-in, che sono ad esempio i comandi **`cd`** (che modifica la cartella di lavoro della shell), **`exit`** (che termina il processo della shell) o **`export`** (che modifica l'ambiente della shell).
Solitamente, quando la shell riceve un comando crea un processo e chiama la funzione `wait()`, aspettando che quel processo termini per passare al prossimo comando. È possibile però far partire dei processi in **background** aggiungendo alla fine il carattere **`&`**. Infatti, se la shell rileva questo carattere alla fine del comando semplicemente non chiamerà la funzione `wait()`, lasciando il comando in esecuzione in background. Una volta terminato, il kernel manderà il segnale `SIGCHLD` al padre e questo eseguirà un `waitpid` per raccoglierlo (ossia leggerne il valore di uscita ed eliminarlo dalla tabella dei processi), così da non lasciare nessun processo zombie.

2.2 Scheduling

Abbiamo visto con quali funzioni il sistema operativo gestisce i vari processi e come questi sono catalogati, ma è arrivato il momento di farci un'ulteriore domanda: come fa il sistema operativo a decidere in che ordine eseguire più processi? Questa scelta spetta a un componente software che prende il nome di **scheduler**.

Definizione 2.15 – Scheduler

Lo **scheduler** è il componente software del computer che decide l'ordine di esecuzione dei diversi processi.

Esistono diverse **politiche di scheduling**, ossia diversi modi per decidere con quale ordine mandare i processi in esecuzione sulla CPU. Per iniziare a studiare queste diverse politiche faremo delle assunzioni sul nostro carico di lavoro (ossia sui nostri processi che dobbiamo eseguire) che pian piano toglieremo. Diremo quindi che i nostri processi

1° Hanno tutti **tempo di esecuzione uguale**
2° Arrivano tutti **allo stesso momento**
3° Una volta iniziati **non possono essere interrotti**, ossia non vi è **preemption** (intesa come la capacità del SO di togliere la CPU a un processo prima che questo termini, per assegnarla a un altro)
4° Lavorano **solo con la CPU** (no I/O)
5° Hanno un **tempo di esecuzione conosciuto**

Sono assunzioni molto forti e molto distanti dalla realtà, ma sono per il momento necessarie per studiare le prime politiche più basilari. In particolare la 5° assunzione è quella più distante dalla realtà: infatti, l'**Halting Problem** dimostra che non esiste un algoritmo in grado di determinare se un processo terminerà mai o no, quindi è proprio assurdo pensare che ne possa esistere uno che riesca a determinare non solo se finirà o meno ma anche in quanto tempo.

Per valutare una policy di scheduling useremo tre metriche: il **tempo di turnaround**, ossia il tempo che passa dall'arrivo di un processo nella coda pronti fino al suo completamento, il **tempo di risposta**, ossia il tempo che passa dall'arrivo di un processo alla sua prima esecuzione, e l'**equità**, che rappresenta quanto una policy gestisce la CPU in modo equo tra tutti i processi. Purtroppo, performance ed equità sono spesso in conflitto: si può avere una policy perfettamente equa ma poco efficace, che rende il computer lentissimo, o una super efficace ma poco equa, che lascia quindi alcuni processi sempre in attesa. Per questo, è necessario trovare un *compromesso* tra questi due fattori.

N.B. Per calcolare matematicamente l'equità di una policy, si usa il **Jain's Fairness Index**, ossia la formula

$$J(x_1, \ldots, x_n) = \frac{(\sum_{i=1}^{n} x_i)^2}{n \cdot \sum_{i=1}^{n} x_i^2}$$

dove x_i rappresenta il tempo di CPU allocato a un processo i. Se tutti i processi hanno la stessa quantità x, avremo che $J(x, \ldots, x) = \frac{n^2x^2}{n \cdot nx^2} = 1$, che rappresenta l'equità perfetta. Se invece abbiamo un processo che monopolizza la CPU, avremo un valore vicino a $\frac{1}{n}$.

2.2.1 Policy di scheduling

FIFO

La policy più semplice di tutte è quella **FIFO** (First-In First-Out), la stessa usata nelle code, anche chiamata **FCFS** (First-Come First-Served). In questo caso la coda pronti è una vera e propria coda informatica, dove i processi **vengono eseguiti nell'ordine di arrivo**.

Supponiamo di avere tre processi A, B e C arrivati tutti a tempo $t = 0$ (ipotizziamo che A sia arrivato leggermente prima di B e che questo sia arrivato leggermente prima di C), tutti con un tempo di esecuzione di 10 secondi. In questo caso, avremo come tempo medio di turnaround **20 secondi** (10 secondi di A + 20 di B + 30 di C tutto diviso 3). È un ottimo tempo, ma vediamo cosa succede appena togliamo la prima assunzione, ossia quando prendiamo 3 processi con tempi di esecuzione diversi.
Ipotizziamo gli stessi tre processi di prima ma modifichiamo il tempo di esecuzione di A da 10 secondi a 100 secondi. In questo caso il tempo medio di turnaround arriva a **110 secondi** (100 secondi di A + 110 di B + 120 di C tutto diviso 3). Questo succede per l'**effetto convoglio**: il processo A, più lento degli altri, essendo eseguito per primo rallenta anche gli altri che senza di lui sarebbero molto più veloci.

SJF

Per risolvere l'effetto convoglio si è pensato di eseguire prima i processi più veloci e dopo quelli più lenti, dando così vita a una nuova policy che prende il nome di **SJF** (Shortest-Job-First). Se proviamo a eseguire i tre processi visti prima con la nuova policy vediamo un netto miglioramento nel tempo di turnaround, che arriva a una media di **50 secondi** (10 secondi del processo B + 20 del C + 120 di A tutto diviso 3).

Proposizione 2.2

La policy SJF è la policy **migliore per ottimizzare il tempo di turnaround** se tutti i processi arrivano allo stesso istante.

Dimostrazione

Partendo dall'ipotesi che tutti i processi arrivino a tempo 0, il loro tempo di turnaround coincide con il loro istante di completamento. Se chiamiamo R_i il tempo di esecuzione (attenzione, non di turnaround) di un processo i-esimo, troviamo che

$$T_{\text{turnaround del processo i}} = \sum_{j=1}^{i} R_j$$

Il nostro tempo totale di turnaround sarà la somma di tutti questi tempi dei singoli processi, ossia

$$T_{\text{turnaround}} = \sum_{i=1}^{n} T_{\text{turnaround del processo i}} = R_1 + (R_1 + R_2) + \ldots = \sum_{i=1}^{n} R_i(n - i + 1)$$

N.B. Troviamo come coefficiente proprio $n - i + 1$ perché, guardando bene la sommatoria dei tempi di turnaround dei singoli processi, notiamo che R_1 compare n volte, R_2 compare $n - 1$ volte e così via, fino a R_n che compare solo una volta.

A questo punto è facile notare come ordinando i diversi processi possiamo ordinare i diversi R_i. Ci conviene dunque mettere sempre come primo processo quello con il tempo di esecuzione minore, dato che il suo valore R è quello che appare più spesso nella sommatoria (abbiamo visto come compaia ben n volte), mentre come ultimo processo ci conviene mettere il più lento. Abbiamo quindi dimostrato come ordinare i processi dal più veloce al più lento sia il modo migliore per ottimizzare il tempo di turnaround. ■

Proviamo ora a togliere anche la seconda assunzione, ossia prendiamo 3 processi con tempi di arrivo diversi.
Ipotizziamo gli stessi tre processi di prima A, B e C, ma ipotizziamo che i processi B e C arrivino dopo 10 secondi rispetto ad A. In questo caso, la SJF a tempo 0 farà partire il processo più veloce, e dato che è presente solo A sarà questo a partire. Dunque troviamo che il tempo medio di turnaround sarà in questo caso **103.3 secondi** (100 secondi di A + 100 di B + 110 di C tutto diviso 3). Questo succede perché, per la terza assunzione, i nostri processi una volta iniziati non possono essere interrotti.

PSJF/STCF

Per risolvere il problema della SJF viene introdotto un nuovo concetto chiamato **preemption**. Il SO non è più vincolato a dover eseguire un processo per intero prima di passare a un altro, ma può effettuare ora un context switch per passare da un processo all'altro. Stiamo dunque eliminando la terza assunzione per risolvere i problemi relativi alla SJF. È nata così un'altra policy, chiamata **PSJF** (Preemptive Shortest Job First), anche detta **STCF** (Shortest Time-to-Complete First). In questa policy, ogni volta che arriva un nuovo processo lo scheduler si sveglia per controllare quale tra i processi presenti ha il minor tempo rimanente per terminare (per quello già in esecuzione si considera solo quanto gli manca), e sceglie di mandare in esecuzione sempre quello, anche se la CPU è occupata con un altro. Ovviamente, se un processo appena arrivato ha lo stesso tempo che rimane al processo che si trova già sulla CPU, non avviene nessuno scambio per non sprecare tempo con il context switch.

Riprendendo l'esempio di prima, in questo caso troviamo che il tempo medio di turnaround è tornato ad essere **50 secondi** (120 secondi di A + 10 di B + 20 di C tutto diviso 3). Infatti,

seguendo questa policy troviamo che il processo A parte per primo e viene eseguito per 10 secondi, ma poi questo viene fermato per far eseguire i processi B e C, appena arrivati e più veloci.

Proposizione 2.3 – Corollario

La policy PSJF è la policy **migliore per ottimizzare il tempo di turnaround**.

La dimostrazione è praticamente identica a quella della Proposizione 2.2, ma ora abbiamo tolto l'ipotesi che tutti i processi arrivino insieme.

Definizione 2.16 – Starvation

La **starvation** è un fenomeno che si verifica quando un processo non viene **mai completato**. È un classico problema di equità.

Nella policy PSJF il problema della starvation è evidente: preso infatti un processo molto lento, è possibile che questo non venga mai eseguito se arrivano in continuazione nuovi processi sempre più rapidi di lui.

Notiamo quindi come la PSJF non sia perfetta. Infatti, oltre al problema della starvation, c'è anche la quinta assunzione su cui si basa che è molto irrealistica (vedremo più avanti che è possibile stimare la durata di un processo, ma non avremo mai una certezza al 100%). Inoltre, vi è anche un altro problema: durante l'analisi di queste policy abbiamo totalmente trascurato il tempo medio di reazione dei processi. Questo dato, però, non è da sottovalutare affatto. Infatti, nell'utilizzo quotidiano del computer l'utente richiede feedback immediato (o quasi, diciamo in tempo <100 ms), e queste nostre policy non rispettano sempre questa condizione. È infatti possibile che un processo richiesto dall'utente vada indietro nella coda pronti perché più lento, aumentandone così il tempo di attesa. Questa cosa è assolutamente da evitare, dunque dobbiamo cercare una nuova policy di scheduling che provi a risolvere alcuni tra questi problemi.

Round Robin

Si è arrivati dunque a una nuova policy di scheduling che prende il nome di **Round Robin** (RR), anche detta **Time-slicing**. Questa gestisce i processi con un loop circolare: ogni processo che si trova nel loop ha solo poco tempo a disposizione sulla CPU prima di lasciare il posto al processo successivo e tornare nella coda pronti fino a quando il processo non termina. Inoltre, quando un processo arriva viene messo in fondo alla coda: dato che ogni processo usa la CPU solo per poco tempo, dovrà aspettare al più qualche time slice prima di essere eseguito, migliorando in modo importante il tempo di reazione.

Definizione 2.17 – Time Slice

Il periodo di tempo che ogni processo ha a disposizione sulla CPU è detto **Time Slice** (o **quantum**).

Scegliere la durata del Time Slice è una delle scelte più importanti nella policy RR. Infatti, bisogna scegliere un periodo né troppo lungo (altrimenti l'utente non avrebbe più l'illusione delle esecuzioni simultanee) ma nemmeno troppo corto (altrimenti la CPU spenderebbe tutto il tempo a cambiare tra un processo e l'altro senza veramente lavorare sui processi). Per calcolare un tempo corretto, basta usare la formula

$$\text{Overhead} = \frac{\text{Tempo di switch tra un processo e l'altro}}{\text{Time Slice} + \text{Tempo di switch tra un processo e l'altro}}$$

e trovare un valore tale che l'overhead sia minore dell'1%. Ad esempio, se abbiamo come tempo di switch 1 ms, ha senso porre come Time Slice **100 ms**, dato che $\frac{1}{101} = 0.99\% < 1\%$.

Vediamo con un esempio come funziona il RR. Presi tre processi A, B e C che arrivano tutti insieme e che durano tutti 10 secondi, con la policy SJF avremmo come tempo di reazione medio 10 secondi (0 secondi di A + 10 di B + 20 di C tutto diviso 3), mentre con il RR (con un time slice di 2 secondi) avremmo come tempo medio solo **2 secondi** (0 secondi di A + 2 di B + 4 di C tutto diviso 3). Dunque notiamo un grande miglioramento per quanto riguarda i tempi di risposta. Purtroppo, però, lo stesso non si può dire per i tempi di turnaround. Infatti, avremo come tempo medio di turnaround **28 secondi** (26 secondi di A + 28 di B + 30 di C tutto diviso 3), contro i 20 secondi della SJF o della FIFO. Questo accade perché nella policy RR tutti questi processi che durano lo stesso quantitativo di tempo finiscono praticamente insieme verso la fine, dunque hanno un tempo di turnaround tutti più o meno uguale e molto alto, cosa che invece non succedeva ad esempio con la SJF, dove i processi erano eseguiti in modo *sequenziale*. Notiamo, però, come la policy RR sia anche più **equa** di tutte le altre policy sequenziali. Infatti, qui ogni processo ha sempre lo stesso tempo a disposizione sulla CPU di tutti gli altri, rendendola quindi una policy più equa. Per quanto riguarda invece la quinta assunzione, nemmeno con il RR siamo riusciti a eliminarla.

Proposizione 2.4

- Una policy equa e che ottimizza il tempo di risposta avrà sempre un tempo di turnaround alto
- Una policy che ottimizza il tempo di turnaround sarà sempre poco equa e poco reattiva

Questa proposizione ci costringe quindi a fare una scelta tra equità + velocità di risposta e velocità di esecuzione. Il sistema operativo dovrà quindi scegliere la policy corretta in base al proprio carico di lavoro e alle proprie necessità.

2.2.2 Incorporare operazioni I/O

Abbiamo quindi visto diverse policy di scheduling, ma non siamo mai riusciti a eliminare con nessuna di queste la quarta assunzione, ossia quella che prevede che **non ci siano operazioni I/O**. Anche questa è un'assunzione del tutto irrealistica: i programmi veri non fanno solo calcoli sulla CPU, ma leggono anche dati sul disco, si connettono a internet, o aspettano che l'utente scriva qualcosa. Quando qualcosa di questo succede, il processo passa immediatamente allo stato BLOCKED, e non può usare la CPU fino a quando l'operazione I/O non è stata completata. Di solito, un programma alterna tra operazioni CPU e operazioni I/O, dunque l'obiettivo dello scheduler deve essere quello di massimizzare l'utilizzo delle risorse dando a ogni processo la CPU solo quando questo ne ha realmente bisogno. Per farlo, dividiamo gli utilizzi della CPU di un processo in dei **piccoli sottoprocessi**, uno per ogni tratto di CPU tra due operazioni I/O (detto **CPU burst**). Se ad esempio abbiamo un processo che usa la CPU per 10 ms e poi fa un'operazione I/O, per 5 volte, lo scheduler lo vedrà come 5 sottoprocessi da 10 ms l'uno e li tratterà come processi a sé.

Per capire come funziona tutto ciò, vediamo un esempio. Ipotizziamo di avere due processi: *A* è un processo da 100 ms totali, 50 dei quali sono di operazioni CPU mentre 50 di operazioni I/O, mentre *B* è un processo da 50 ms solo di operazioni CPU. Il nostro scheduler vedrà quindi i 50 ms di operazioni CPU di *A* come 5 sottoprocessi da 10 ms, separati dalle operazioni I/O. Ora, se ad esempio usiamo una policy di PSJF, avremo all'inizio 2 processi a competere: il primo sottoprocesso di *A* da 10 ms e *B* da 50 ms (i sottoprocessi successivi di *A* arriveranno uno alla volta, ogni volta che *A* finisce un'operazione I/O). Dato che il sottoprocesso di *A* è più veloce, lo scheduler prenderà il primo sottoprocesso di *A* e lo manderà in esecuzione. Una volta terminato, il processo *A* si troverà in attesa di un'operazione I/O, e nel frattempo che attende sulla CPU ci sarà il processo *B*. In questo modo non stiamo sprecando alcun tempo vuoto sulla CPU, e una volta che *A* sarà tornato disponibile dopo l'operazione I/O, verrà subito rimesso al lavoro dato che sarà nuovamente il processo più corto, a meno che *B* nel frattempo non abbia quasi finito (ossia gli siano rimasti <10 ms di tempo).

2.2.3 Implementazioni delle diverse policy in C

Ogni policy in C viene implementata in modo differente:

- La policy **FIFO**, come già accennato, è una semplice coda, implementata tramite una **lista puntata semplice**
- Le policy **SJF/PSJF** sono code con priorità, ossia una **lista puntata semplice ordinata** o un **heap binario**
- La policy **RR** è una **lista puntata circolare**

Dagli anni '50 agli anni '60 erano la FIFO e la SJF le policy più usate nei computer, mentre nei sistemi moderni time-sharing il Round Robin e la PSJF sono le policy più importanti e usate. In questi sistemi multicore moderni il problema si è spostato più sul coordinamento: bisogna saper distribuire i processi sui core in modo equilibrato e saper riassegnare uno stesso processo allo stesso core quando possibile.

2.3 Policy di scheduling più complesse

2.3.1 MLFQ

Abbiamo visto come l'assunzione più irrealistica tra tutte quelle fatte sia il conoscere a priori il runtime di un processo. Dato che ciò è impossibile nei sistemi reali, abbiamo bisogno di uno scheduler che riesca a **prevedere** il runtime di un processo in qualche modo. Per cercare di creare uno scheduler del genere, dobbiamo prima dividere i processi in due macrogruppi:

- **Processi interattivi**, ossia processi composti da brevi calcoli sulla CPU seguiti da periodi di WAITING, che hanno bisogno di poco tempo di reazione
- **Batch/Processi CPU**, ossia processi composti da lunghi periodi sulla CPU e da poco I/O, che hanno bisogno di un tempo di turnaround ottimale

Il nostro scheduler deve essere in grado di capire ogni processo di che classe fa parte e di trattarlo nel modo corretto. Ad esempio, un processo interattivo deve avere un livello di priorità più alto di un batch. Questa idea è alla base dell'**MLFQ** (**Multi-Level Feedback Queue**).

Definizione 2.18 – MLFQ

L'**MLFQ** (Multi-Level Feedback Queue) è una gerarchia dei processi a più livelli. Questo si divide in più code con livelli di priorità differenti.

Vediamo un MLFQ a 4 livelli:

1° Q_0, livello con la priorità più alta dove vanno i nuovi processi, che utilizza come policy un RR
2° Q_1, livello con priorità alta che utilizza come policy sempre un RR
3° Q_2, livello con priorità media che utilizza come policy sempre un RR
4° Q_3, livello più basso dove vanno a finire i processi batch che utilizzano molto la CPU, che utilizza come policy una FIFO

N.B. Non è detto che un MLFQ abbia sempre 4 livelli, ma anzi spesso troviamo implementazioni tra gli 8 e i 32 livelli.

Un processo parte sempre dal livello Q_0 ma appena finisce il suo **time allotment** sulla CPU scende di un livello, e riceve ogni tanto dei boost in alto così da non finire mai in starvation.

Questo time allotment è quindi il quantitativo di tempo permesso a un livello di priorità a ogni processo prima che venga declassato. Inoltre, i primi tre livelli che utilizzano tutti il RR come policy differiscono per il Time Slice, che aumenta abbassandoci di livello, fino ad arrivare all'ultima coda che è sempre una FIFO e che quindi ha un time slice infinito.

Proprietà 2.4 – Regole dell'MLFQ

Presi due processi A e B,

1) **Priorità(A) > Priorità(B)** allora ***A* viene eseguito** e ***B* no** (se A è in Q_0 e B in Q_2 allora viene eseguito A)
2) **Priorità(A) = Priorità(B)** allora ***A*** e ***B* sono eseguiti in RR** (se A e B sono in Q_1 sono eseguiti entrambi in RR)
3) Quando un processo arriva **è sempre il processo con più priorità**
4) Quando un processo usa tutto il suo time allotment sulla CPU che gli è permesso al suo livello, **scende di livello** sempre (il tempo è *cumulativo*: anche se lascio la CPU prima che finisca il mio tempo, quando torno sulla CPU il tempo non si resetta ma continua. Dunque ogni processo non può stare più tempo di quanto gli spetti sulla CPU senza essere declassato)
5) Dopo un tot di tempo (*boost period*) **tutti i processi vengono spostati nella coda di priorità più alta** (così da non far finire in starvation nessun processo ed evitare che un processo che, finito un grosso lavoro sulla CPU, rimanga in basso nelle gerarchie)

Scegliere un **boost period** S adeguato non è facile. Non può essere troppo piccolo perché altrimenti i processi interattivi sono sempre interrotti da batch più lenti, ma se S è invece troppo grande rischio di far attendere troppo i batch che si trovano nella coda a bassa priorità.

Nella regola 4 è fondamentale che il tempo sia cumulativo: infatti in passato questo non era cumulativo, e questo lo rendeva vulnerabile a degli attacchi mirati che usavano il 99% del time allotment prima di richiedere un I/O per resettare il timer. In questo modo un processo poteva rimanere tutto il tempo che voleva nella coda di priorità più alta senza conseguenze.
Un altro tentativo di rompere questo sistema di scheduling è quello di usare la funzione `fork()` per duplicare il processo e resettare così il timer. Per questo motivo, bisogna associare un unico timer uguale tra i figli e il padre, così da far ereditare a un processo figlio il timer del padre.

Abbiamo quindi risolto il problema del runtime noto a priori creando uno scheduler che divide i nostri processi tra più livelli. In questo modo, mentre eseguiamo un processo iniziamo a capire quanto questo sia lento o veloce e lo spostiamo tra le diverse priorità, così da non eseguire mai per troppo tempo un processo lento e non far aspettare mai troppo tempo un processo veloce.

MLFQ nei sistemi reali

I sistemi reali implementano l'MLFQ in modi differenti. Ad esempio, BSD assegna un valore di priorità ad ogni processo, che può andare da 0 a 255. Solitamente i processi con valore da 0 a 63 sono processi kernel, quelli da 64 a 127 sono processi di sistema e i restanti processi da 128 a 255 sono processi utente. Linux O(1) invece utilizza un sistema a 140 livelli dove ogni livello ha una coda di esecuzione separata. È presente anche un processo di rilevazione dei processi: quelli che vanno spesso in SLEEP (ossia si staccano spesso dalla CPU) sono i processi più interattivi e ricevono un bonus, mentre quelli che utilizzano di più la CPU ricevono un malus.

Problemi dell'MLFQ

L'MLFQ è perfetto per ottimizzare performance e tempo di risposta, ma non è in grado di garantire **equità**. Infatti, l'MLFQ non riesce a garantire che un processo che merita il doppio del tempo del processo B riceva effettivamente il tempo che merita. Abbiamo bisogno di un paradigma differente, che prende il nome di **Proportional Share Scheduling**.

2.3.2 Proportional Share Scheduling

Esistono due algoritmi proportional-share:

- **Lottery scheduling**
- **Stride Scheduling**

2.3.2.1 Lottery Scheduling

Immaginiamo di avere due processi da dover eseguire A e B, e di voler distribuire 100 biglietti della lotteria ai nostri due processi. Decidiamo che il processo A è più importante del B e che debba ricevere 75 biglietti, mentre B ne riceverà solo 25. Una volta fatto, lo scheduler prende a caso uno dei 100 biglietti: **il processo che ha quel biglietto verrà eseguito**. A lungo andare troveremo che il tempo di accesso alla CPU del processo A è il triplo del tempo che ha il processo B, dato che $P(A) = \frac{75}{100} = 75\%$ mentre $P(B) = \frac{25}{100} = 25\%$. Ovviamente più estrazioni vengono fatte più i tempi di esecuzione di A e B si avvicinano a queste percentuali.

Se poi a questi due processi si aggiunge C, qualsiasi numero di biglietti che verranno dati a C modificherà la priorità degli altri processi. Ad esempio, se diamo a C 50 biglietti, troviamo che $P(C) = \frac{50}{150} = 33\%$, mentre le percentuali $P(A) = \frac{75}{150} = 50\%$ e $P(B) = \frac{25}{150} = 17\%$ sono scese. Se volessimo modificare queste percentuali dovremmo poi aumentare o diminuire il numero di biglietti di A o di B.

Il problema di usare estrazioni casuali è che, sul breve termine, queste possono sbilanciarsi molto su scenari improbabili. Ad esempio, presi due processi A e B con il 50% di biglietti per entrambi, è possibile che dopo 10 estrazioni il processo A vinca 8 estrazioni, mentre B solo 2. Calcoliamo in questo caso l'**indice di equità**:

$$U = \frac{\min(C_A, C_B)}{\max(C_A, C_B)}$$

dove C_A e C_B sono rispettivamente il numero di estrazioni vinte dal processo A e B. In questo caso avremmo quindi $U = \frac{2}{8} = 0.25$, che è molto basso. L'ideale sarebbe ovviamente di avere $\boldsymbol{U = 1}$, che indica **equità perfetta**.

Dato che l'equità perfetta nel breve periodo non sempre è raggiungibile, per i processi molto brevi il lottery scheduling potrebbe non essere la soluzione corretta. Per questo motivo introduciamo una nuova policy chiamata **stride scheduling**.

2.3.2.2 Stride scheduling

Nello **stride scheduling** a ogni processo vengono assegnati due valori:

- $\mathbf{stride_i = \frac{W}{tickets_i}}$, dove W è una costante molto grande
- $\mathbf{pass_i = 0}$ inizialmente

In questa policy noi eseguiamo sempre il processo con il pass più basso, e per ogni esecuzione di ogni processo modifichiamo il suo valore di pass aggiungendogli il suo valore di stride.

Notiamo quindi come un processo con molti tickets avrà uno stride più basso e quindi verrà eseguito più volte, perché il suo pass aumenterà più lentamente rispetto ad altri. Prendiamo ad esempio tre processi A, B e C dove $\text{stride}_A = \frac{10000}{100} = 100$, $\text{stride}_B = \frac{10000}{50} = 200$ e $\text{stride}_C = \frac{10000}{250} = 40$. Ci aspettiamo che il processo A venga eseguito il 25% del tempo, il B il 12.5% e il C il 62.5%. Vediamo dopo 10 tick cosa succede:

1° tick: A eseguito $\implies \text{pass}_A = 100$
2° tick: B eseguito $\implies \text{pass}_B = 200$
3° tick: C eseguito $\implies \text{pass}_C = 40$
4° tick: C eseguito $\implies \text{pass}_C = 80$
5° tick: C eseguito $\implies \text{pass}_C = 120$
6° tick: A eseguito $\implies \text{pass}_A = 200$
7° tick: C eseguito $\implies \text{pass}_C = 160$
8° tick: C eseguito $\implies \text{pass}_C = 200$
9° tick: A eseguito $\implies \text{pass}_A = 300$
10° tick: B eseguito $\implies \text{pass}_B = 400$

Dunque il processo C viene eseguito 5 volte, A 3 e B 2, dunque $P(C) = 5/10 = 50\%$, $P(A) = 3/10 = 30\%$ e $P(B) = 2/10 = 20\%$, già vicine alle percentuali che ci aspettavamo dopo solo 10 tick (e dopo 8 tick, quando tutti i pass valgono 200, sono esattamente 62.5%, 25% e 12.5%). Dunque lo stride scheduling **converge molto più velocemente alle percentuali previste** rispetto al lottery scheduling.

Un problema importante di questo scheduling è che un processo nuovo monopolizza la CPU fino a quando il suo valore pass non raggiunge i pass degli altri processi. Per questo, è importante impostare come base per i pass dei nuovi processi il valore **minimo tra i pass dei processi attuali**.

2.3.3 Linux passa a CFS

Abbiamo già accennato allo scheduler Linux O(1) che utilizzava una policy MLFQ. Linux, però, nell'ottobre del 2007 ha deciso di cambiare scheduler e passare al **CFS** (Completely Fair Scheduler). Questo per due motivi:

- La strategia bonus/malus era troppo complessa e a volte anche imprecisa
- Con molti processi gli errori di arrotondamento nel calcolo della priorità di un processo si accumulavano, portando alcuni processi ad avere più o meno tempo sulla CPU di quanto gli dovesse spettare

Il **CFS** non utilizza delle code di priorità, ma cerca piuttosto di modellare una CPU perfettamente equa, ossia una CPU che esegue tutti i processi in contemporanea dando a ogni processo un quantitativo di CPU uguale per tutti. Ovviamente ciò non è realizzabile realmente, dato che la CPU può eseguire al più un processo per core. Il CFS cerca quindi di emulare questo comportamento calcolando sempre quanto tempo ogni processo ha ricevuto e mandando in esecuzione sempre il **processo che ha ricevuto fino a quel momento meno CPU rispetto a tutti gli altri**. Per fare ciò, lo scheduler assegna a ogni processo un **virtual runtime** (`vruntime`), calcolato in nanosecondi e aggiornato ad ogni esecuzione seguendo questa formula: $\texttt{vruntime} += \Delta t \cdot \frac{W_0}{w_i}$ (con il `vruntime` di un nuovo processo che parte dal minimo `vruntime` tra i processi presenti, così da non fargli monopolizzare la CPU), dove Δt è il tempo di esecuzione del processo, W_0 è il peso del processo a `nice = 0` (di solito 1024), e w_i è il peso del processo i determinato basandosi sul suo valore `nice`. Quindi, un processo con alta priorità avrà il proprio valore `nice` più basso, quindi il valore w_i più alto e quindi un `vruntime` più basso. Questo scheduler altro non usa che una policy **stride scheduling**, implementata però non con una sorted list ma con un red-black tree (una specie di AVL), che seleziona velocemente il prossimo processo da eseguire, ossia quello con il `vruntime` più basso che è sempre nel nodo più a sinistra dell'albero, il cui puntatore è salvato nella cache per poterlo raggiungere velocemente.

Ci sono due importanti parametri nel CFS:

- **Target Scheduling Latency**, ossia il tempo massimo che può passare entro il quale TUTTI i processi vanno eseguiti
- **Minimum Granularity**, ossia il tempo minimo assegnato a ogni processo sulla CPU

Ovviamente questi due valori vanno bilanciati: non posso avere come TSL 1 s e poi come MG 0.5 s, perché già con soli 3 processi questi due valori vanno in conflitto. Con un numero elevato di processi si arriverà sempre a una scelta tra TSL e MG: in questo caso, il minimum granularity avrà la priorità sul TSL, dunque ogni processo avrà sempre quel tot minimo di tempo sulla CPU **indipendentemente** dal numero di processi.

2.4 Virtualizzazione della memoria

2.4.1 Perché virtualizzare la memoria?

Il nostro computer ha una sola RAM fisica che deve dividere tra tutti i processi. È proprio il sistema operativo a dover dividere questa memoria in modo adeguato tra i diversi processi, e assicurarsi che ogni processo non vada a toccare una zona di memoria non propria. Astrarre la memoria ci facilita questa operazione.

È importante virtualizzare la memoria perché facilita di molto questo lavoro del sistema operativo. Basti pensare che i primi computer non avevano alcuna astrazione della memoria: ogni processo doveva usare direttamente gli indirizzi fisici. Questo non solo era molto pericoloso, perché ogni programma poteva scrivere su un blocco di memoria non proprio e corrompere il SO, ma rendeva anche impossibile il **multiprogramming**, dunque la CPU spesso si trovava ferma ad aspettare un'operazione I/O perdendo tempo, e ogni computer poteva quindi eseguire solo un processo alla volta. Oggi col multiprogramming invece possiamo eseguire più processi insieme, che si alternano sulla CPU così da eliminare i momenti morti nei quali la CPU non esegue alcun processo in attesa di un'operazione di I/O.

Dunque si è passati da un solo processo a più processi eseguiti in contemporanea sulla CPU. Rimangono però diversi problemi, sia di protezione sia di facilità di utilizzo. Per rispondere a questi problemi nasce la **memoria virtuale**.

Proprietà 2.5 – Proprietà della memoria virtuale

La memoria virtuale deve avere tre proprietà fondamentali:

- **Trasparenza**: i processi devono credere che la memoria virtuale che l'OS gli consegna sia la vera memoria fisica del computer
- **Efficienza**: la virtualizzazione deve essere veloce e non deve sprecare RAM (ciò avviene grazie a un supporto hardware, che rende queste operazioni più veloci)
- **Protezione**: nessun processo deve poter toccare lo spazio di memoria dedicato a un altro processo

Definizione 2.19 – Spazio d'indirizzamento

Lo **spazio d'indirizzamento** è l'astrazione che fa il sistema operativo sulla memoria fisica, che altro non è che un **range** di indirizzi virtuali. Ogni processo ha il proprio spazio d'indirizzamento privato con i propri indirizzi virtuali, che però non corrispondono direttamente a un indirizzo fisico. Infatti, due processi possono avere lo stesso indirizzo virtuale `0x0ABCDEF0` ad esempio, che però corrisponde per i due processi a due indirizzi fisici diversi.

N.B. QUALSIASI indirizzo visibile sul computer è sempre un indirizzo virtuale. L'hardware (la MMU), configurato dall'OS, lo traduce infatti in modo trasparente senza che nemmeno il programma ne sia a conoscenza.

Osservazione 2.2 – Range degli spazi d'indirizzamento

Gli spazi d'indirizzamento più vecchi avevano 16 o 32 bit di spazio allocato. Nei sistemi moderni troviamo indirizzi detti a **64 bit**, che però in realtà sono indirizzi a 48 bit, che creano spazi d'indirizzamento di addirittura 256 TiB, più che sufficienti, o al massimo 57 bit (con la paginazione a 5 livelli) che creano spazi da 128 PiB (gli indirizzi fisici invece arrivano al massimo a 52 bit, ossia 4 PiB).

Uno spazio d'indirizzamento virtuale contiene tre regioni logiche distinte:

- **Stack + Heap**, dove si trovano le variabili locali e le allocazioni dinamiche richieste con `malloc`
- **Static Data**, dove si trovano le variabili statiche e globali (molto piccolo, lo trascureremo spesso)
- **Code**, dove si trovano le istruzioni del programma

Queste sono posizionate in ordine dall'indirizzo di memoria più alto (per Stack e Heap), al più basso (per il codice). Questo segmento, in particolare, è read-only, con una dimensione fissa, e

condiviso tra i diversi thread di uno stesso processo.

Per quanto riguarda la regione Heap, questa è quella che come già detto contiene la memoria allocata dinamicamente tramite `malloc`. Cresce verso l'alto, ossia verso indirizzi di memoria più alti. La regione Stack invece contiene le variabili locali. Si trova in cima allo spazio d'indirizzamento e cresce verso il basso, ossia verso indirizzi di memoria più bassi. Dunque queste due strutture crescono una verso l'altra: se si toccano, l'OS uccide il programma, trovando un errore di *stack overflow* o di *heap exhaustion*.

Abbiamo già parlato di Heap e di `malloc` e di tutti i problemi che ne conseguono nel primo capitolo quando abbiamo studiato un po' di C. Tutti quei problemi che abbiamo studiato riguardano proprio questi indirizzi di memoria virtuali, dunque è importante avere quei concetti ben assimilati per poter capire ciò che vedremo da qui in avanti.

2.4.2 Passare da memoria virtuale a fisica

Riallocazione statica

Prima dell'introduzione dei dispositivi hardware per la traduzione dalla memoria logica (non ancora virtuale, la virtualizzazione vera e propria verrà dopo) a quella fisica si utilizzava una tecnica puramente software chiamata **riallocazione statica**.

Definizione 2.20 – Riallocazione statica

La **riallocazione statica** è una tecnica software per tradurre indirizzi logici in fisici. In poche parole l'OS riscrive tutti gli indirizzi logici richiesti dal programma in indirizzi fisici in base a dove è posizionato il programma. Questa riallocazione è sempre *contigua*.

Se ad esempio il nostro programma utilizza indirizzi logici tra `0` e `100`, e questo viene posizionato all'indirizzo fisico di RAM `5000`, la riallocazione statica traduce tutti gli indirizzi logici in indirizzi da `5000` a `5100`.

I vantaggi di questa tecnica sono:

- **zero supporto hardware**
- **zero tempo sprecato a runtime**, dato che tutte le allocazioni avvengono a loadtime
- **implementazione semplice**

Gli svantaggi però sono molti e importanti:

- **nessuna protezione**: infatti, non c'è alcun controllo hardware che verifichi che nessun processo tocchi memoria destinata ad altri processi

- una volta che il processo viene allocato in una zona di memoria, **non può essere spostato**, dato che gli indirizzi vengono modificati a loadtime prima che il processo parta

È evidente quindi che è necessario anche un supporto hardware per poter gestire correttamente la memoria virtuale. Questo ci porta al primo meccanismo hardware che prende il nome di **base and bounds**.

Base and bounds

Con un supporto hardware possiamo infatti rimpiazzare le vecchie traduzioni statiche con nuove allocazioni dinamiche. Queste rimangono comunque trasparenti ai programmi, ma diventano ora più **efficienti**, **veloci** e **sicure**.

Il primo meccanismo hardware mai creato è il **base and bounds**, introdotto alla fine degli anni '50. Nell'MMU, che è il supporto hardware per la traduzione degli indirizzi e la protezione della memoria, vengono inseriti due registri:

- **base register**, che contiene l'inizio dell'indirizzo di memoria fisico di un processo
- **bound register**, che contiene la dimensione dello spazio d'indirizzamento del processo o l'indirizzo fisico finale disponibile per il nostro processo (le due interpretazioni sono equivalenti, ma ragioneremo con la prima)

Ogni CPU ha una sola coppia di questi registri, dunque quando il sistema operativo deve fare un context switch, deve salvare la coppia di valori del processo che stavamo eseguendo e caricare quella del processo che vogliamo eseguire. Questa coppia di valori viene salvata nel PCB del programma.

Preso quindi un indirizzo virtuale, dopo aver fatto il check indirizzo virtuale < bound, possiamo calcolarci l'indirizzo fisico con questa semplice formula:

$$\text{indirizzo fisico} = \text{indirizzo virtuale} + \text{base}$$

Nel caso il check fallisca, l'MMU solleva un'eccezione, la CPU passa in modalità kernel e il gestore dell'OS termina il processo che ha richiesto una zona di memoria non sua.

Il base and bounds ci permette anche di **muovere un processo** mentre questo già è in esecuzione, al contrario della riallocazione statica. Banalmente, l'OS lo toglie dalla CPU con un context switch, trova una nuova porzione di memoria dove spostare il processo e aggiorna il suo base register. Per cercare un indirizzo vuoto, l'OS cerca in una linked list dove salva tutti i range di memoria liberi sulla RAM.
Ricordiamo che il processo è all'oscuro di tutto ciò: non saprà mai dove si trova a livello fisico e nemmeno gli interessa, dato che usa gli indirizzi virtuali.

Alla conclusione di un processo, il sistema operativo deve verificare se la memoria appena liberata sia adiacente ad altri spazi vuoti e, in tal caso, unire questi blocchi per formarne uno più grande da allocare in futuro a un altro programma, in un processo che prende il nome di **coalescing**. Per decidere quale regione di memoria assegnare a un nuovo processo, il sistema operativo può seguire diverse politiche:

- **First-Fit**: scansiona la lista dall'inizio e seleziona la prima regione di dimensione sufficiente. È una tecnica veloce, ma tende ad accumulare molti piccoli frammenti nella parte iniziale della lista.
- **Best-Fit**: cerca la regione più piccola in grado di contenere il processo. Minimizza lo spazio sprecato nella singola allocazione, ma tende a lasciare un gran numero di frammenti di piccolissime dimensioni.
- **Worst-Fit**: seleziona la regione libera più grande disponibile. Consuma il blocco di dimensione maggiore (che potrebbe servire a processi più grandi), ma tende a lasciare frammenti residui più ampi, che hanno più probabilità di essere riutilizzati.

Nessuna politica è perfetta in assoluto: la scelta va effettuata valutando lo specifico contesto d'uso.

Il problema principale del base and bounds è il requisito di memoria **contigua**, del tutto analogo a quello della riallocazione statica. Ciò comporta un limite enorme: anche in presenza di una grande quantità di memoria libera complessiva, se questa è fortemente frammentata potrebbe non esistere un singolo blocco contiguo sufficientemente ampio per contenere l'intero processo.

Questo fenomeno di inefficienza si presenta anche all'interno dello spazio d'indirizzamento del processo. Se ad esempio vengono allocati 16 KB di RAM a un processo, 12 KB potrebbero essere riservati preventivamente allo Stack e all'Heap (ovvero lo spazio intermedio previsto per la loro espansione), rimanendo inizialmente inutilizzati. Di conseguenza, in ogni processo è presente una porzione di RAM fisica non sfruttata, un problema che si amplifica all'aumentare dei processi attivi. Ciò accade perché il base and bounds alloca fin da subito tutto lo spazio necessario al processo in modo preventivo, anche se al momento non serve: un processo che richiede inizialmente solo 3 KB di RAM si vedrà assegnare un blocco molto più grande, trasformando la parte residua in memoria sprecata.
Un altro problema del base and bounds è l'impossibilità di eseguire qualsiasi processo il cui spazio d'indirizzamento virtuale sia più grande della memoria RAM fisica disponibile.
Infine, con il base and bounds non è possibile condividere alcuna porzione di memoria tra processi distinti, anche quando questi condividono dati o codice.

Segmentazione (evoluzione del base and bounds)

Per risolvere alcuni di questi problemi del base and bounds, si è pensato di dividere il blocco contiguo di memoria di un singolo processo in tre diversi segmenti: **code segment**, **heap segment**

e **stack segment**, ognuno con la sua coppia di registri base and bounds e con i propri permessi (il code segment ha permessi R-X, ossia di lettura ed esecuzione ma non scrittura, mentre l'heap e lo stack segment hanno permessi RW-, ossia di lettura e scrittura ma non di esecuzione). In questo modo eliminiamo tutta la memoria allocata in eccesso tra l'heap e lo stack in modo preventivo, così da allocare solo la memoria necessaria per il processo in sé. Inoltre, rendendo il blocco del codice di un programma un segmento indipendente dal resto del processo, è possibile ora usare un singolo code segment per più processi di uno stesso programma, risparmiando ulteriore memoria.

Gli indirizzi di memoria virtuale si presentano ora in questo modo:
2 bit per il **segmento** (00 per il code, 01 per l'heap e 11 per lo stack), e il resto dei bit per l'**offset**, ossia quanto dobbiamo spostarci dal base del segmento interessato per trovare l'indirizzo fisico che realmente vogliamo. Ad esempio, se abbiamo VA = 01000000001100 (dove VA significa virtual address, ossia l'indirizzo di memoria), allora stiamo puntando a un indirizzo fisico nell'heap segment (01, calcolato dal computer facendo uno shift a destra di 12) con offset $000000001100 = 12_{10}$ (calcolato dal computer facendo una maschera, facendo un `AND` con $0x0FFF$), dunque dobbiamo prendere il base register dell'heap segment di tale processo e aggiungere 12 per trovare l'indirizzo fisico richiesto (ricordiamo che tutto ciò è eseguito dall'MMU a livello hardware, che controlla anche che questo valore di 12 + base rientri nel bound di quello specifico segmento, in questo caso dell'heap segment).

Lo stack è un caso particolare. Infatti, abbiamo già visto come questo *si muova verso il basso*, dunque il ruolo dell'offset è differente per lui. Avremo un valore base che indica la cima della pila, e dunque l'indirizzo più alto raggiungibile, e un offset che quindi non va aggiunto al valore base, perché altrimenti andremmo oltre l'indirizzo più alto assegnato allo stack, ma va usato per calcolarsi il **negative offset**. In sostanza, preso un offset di 24, una dimensione massima del segmento di 32, un bound di 20 e un base di 30, per trovare l'indirizzo fisico dobbiamo calcolare il negative offset, che è pari all'offset - la dimensione massima del segmento, e poi dobbiamo sommare questo valore al base, che quindi diminuirà, scendendo verso il basso. L'MMU controlla poi che il valore assoluto del negative offset non superi il bound, altrimenti andiamo fuori dallo stack. Nel nostro esempio avremo quindi un negative offset pari a 16 - 24 = -8 ($8 \leq 20$, quindi l'accesso è valido), per cui il nostro indirizzo di memoria fisico si troverà al byte 30-8 = 22.

Anche l'heap si comporta in modo particolare, ma non per quanto riguarda la traduzione da VA a PA (physical address), ma per quanto riguarda la richiesta di memoria aggiuntiva. Se ad esempio un programma esegue una `malloc()` ma l'heap è pieno, questa non restituisce un errore, ma chiama prima la syscall **sbrk()**, che prende un valore in input pari alla memoria in più richiesta e cerca se c'è memoria fisica contigua grande abbastanza per ospitare l'heap più la nuova memoria richiesta. Prima di spostare l'intero blocco, però, l'OS controlla sempre se c'è memoria fisica vuota vicina all'heap grande quanto la richiesta di `sbrk()`, e solo se non trova questa memoria vicina va a cercarla altrove. Se poi non dovesse trovare mai questa memoria, restituirebbe un

errore, e questa che sembra una situazione molto difficile in realtà con la segmentazione è più frequente di quello che si potrebbe pensare. Infatti, se da una parte abbiamo eliminato la frammentazione interna, rimuovendo lo spreco di memoria che vi era prima tra heap e stack, dall'altra diventa ancora più grave la **frammentazione esterna** (già presente con il base and bounds): in poche parole, con la segmentazione si vengono a formare velocemente tanti piccoli blocchi di memoria contigua che occupano da soli poca memoria, ma che insieme formano uno spreco di RAM considerevole. Questi blocchi, essendo troppo piccoli per qualsiasi segment di qualsiasi processo importante, rimangono lì inutilizzati, sprecando memoria.
Vediamo visivamente perché questo è un problema. Supponiamo di richiedere al sistema operativo 25 KB di memoria contigua libera, e di avere 4 processi A, B, C e D che già occupano memoria sulla RAM, che si trova in questa situazione:



La nostra memoria libera totale è 37 KB (2+3+8+1+23), dunque 25 KB ci entrano senza alcun problema, ma purtroppo la nostra RAM è frammentata in piccoli blocchi di RAM da pochi KB, e il blocco contiguo più grande è di soli 23 KB, non sufficiente. Questo spreco non è assolutamente accettabile! Si potrebbe risolvere compattando la memoria, ma è una soluzione con costo troppo elevato, dunque non attuabile. Per questo motivo nasce un nuovo modo di virtualizzare la memoria, che prende il nome di **memoria con paginazione**.

Memoria con paginazione (paging)

L'idea alla base della memoria con paginazione (o **paging**, in inglese) è semplice: smettiamo di allocare blocchi di memoria variabili, come nel base and bounds, e iniziamo a dividere la memoria in **blocchi fissi**, che prendono il nome di **pagine** (**N.B.** pagine è il termine per la memoria virtuale, per quella fisica dovremmo parlare di frame, ma spesso useremo pagine in entrambi i contesti). Il sistema operativo ha poi il compito di mantenere una **page table** per ogni processo dove ad ogni sua pagina virtuale vi è mappata una fisica (tramite un **PFN**, physical frame number), in una corrispondenza uno a uno essendo le pagine e i frame della stessa dimensione. Questa divisione in pagine rompe il vincolo più pesante che ci siamo sempre portati con noi, ossia il **vincolo di contiguità**. Infatti, pagine contigue nella memoria virtuale NON devono corrispondere per forza a pagine contigue nella memoria fisica, eliminando quindi il problema della frammentazione esterna. Torna però ovviamente il problema della frammentazione interna: infatti, se le pagine sono tutte di 4 KB (standard nei sistemi moderni) e io ho un programma che richiede 4 KB + 1 B, a questo verranno allocate due pagine, di cui una però sarà totalmente vuota fatta eccezione per un byte solo.

I VA in questa nuova virtualizzazione si dividono in due: i primi n bit servono per la **VPN** (virtual page number), ossia per indicare in quale pagina virtuale si trova quell'indirizzo, mentre i restanti m bit servono per l'**offset**, ossia per dire dove si trova l'informazione all'interno della pagina virtuale. Ad esempio, in un sistema a 32 bit con pagine da 4 KB, useremo 12 bit per l'offset e 20 per la VPN, permettendoci quindi di avere $2^{20} \approx 1000000$ pagine virtuali diverse, anche se magari abbiamo una memoria fisica che ci permette di avere solo 500000 pagine diverse.

Per capire meglio come funziona il paging vediamo un esempio. Ipotizziamo di avere una memoria RAM di 128 byte divisa in 8 frame da 16 byte ciascuno con all'attivo un processo A così diviso.

`Frame 7`	`(16 byte)`	`[FREE]`
`Frame 6`	`(16 byte)`	`[FREE]`
`Frame 5`	`(16 byte)`	`[Page 3 of A]`
`Frame 4`	`(16 byte)`	`[FREE]`
`Frame 3`	`(16 byte)`	`[Page 2 of A]`
`Frame 2`	`(16 byte)`	`[Page 0 of A]`
`Frame 1`	`(16 byte)`	`[Page 1 of A]`
`Frame 0`	`(16 byte)`	`[OS Kernel]`

Nella page table di A avremo quindi quattro righe, una per ogni pagina. La pagina virtuale 0 sarà quindi mappata a quella fisica 2, è valida (vedremo ora cosa sono questi bit di validità), e ha protezione R-X (notiamo quindi come non ci interessi più sapere se è code/heap/stack, ma solo che operazioni possiamo eseguire su di essa). La pagina virtuale 1 è mappata a quella fisica 1, è valida e ha protezione RW- e così via.

VPN	Page Table Entry (PTE)
0	PFN = 2, valid=1, prot=R-X
1	PFN = 1, valid=1, prot=RW-
2	PFN = 3, valid=1, prot=RW-
3	PFN = 5, valid=1, prot=RW-
4	PFN = 0, valid = 0
⋮	⋮
$2^{12} - 1$	PFN = 0, valid = 0

Ipotizzando di trovarci in un sistema a 16 bit, useremo quindi 12 bit per la VPN e 4 per l'offset. Preso ad esempio un virtual address 0000000000110101, l'MMU sa già che ci stiamo riferendo alla virtual page $000000000011_2 = 3_{10}$, con un offset di $0101_2 = 5_{10}$, dunque andrà nella Page Table del processo A alla riga 3, prenderà il PFN, troverà la page fisica giusta e aggiungerà l'offset di 5, che è **uguale tra pagine fisiche e virtuali** proprio perché la dimensione di queste due pagine è uguale (sono tutte e due pagine da 16 byte). Tutto ciò si riassume quindi nella formula **PA = PFN × page_size + offset**.

Per quanto riguarda le PTE (page table entry), oltre al PFN queste hanno, come già visto nella tabella, altre informazioni, come la validità, la possibilità di leggere/scrivere/eseguire la pagina e anche diverse altre. Tra i bit più importanti troviamo:

- **Valid bit**: 1 se la pagina fa parte dello spazio d'indirizzamento del processo, 0 altrimenti (se si accede a una pagina con valid bit pari a 0, la MMU lancia un page fault e l'OS, visto che l'accesso non è lecito, invia al processo il segnale **SIGSEGV**)

- **Readable/Writable/Executable bit** (3 bit differenti, uno per leggere, uno per scrivere e uno per eseguire): 1 se la pagina è leggibile/scrivibile/eseguibile, 0 se non lo è
- **User bit**: 1 se è accessibile in user-mode, 0 se è accessibile solo in kernel mode
- **Accessed bit**: 1 se la pagina è stata usata (letta o scritta) di recente, 0 se non lo è stata
- **Dirty bit**: 1 se la pagina è stata riscritta, 0 se non lo è stata
- **Global bit**: 1 se la pagina è condivisa tra tutti i processi (come una kernel-page), 0 se non lo è

Questo nuovo sistema, che sembra perfetto tralasciando un po' di spreco riguardante la frammentazione interna, ha ovviamente anche lui dei problemi importanti. Infatti, per effettuare un accesso in memoria in questo modo avremo bisogno non di un solo accesso, ma di **due accessi**: uno per arrivare alla page table e uno per arrivare all'indirizzo che realmente ci interessa. Inoltre, una singola page table consuma molto spazio da sola. Riguardiamo la page table di A dell'esempio precedente: questa ha 2^{12} righe, ognuna di 2 byte, dunque consuma $2^{12} \cdot 2 = 2^{13}$ byte, ossia 8 KB. Sembra poco, ma se ci spostiamo in un sistema a 64 bit (ossia 48 bit realmente utilizzati), con pagine da 4 KB, avremo $2^{48}/2^{12} = 2^{36}$ righe di 8 byte ciascuna, per un costo totale di $2^{36} \cdot 8 = 2^{39}$ byte, ossia 512 GB di RAM solo per una singola page table! Cerchiamo quindi di capire come possiamo risolvere questi problemi.

TLB

Per quanto riguarda il doppio tempo di accesso, viene introdotto un nuovo dispositivo hardware che prende il nome di **TLB** (Translation Lookaside Buffer), che non è altro che una piccola cache all'interno dell'MMU che contiene un insieme delle **PTE usate recentemente**. Può contenere tipicamente dalle 32 alle 1024 entrate, a cui si accede di solito in pochissimo tempo, qualche nanosecondo. Essendo una cache hardware interna all'MMU, il programma non sa mai che questa esiste: per lui prendere un indirizzo fisico da questa o dalla page table è la stessa identica cosa.
Ogni entrata del TLB ha, oltre alla VPN e alla sua PTE corrispettiva, un **ASID** (Address Space ID), ossia un identificatore che indica a quale processo appartiene tale PTE. Senza di lui, ad ogni context switch bisognerebbe cancellare l'intero TLB e riempirlo da capo con il nuovo processo, provocando tanti miss inutili.

Ogni volta che viene effettuato un memory access da un programma l'MMU controllerà quindi sempre prima il TLB, e se l'entrata richiesta è lì allora non deve passare dalla page table per poter trovare l'indirizzo fisico. Per calcolare il tempo medio di accesso alla memoria, ossia l'**EMAT** (che ricorda molto l'AMAT, visto quando abbiamo parlato di cache), dobbiamo introdurre l'**hit-rate**, ossia la probabilità che l'entrata richiesta da un processo sia effettivamente nel TLB. Troviamo quindi questa formula:

$$\text{EMAT} = h \cdot (T_{\text{TLB}} + T_{\text{mem}}) + (1 - h) \cdot (T_{\text{TLB}} + T_{\text{PT}} + T_{\text{mem}})$$

Sembra molto complicata ma in realtà è molto semplice: semplicemente, il costo di un accesso in memoria è pari alla probabilità che l'entrata richiesta sia nel TLB moltiplicata per il costo di accesso al TLB e il costo di accesso alla RAM più la probabilità che l'entrata non sia nel TLB moltiplicata per il costo di accesso al TLB, alla page table e alla RAM. Se assumiamo che $T_{\text{TLB}} \approx 0$, essendo questo tempo molto piccolo, semplifichiamo la formula in $\textbf{EMAT} \approx \boldsymbol{T_{\text{mem}} + (1-h) \cdot T_{\text{PT}}}$. Se ipotizziamo di avere $T_{\text{mem}} = 100$ ns, $T_{\text{PT}} = 100$ ns, $h = 0.99$, troviamo che EMAT $\approx 100 + 0.01 \times 100 = 101ns$, un costo ben migliore rispetto a quello che avremmo avuto senza TLB, che sarebbe pari a $200ns$. È indispensabile però che l'hit-rate sia molto alto (generalmente tra 95 e 99%): questo per fortuna è quasi sempre rispettato anche con poche entrate nel TLB. Per capire perché questo è vero, guardiamo questo codice:

```
int sum = 0;
for (int i = 0; i <1000; i++){
    sum += A[i]; // A è un array di 1000 interi
}
```

Salvando la VPN della pagina dello stack dove si trova la variabile `sum` e la variabile `i`, e salvando le VPN delle pagine dove si trova l'array `A` (per 1000 interi serviranno circa 1 o 2 pagine) nel TLB, dopo i primi 3 o 4 miss quando ancora le variabili non sono salvate avremo sempre degli hit per 996 o 997 iterazioni, arrivando a un hit rate > 99%.
La **TLB coverage** è poi un altro valore importante che corrisponde alla quantità di spazio di indirizzamento virtuale mappabile dal TLB. Banalmente, TLB coverage = entrate del TLB × grandezza della pagina. Se ad esempio abbiamo una pagina di 4 KB e 1024 entrate sul TLB, avremo una TLB coverage di $4 \times 1024 = 4096$ KB, ossia 4 MB.

Il TLB può essere gestita sia a livello hardware (cosa che fanno architetture tipo x86 o ARM, ossia le architetture più moderne), sia a livello software (cosa che fanno architetture tipo MIPS R2000). Vediamo le differenze:

- Se è gestita a livello **hardware**, l'MMU si sposta sulla page table del processo in esecuzione e cerca la PTE come se il TLB non fosse esistito, rendendo il processo veloce (dato che il sistema operativo non viene mai coinvolto), ma a patto che l'hardware conosca l'esatto formato della page table che usa il sistema operativo (altrimenti non riuscirebbe a trovare la PTE corretta)
- Se è gestita a livello **software**, l'hardware lancia un'eccezione (una trap) alla CPU, che passa a kernel mode e scorre la page table tramite codice software, rendendo il processo più lento (dato che entra in azione il sistema operativo), ma più flessibile, perché può lavorare con qualsiasi formato di page table esistente (questo perché la page table viene letta tramite software)

Un fenomeno da tenere bene a mente quando si lavora con il TLB è il **TLB thrashing**. In poche parole, se si hanno diverse pagine che lavorano in circolo, senza averne nessuna prediletta

rispetto alle altre, avremo un continuo circolo di indirizzi che vengono scritti per poi essere eliminati prima ancora di essere letti dall'MMU, cosa che non porta alcun vantaggio. Ipotizziamo ad esempio di avere un TLB con spazio per 64 PTE, e di avere un ciclo che gira sempre tra le stesse 100 pagine. Una volta arrivata alla 65° pagina, il TLB è pieno, dunque per far spazio a questa sul TLB bisogna eliminare la 1° pagina, ma facendo così quando torneremo alla 1° pagina non la avremo di nuovo sul TLB, e per rimetterla dovremo eliminare un'altra pagina. Continuando così, notiamo che ogni volta che il TLB potrebbe rivelarsi utile la pagina da noi richiesta è già stata tolta per fare spazio a un'altra.
Un modo per risolvere il TLB thrashing è quello di usare delle **pagine più grandi** (**huge pages**, da 1 GB o da 2 MB nelle architetture x86-64, dove le pagine da 2 MB prendono il nome di large pages), così da aumentare la TLB coverage. In questo modo riusciamo a coprire più RAM ed evitare che una pagina venga sovrascritta prima che torni a essere richiesta. Per usare le huge pages, spesso piuttosto che modificare tutta la virtualizzazione della memoria cambiando dimensione delle pagine si preferisce adottare un altro sistema, ossia quello delle **THP** (transparent huge pages). In poche parole, il kernel in automatico, se vede più pagine diverse che lavorano, cerca un blocco di memoria contiguo dove riscriverle tutte insieme per salvarle come un'unica pagina da mettere nel TLB, usandone così solo un'entrata.

Page table a più livelli

Per quanto riguarda invece il problema dello spazio eccessivo delle page table, si è osservata una cosa molto importante: una page table alloca sempre il numero massimo di entrate possibili, ma un processo normale non utilizza mai questo numero di entrate: non ci va nemmeno vicino! Un processo di solito utilizza circa 500 pagine da 4 KB, ossia circa 2 MB, ma in un sistema a 64 bit abbiamo visto che gli vengono allocate 2^{36} righe nella page table, un numero decisamente eccessivo. Nascono per questo le **page table a più livelli**. Da questo momento in poi non esiste più una singola page table per ogni processo, ma tante page table diverse, puntate da una tabella più grande chiamata **page directory**, le cui righe prendono il nome di **PDE** (page directory entry). La memoria diventa quindi una specie di albero: la page directory è la radice, ogni PDE punta a una page table, e le PTE di quest'ultima puntano ai frame. Nel caso a 32 bit, la page directory ha 1024 PDE e ogni page table ha 1024 PTE. Un valid bit pari a 1 nella PDE i-esima indica che la page table a cui punta esiste (ossia contiene almeno una PTE valida); se è 0, quella page table non viene nemmeno allocata.
Con la page table a 2 livelli stiamo però aggiungendo un nuovo accesso in memoria: arriviamo quindi a 3 accessi in memoria se il TLB fallisce. Purtroppo, questo è un costo necessario: è il famoso trade-off di cui spesso parliamo. Non sarà mai possibile avere un sistema velocissimo e perfettamente efficiente a livello di memoria, ma bisogna piuttosto trovare un giusto compromesso così da avere un sistema funzionante.

I VA dunque vengono modificati: in un sistema a 32 bit, i 20 bit prima destinati alla VPN si dividono ora in due gruppi. Il primo, che rappresenta la **VPN1**, serve per indicare a quale PDE bisogna andare a cercare, mentre il secondo gruppo, che rappresenta la **VPN2**, serve per indicare a quale PTE bisogna andare nella tabella puntata dalla PDE.

In un sistema a 64 bit, però, una page table a 2 livelli non basta più. Infatti, avremmo 36 bit da dividere per le due VPN, arrivando a 18 bit per ogni VPN, ossia 2^{18} entrate per ogni livello. Dato che ogni entrata costa 8 byte, avremmo un costo totale di $2^{18} \times 8 \approx 2$ MB per la page directory più 2 MB per ogni page table interna, un costo comunque troppo elevato. Per questo motivo, si introducono le page table **a quattro livelli** (standard per l'architettura Intel x86-64). Qui dividiamo i 36 bit tra 4 VPN, ognuna da 9 bit, arrivando a tabelle di massimo 4 KB, pochissimo rispetto ai 2 MB visti con le page table a due livelli, e infinitamente piccolo rispetto ai 512 GB visti per le page table a un solo livello (ora arriviamo a circa 512 GB di page table solo se un processo mappa davvero tutto lo spazio virtuale di 256 TiB, cosa però ovviamente praticamente impossibile). I diversi gruppi nei sistemi x86-64 non sono chiamati semplicemente VPN1, VPN2..., ma invece sono detti, in ordine dal più alto in gerarchia al più basso:

1° **PML4** (Page Map Level 4, la radice puntata dal registro `CR3`)
2° **PDPT** (Page Directory Pointer Table)
3° **PD** (Page Directory)
4° **PT** (Page Table, le foglie)

Il SO gestisce una page table a quattro livelli in modo molto schematico:

1° Appena un **processo viene creato**, il sistema operativo crea la page table radice PML4, con tutte entrate invalide.
2° Quando una **pagina viene richiesta**, se il TLB missa, inizia la camminata lungo le diverse page-table. Se lungo questa camminata si arriva a un'entrata con bit di validità pari a 0, la MMU lancia un'eccezione **page fault**, gestita dal page-fault handler del sistema operativo che controlla se la richiesta era valida o meno. In caso questa sia valida, l'OS identifica quali livelli lungo la camminata mancano e alloca in memoria fisica le pagine necessarie per contenere queste sottotabelle intermedie ancora non esistenti, che vengono poi linkate ai rispettivi genitori. Se poi è necessario, l'OS alloca anche la memoria fisica per i dati richiesti, e chiude l'eccezione: l'istruzione che aveva causato il fault viene rieseguita e la nuova camminata avrà ora successo.

Per quanto riguarda le large e huge pages, queste saltano rispettivamente uno e due livelli della gerarchia: dunque le large pages seguono una page table a 3 livelli, con il PD che fa da foglia, mentre le huge pages seguono una page table a 2 livelli, con il PDPT che fa da foglia.

2.4.3 Swapping

Arriviamo ora a una delle domande più importanti da farsi quando si parla di memoria virtuale. Abbiamo visto come ogni processo, tramite il paging, in un sistema a 64 bit possa avere a disposizione potenzialmente fino a 2^{36} pagine, ossia 256 TiB di informazioni (se si parla di pagine standard da 4 KB). Moltiplicato questo per diversi processi attivi in contemporanea, si arriva a quantità di memoria che anche i migliori pc si sognano. Arriveremo quindi a un certo punto a dover affrontare un problema: cosa succede quando la somma degli spazi di indirizzamento virtuale supera la memoria fisica a disposizione? Nasce una nuova tecnica per gestire questa situazione, che prende il nome di **swapping**. L'illusione che dà lo swapping è semplice: ogni processo pensa di avere RAM illimitata, sempre e comunque. Questo però ovviamente non è vero, e il SO gestisce questa situazione semplicemente spostando (swapping) sulla memoria secondaria le pagine meno usate quando la RAM non basta, così da liberare memoria sulla RAM. Queste pagine vanno a finire in una zona specifica della memoria secondaria, che prende il nome di **swap space**. Quando poi queste pagine swappate verranno richieste dal processo, viene chiamata l'eccezione page-fault e la pagina viene ricaricata seguendo un processo che prende il nome di **page-in** (o major page fault), che legge dove si trova sul disco secondario dalla PTE e trova uno spazio in memoria RAM dove inserire la pagina in questione, in un tempo che varia dai $100\mu s$ per le SSD ai $10ms$ per gli HDD (ciò avviene non solo quando una pagina viene riswappata dalla memoria secondaria alla RAM, ma si verifica anche ogni volta che una nuova pagina mai letta che si trova sulla memoria secondaria viene richiesta). Tutto ciò, ricordiamo, avviene totalmente all'oscuro del processo, che pensa di aver ricevuto l'informazione come se fosse sempre stata sulla RAM.

Per supportare lo swapping, nella PTE nasce un nuovo bit: il **present bit**, che vale 1 se la pagina è presente in RAM e 0 se si trova sul disco (questo però solo se valid bit = 1, altrimenti l'indirizzo non è nemmeno presente sul disco). Entra qui in gioco anche un altro bit di cui abbiamo già parlato, ossia il **dirty bit**, che ricordiamo evidenzia se una pagina è stata sovrascritta o meno. Ipotizziamo di avere una pagina sulla memoria secondaria che, dopo un page-in, viene trasferita nella RAM (ricordiamo che, dopo un page-in, la pagina swappata rimane su entrambe le memorie, non viene eliminata dalla memoria secondaria). Quando questa deve essere riswappata, ossia deve essere effettuato un **page-out**, il SO controlla prima di tutto il dirty bit: se questo è pari a 1 allora bisognerà sovrascrivere la pagina in memoria secondaria con quella nuova perché questa è stata modificata, ma se questo è invece pari a 0 significa che la pagina presente in memoria secondaria è una copia identica di quella sulla RAM, dunque non serve riscrivere nulla sulla memoria secondaria ma basta solo eliminare sulla RAM la pagina da swappare.

Page fault

Abbiamo parlato spesso dell'eccezione **page-fault**, ma vediamo bene ora cos'è e da cosa può essere provocato.
Un page fault è un'eccezione sollevata dall'MMU quando un accesso in memoria riscontra dei problemi. Questo può essere provocato in diverse situazioni:

- Una pagina swapped out, che ha quindi un valid bit = 1 e present bit = 0
- Una pagina mai allocata, che ha quindi un present bit = 0 e che, se non accessibile dal processo che la chiama, può sfociare in un SIGSEGV, uccidendo il processo chiamante
- Un tipo di accesso non permesso su una pagina, come ad esempio una scrittura su una pagina riguardante il codice che ha protezione R-X, e che sfocia in un SIGSEGV che uccide il programma
- Un'azione write su una pagina *Copy-on-Write*: brevemente, quando un processo esegue una `fork`, padre e figlio puntano allo stesso frame fisico. Questa pagina, che logicamente dovrebbe essere scrivibile, viene contrassegnata dall'OS come read-only nella page table: quando un processo prova a scriverci, la MMU genera un page-fault. L'OS gestisce l'eccezione allocando un nuovo frame, copiandovi i dati e rendendolo writable per il processo che ha scritto. Il vecchio frame del padre tornerà writable solo se il padre rimarrà l'unico processo a puntare a quel frame fisico.
- *Minor page fault*, ossia un page fault che si risolve senza leggere dalla memoria secondaria (ad esempio perché la pagina si trova già sulla RAM, ma non nella page table di questo processo). Ciò si verifica spesso quando parliamo di spazi di memoria condivisi tra più processi o quando si fa un primo accesso alla memoria anonima: in poche parole, quando con una `malloc` allochiamo un oggetto dinamico, il sistema operativo non assegna subito dei frame fisici, dunque al nostro primo accesso su tale memoria la MMU genererà un page fault e il SO prenderà un frame vuoto in RAM e lo mapperà alla PTE corrispondente. In entrambi i casi, però, tutto ciò avviene senza dover effettuare alcuna operazione I/O sulla memoria secondaria, dunque le operazioni non sono lente come nei major page fault, che richiedono in media $100\mu s$ (su una SSD), ma richiedono solo $1 - 10\mu s$.
- *Major page fault*, ossia un page-in.

Page replacement policy

I major page fault sono tra tutte le cose quella che rallenta di più un processo, a causa delle loro richieste di I/O sulla memoria secondaria. Per questo motivo, è necessario minimizzarli studiando delle corrette **page replacement policy**, delle policy che ogni volta che la RAM è piena devono decidere quali pagine swappare e quali no. Ovviamente la scelta migliore è quella di swappare le pagine che non saranno utilizzate per molto tempo, e non quelle che vengono invece utilizzate spesso. Nasce proprio a questo proposito la **Optimal replacement policy** (o

OPT, che sta per OPTimal), una replacement policy che sceglie di swappare sempre la pagina che più tra tutte *verrà usata il più tardi possibile*. Per fare ciò sfruttiamo la **reference string**, ossia la sequenza dei numeri di pagina a cui il programma accede. Per capire meglio come funziona questa policy vediamo un esempio. Supponiamo di avere una memoria RAM con spazio per soli 3 frame, e di avere la seguente reference string: $7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0$. Dopo 3 page fault, che arrivano perché nessuna delle prime tre pagine inizialmente è in memoria, arriviamo ad avere in RAM $7, 0, 1$ e ad avere come reference string $2, 0, 3, 0, 4, 2, 3, 0$. Ora, dato che 2 non è in RAM arriverà un major page fault e dovremo eseguire uno swapping. Per decidere con quale pagina fare lo swapping, l'OS guarda avanti lungo la reference string: nota che né 1 né 7 verranno più riutilizzati, dunque uno dei due è un candidato perfetto per essere swappato. Decidiamo di swappare 1, e arriviamo quindi ad avere in RAM $7, 0, 2$. Quando arriviamo alla pagina 3 swappiamo poi il 7, e quando arriviamo alla pagina 4 dobbiamo nuovamente decidere chi swappare: in questo momento ci sono in RAM $3, 0, 2$ e abbiamo lungo la reference string $4, 2, 3, 0$. Dato che la pagina 0 è la pagina a cui accediamo più tardi nel tempo, la swappiamo con 4, e poi rieseguiremo uno swap finale quando ci servirà nuovamente. Seguendo questo algoritmo, siamo riusciti a minimizzare il numero di major page fault sollevati dall'MMU.

Purtroppo, però, l'OPT si basa su un qualcosa che abbiamo già visto in passato essere impossibile: conoscere l'esecuzione di un processo a priori. La reference string nella pratica non può mai sapere tutte le richieste di pagine effettuate da un processo prima che questo le esegua. Dunque l'OPT non può essere un algoritmo utilizzabile nella pratica, ma rappresenta il *massimo* a cui ogni replacement policy dovrebbe puntare: più una policy si avvicina al valore trovato con l'OPT, più questa è efficiente. Vediamo dunque alcune policy di replacement realmente utilizzate.

La più intuitiva di tutte è la **FIFO**, già vista nelle scheduling policy. Semplicemente, si swappa la pagina che è stata in memoria *per più tempo di tutte*. Non è molto efficace, dato che una pagina che è stata molto in memoria non indica per forza una pagina usata poco, ma smette di essere un candidato effettivo tra le replacement policy per un'anomalia importante che prende il nome di **anomalia di Belady**. In poche parole, aumentare il numero di frame disponibili in RAM può aumentare, se si usa come policy la FIFO, il numero di page fault. Per capire perché vediamo un esempio. Ipotizziamo di avere come reference string $1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$, e di avere due RAM, una con solo 3 frame disponibili e una con 4. Intuitivamente, la RAM con 4 frame dovrebbe generare meno page fault, ma in realtà si scopre che questa ne genera 10 contro i 9 generati dalla RAM con solo 3 frame. Questo comportamento è pericoloso: aumentare la memoria fisica rende *più lento* il processo, una cosa inaccettabile. Dobbiamo quindi trovare una nuova policy.

Si arriva dunque alla **LRU** (Least Recently Used), una replacement policy che non si basa più sul futuro, come la OPT, ma sul **passato**. L'idea di base è semplice: quando siamo costretti a swappare, decidiamo sempre di swappare la pagina *non usata per il maggior tempo possibile* tra tutte le pagine in RAM. Questo ovviamente è possibile perché il passato lo conosciamo, e nella pratica è anche molto efficace, registrando valori vicini all'OPT.

Per implementare questa policy è ovviamente necessario conoscere il tempo esatto dell'accesso più recente ad ogni pagina. Si potrebbe pensare di salvare all'accesso a una pagina il clock tick esatto in un registro speciale per ogni frame, per poi trovare, quando bisogna eseguire uno swap-out, il clock tick più piccolo e swappare la pagina associata a tale valore. Purtroppo però quest'idea non è praticabile. Infatti, avere un singolo registro per ogni frame richiederebbe un costo hardware importante, e controllarli tutti prima di ogni swap-out porterebbe a un costo computazionale in $O(n)$, con n = numero di frame sulla RAM. Per questo motivo, nella realtà non riusciamo a usare nemmeno la LRU, ma dobbiamo accontentarci di un'approssimazione, che prende il nome di **clock algorithm** (o second-chance algorithm). In questo algoritmo posizioniamo tutti i frame fisici in una lista circolare, e creiamo un puntatore per tenere traccia della posizione corrente sulla lista. Di ogni frame ci interessa in particolare l'accessed bit, che vale 1 se la pagina è stata usata (letta o scritta) di recente e 0 se non lo è stata. Quando arriva un major page fault, l'algoritmo fa avanzare il puntatore lungo la lista, e per ogni pagina che legge ci sono due possibili casi:

- se l'accessed bit è pari a 1, allora viene data una *seconda possibilità* alla pagina: il suo bit viene azzerato ma la pagina non viene swappata, spostando il puntatore alla pagina successiva
- se l'accessed bit è pari a 0, allora la pagina viene swappata e la nuova pagina arrivata in RAM avrà assegnato un accessed bit con valore 1

2.4.4 Thrashing

Nonostante tutte queste contromisure atte a gestire la memoria RAM nel modo più efficiente possibile, può capitare al SO di aver bisogno di più RAM fisica di quanta se ne abbia a disposizione. Quando ciò succede, entriamo in un circolo pericolosissimo: ogni volta che un processo A avrà bisogno di una nuova pagina, swapperà una pagina necessaria a un altro processo B, e quando B avrà bisogno di quella pagina swapperà a sua volta una pagina necessaria a un altro processo C, e così via, in un loop che non finisce mai (questo succede quando il SO utilizza una politica di **global replacement**, che mette tutti i frame di diversi processi in un calderone unico). È un fenomeno che abbiamo già visto per la cache e per il TLB, che prende sempre il nome di **thrashing**. Una risposta a questo fenomeno può essere il **local replacement**, una politica del SO che assegna a ogni processo un numero fisso di frame, così da evitare che un processo faccia swap-out su un frame non proprio. Questo però porta con sé un altro problema: se un processo A riceve 20 frame ma ne utilizza solo 5, rimangono ben 15 frame inutilizzati! Per minimizzare ciò dovremmo già sapere a priori di quanti frame un processo necessita prima di eseguirlo, un problema che abbiamo già visto essere irrisolvibile.

Per cercare di risolvere quindi il problema del thrashing, il sistema operativo può fare diverse scelte:

- Può decidere di **sospendere alcuni processi**, così da liberare tutti i frame a loro destinati per fare spazio ai processi rimasti attivi. I parametri per scegliere i processi da sospendere sono i seguenti:
  1) Priorità del processo (più un processo ha priorità bassa più è probabile che venga sospeso)
  2) Numero di frame assegnati (più un processo ha frame assegnati più è probabile che venga sospeso)
  3) Durata del processo (più un processo è attivo da tanto tempo più è probabile che venga sospeso)
- Può gestire il numero di frame di ogni processo studiando la sua **PFF** (page fault frequency): in poche parole, se un processo solleva tanti page fault allora ha bisogno di più frame, se invece ne solleva pochi può lasciare qualche frame ad altri processi che ne necessitano
- Se nessuna delle due opzioni viste sopra funziona, Linux sceglie di fare un'operazione drastica: **uccide il processo con il punteggio OOM più alto** (il calcolo del punteggio OOM si basa sulla percentuale di RAM consumata, sui processi figli posseduti e sui privilegi). Questa tecnica prende anche il nome di **OOM Killer**.

Tra tutte, sospendere i processi è quasi sempre la scelta migliore. Infatti, è sempre meglio eseguire pochi processi bene piuttosto che tanti processi male: ridurre il multiprogramming da 10 a 5 processi può aumentare di 10 volte la velocità dei 5 processi rimasti in esecuzione.

Capitolo 3

Concorrenza

Definizione 3.1 – Concorrenza

La **concorrenza** è la capacità di un sistema operativo di gestire più processi/thread in contemporanea, permettendo di portare avanti più attività contemporaneamente.

3.1 Thread

Abbiamo già accennato nel primo capitolo a come un processo possa dividersi in più **thread**, che sono la più piccola unità di esecuzione schedulabile dal sistema operativo. È come se fossero dei processi separati che però condividono lo stesso spazio di indirizzamento e quindi gli stessi dati. Questi, nei sistemi moderni, sono l'unità di allocazione dello scheduling (quella che abbiamo visto finora era quindi una semplificazione, non sono i processi singoli ad essere schedulati ma i loro diversi thread).
Sono importanti perché ci permettono di lavorare su un singolo processo in parallelo, prendendo tutti i suoi sottocompiti ed eseguendoli insieme, risparmiando così tempo. Questo è ovviamente possibile solo se si hanno più core, che lavorano ognuno come una CPU a sé. Se abbiamo solo un singolo core e abbiamo un processo che non fa I/O, non ha senso dividerlo in thread perché c'è comunque solo un posto disponibile.

Si potrebbe pensare di utilizzare la syscall `fork()` per creare diversi processi figli piuttosto che dividerne uno solo in tanti thread. Questo era ciò che si faceva in passato, ma crea un limite non indifferente: infatti, più thread lavorano nello stesso ambiente (stesso code, heap, global data, spazio d'indirizzamento e page table, con invece ognuno un proprio stack, PC e registri), mentre più processi lavorano in ambienti totalmente indipendenti tra loro che per comunicare necessitano di IPC (Inter-Process Communication, come pipes o sockets). Inoltre, un context switch tra due thread è molto più veloce rispetto a uno tra due processi, dato che molti ambienti non devono essere modificati.
Un problema che però si verifica con l'utilizzo dei thread è quando vi è un crash: se creiamo tanti processi figli, se uno crasha il crash rimane isolato, gli altri processi non sono toccati. Invece, con diversi thread, il crash di uno comporta il crash dell'intero processo.

Ogni thread ha poi, oltre al proprio stack, al PC e ai propri registri, anche un proprio **TCB**, dove sono salvati tutti questi dati. Questo è anche usato come identificatore dallo scheduler per

poter selezionare i singoli thread di un singolo processo sulla CPU. Vi è anche salvata un'altra informazione, ossia lo **stato** del thread. Anche qui esistono i tre stati visti con i processi: READY, RUNNING, BLOCKED.

3.1.1 `pthread` API

Per lavorare con i thread in C utilizziamo l'API **`pthread`**. Vediamone alcune funzioni qui presenti.

`pthread_create()` serve per **creare** un thread. Tra i parametri bisogna inserire il *thread ID*, *gli attributi* (`NULL` per gli attributi di default), *una funzione* che il thread dovrà runnare e *gli argomenti passati alla funzione*.

```c
void *funzione_thread(void *arg){
    int id = *(int *)arg;
    return NULL;
}
int main(void){
    pthread_t tid; // In pthread_create viene popolato
    int arg = 1;
    pthread_create(&tid, NULL, funzione_thread, &arg);
}
```

In questo codice stiamo quindi creando un thread per eseguire la funzione `funzione_thread`. Notiamo come il thread ID e gli argomenti della funzione debbano essere passati tramite indirizzo, perché la funzione `pthread` utilizza dei puntatori per contenere questi valori. Anche il tipo di ritorno della funzione che il thread deve eseguire deve essere un puntatore `void*` (ossia un puntatore che punta a una variabile di un qualsiasi tipo), e i suoi argomenti devono anch'essi essere di tipo `void*`.

Si crea però in questa situazione un problema: abbiamo creato un nuovo thread, ma come facciamo a sapere quale thread finirà per primo? Quello che ha creato il nuovo thread, o quello appena creato? Non è possibile saperlo, perché dipende dalla politica di scheduling e da quanta CPU venga assegnata a ogni thread. Per questo motivo nasce la funzione **`pthread_join(tid, NULL)`**, che inserita nel codice del thread chiamante lo blocca finché non finisce il thread creato. Se al posto di `NULL` passiamo poi l'indirizzo di una variabile di tipo `void *` (ad esempio `&ret`), qui dentro ci andrà il valore di ritorno del thread che stiamo aspettando si chiuda. Questa funzione è molto necessaria perché, se il main thread finisce (con un `return` o con `exit()`), anche tutti gli altri thread terminano, anche se avevano ancora istruzioni da eseguire.

N.B. In un for su una variabile `i`, non si può passare come argomento di una funzione in una `pthread_create` il valore `&i`, dato che il main continua a modificarla dentro il for. Se si

vuole fare ciò bisognerà passare come valore `(void *)(intptr_t)i`, che passa il valore in `i` e non la variabile (nella funzione del thread lo si recupera poi con `(int)(intptr_t)arg`, senza dereferenziarlo).

3.1.2 Problemi del multithreading

Vediamo ora un classico problema riguardo alla memoria condivisa tra thread. Ipotizziamo di avere un `counter` condiviso tra due thread, ciascuno dei quali lo incrementa di 1 per un milione di volte. Ci aspetteremmo giustamente come risultato 2 milioni, ma in pratica quasi mai lo otteniamo.

Il motivo è che incrementare `counter` richiede tre istruzioni macchina distinte: una `LOAD`, una `ADD` e una `STORE`. La sequenza nel suo insieme non è atomica, quindi le istruzioni dei due thread possono alternarsi in modi diversi:

- Su un singolo core, un thread può essere tolto dalla CPU tra una di queste istruzioni e l'altra. Quando torna in esecuzione, salva un valore calcolato a partire da un dato ormai vecchio, sovrascrivendo tutti gli incrementi eseguiti nel frattempo dall'altro thread.
- In un sistema multicore i due thread possono invece lavorare in contemporanea. Ad esempio, T_1 può leggere `counter = 0` e aggiungere 1, e prima che questo valore venga salvato è possibile che il thread T_2 legga anch'esso `counter = 0`, aggiunga 1 e salvi `counter = 1`. A quel punto T_1 salverà anch'esso il suo valore `counter = 1`, sovrascrivendo quello di T_2 e rendendo inutile il suo incremento: dopo due incrementi `counter` vale 1 invece di 2.

Questo è un caso classico di **race condition**: il risultato dipende dall'ordine in cui i thread vengono schedulati, ed è quindi non deterministico, potendo cambiare da un'esecuzione all'altra. Ma non solo: si tratta anche di un **data race**, problema che si verifica quando due o più thread accedono alla stessa variabile, almeno uno in scrittura, senza alcuna sincronizzazione. I due concetti qui sono strettamente legati: è proprio perché gli accessi a `counter` non sono sincronizzati (ed è questo proprio il data race) che le istruzioni dei due thread possono alternarsi in modo da far lavorare un thread su un valore ormai vecchio (il che produce la race condition).
N.B. In C un data race è un *undefined behavior* (come già visto con l'heap buffer overflow): il compilatore pensa che questa situazione non capiti mai e ne ottimizza il codice di conseguenza. Il programma potrebbe quindi funzionare correttamente, andare in crash o produrre risultati inspiegabili che noi non possiamo prevedere.

Chiamiamo **sezione critica** la porzione di codice che accede a una risorsa condivisa (in questo caso l'incremento di `counter`). Per evitare il problema, la sezione critica deve essere eseguita in *mutua esclusione*, ossia solo un thread alla volta può eseguire la sezione. La mutua esclusione da sola però non basta, perché sarebbe soddisfatta anche impedendo a tutti i thread di entrare. È

logico che, se nessun thread è nella sezione critica e qualcuno vuole entrarci, questo deve poterlo fare.

Per ottenere tutto questo si introduce un costrutto astratto chiamato **lock**: una variabile, inizialmente libera, su cui si possono eseguire due operazioni, **`lock`** e **`unlock`**. Se un thread esegue `lock` su una lock libero, lo acquisisce ed entra nella sezione critica. Ogni altro thread che esegue `lock` su quel lock resta bloccato finché il primo non esegue `unlock`. Questo vale anche se il thread che detiene la lock viene tolto dalla CPU: gli altri dovranno comunque aspettare che torni in esecuzione e concluda la sezione critica. Per questo motivo le sezioni critiche devono essere il più brevi possibile.
Con la `pthread` API la lock si implementa con una variabile condivisa di tipo **`pthread_mutex_t`** (mutex = MUTual EXclusion), inizializzata con `PTHREAD_MUTEX_INITIALIZER` oppure con `pthread_mutex_init`. Si chiama poi **`pthread_mutex_lock(&lock)`** per entrare nella sezione critica e **`pthread_mutex_unlock(&lock)`** per uscirne. Vedremo poi nella sezione successiva come sono implementate queste due funzioni nel dettaglio.

Oltre alla race condition e al data race esistono altri problemi riguardanti il multithreading. Tra questi uno dei più subdoli e nascosti è quello dell'**ordine di esecuzione delle operazioni**. Ipotizziamo di avere il seguente codice:

```
result = compute_answer();
done = 1;
```

È molto semplice: esegue una funzione, ne salva il valore di ritorno nella variabile `result`, e segna `done = 1`. Se pensiamo di eseguire questo codice in un thread a parte, per capire quando questo è terminato potrebbe venire naturale pensare di porre la condizione `done == 1`. Questo, però, sarebbe un grande errore. Infatti, il compilatore potrebbe ottimizzare il nostro codice in questo modo:

```
done = 1;
result = compute_answer();
```

Questo succede perché il compilatore pensa che questo codice verrà interamente eseguito da un unico thread, e dato che `result` e `done` sono indipendenti fra loro, cambiare l'ordine di esecuzione non dovrebbe essere un problema. Dunque il controllo `if done == 1` potrebbe restituire true ancora prima che `compute_answer()` venga chiamato, generando così un comportamento anomalo.
Si potrebbe pensare di cambiare il compilatore e fare in modo che l'ordinamento del codice non venga toccato. Il problema, però, non verrebbe risolto ma semplicemente spostato, dal software all'hardware. Infatti, ogni core ha un proprio store buffer, un'area privata dove vengono salvate le informazioni prima che vadano a finire nella cache condivisa. Su architetture con un

modello di memoria debole (come ARM) è quindi possibile che un valore che si trova dopo nel codice venga reso visibile prima di uno che si trova prima di lui, ossia che `done = 1` venga reso visibile prima del valore in `result`. Anche su x86, dove le scritture restano in ordine, lo store buffer permette comunque che una lettura venga eseguita prima che una scrittura precedente sia visibile agli altri core, creando un comportamento altrettanto anomalo.

Per risolvere questo problema esiste uno strumento che prende il nome di **memory barrier** (o **fence**). Questo strumento rende vero ciò che a noi è sempre sembrato ovvio, ossia fa in modo che tutto ciò che si trova prima debba essere reso visibile prima di tutto ciò che si trova dopo. In C questo viene fatto utilizzando una funzione built-in, ossia `__sync_synchronize()`, in questo modo:

```
result = compute_answer();
__sync_synchronize();
done = 1;
```

Questa impedisce sia al compilatore sia alla CPU di spostare gli accessi in memoria da una parte all'altra della barriera (in entrambe le direzioni): rappresenta proprio una barriera non oltrepassabile. Questo lavoro, però, non va fatto solo nel thread che lavora, ma anche in quello che attende, altrimenti avremmo lo stesso problema che abbiamo già affrontato. Scriviamo quindi

```
while(done == 0){} // Attesa di avere done = 1
__sync_synchronize();
use(result);
```

N.B. Perché questo codice funzioni davvero, `done` deve essere dichiarata come `atomic_int done = 0;` (con `#include <stdatomic.h>`). Su una variabile atomica, `done = 1` e `done == 0` diventano letture e scritture atomiche, e il compilatore è obbligato a rileggere `done` a ogni giro del ciclo. Con un `int` normale, invece, potrebbe leggerla una sola volta fuori dal ciclo, che girerebbe quindi all'infinito.

Tutto ciò che abbiamo visto fino ad ora viene racchiuso in una relazione che prende il nome di **happens-before**. Semplicemente, dire che "A happens-before B" non significa che A succede prima nel tempo di B, dato che abbiamo già visto che questo non basta a causa del riordinamento del compilatore e degli store buffer, ma significa proprio che *tutti gli effetti derivanti da A sono già visibili prima che B venga eseguito*.

3.2 Implementazione del `lock`

Abbiamo già visto cosa fanno le funzioni `lock` e `unlock` e il motivo per cui queste sono così importanti per gestire le sezioni critiche. È arrivato però il momento di capire **come vengono create** queste due funzioni e gestite in modo efficiente.

Ci sono tre parametri che dobbiamo tenere bene a mente:

1) **Correttezza**: se un thread sta lavorando su una sezione critica, nessun altro può entrarci; se la sezione è libera e qualcuno vuole entrarci deve poterlo fare.
2) **Equità**: bisogna evitare che sempre lo stesso thread abbia l'accesso alla sezione critica. Bisogna implementare una policy FIFO: se ci sono n thread che vogliono una sezione critica, un thread dovrà aspettare al più $n - 1$ volte che un altro finisca, non di più.
3) **Performance**: quanto costa usare una lock quando questo è libero? E quanto quando invece è occupato?

3.2.1 Implementazioni software-only

L'implementazione più semplice di una lock consiste nel **disabilitare gli interrupt**: quando un thread chiama `lock`, le interruzioni vengono disabilitate, e vengono riabilitate solo quando il thread chiama `unlock`. In questo modo il timer interrupt che farebbe uscire il thread dalla CPU non può partire, il sistema operativo non riprende mai il controllo della CPU e lo scheduler non ha occasione di sostituire il thread con un altro: la sezione critica viene quindi eseguita senza interruzioni.

Questa implementazione ha però molti problemi:

- Modificare il timer interrupt è un'operazione privilegiata, e renderla accessibile tramite codice dagli utenti non è una grande idea perché può significare lasciare un thread che non fa nulla sulla CPU per sempre, facendo freezare il computer.
- Nei sistemi multicore questa idea semplicemente non funziona: infatti anche se bloccassimo il timer interrupt di un core, tutti gli altri sarebbero liberi di lavorare e degli altri thread potrebbero così accedere alla sezione critica.
- Lasciare un thread sulla CPU finché non finisce un'operazione che non si conosce non è mai una grande idea: se questa operazione è troppo lunga finirebbe per rallentare tutto il sistema.

Quest'idea, che è attuabile solo per del codice kernel molto breve, non è una buona implementazione generale.

Un'altra idea molto intuitiva sarebbe quella di usare una variabile **`flag`** per segnare se la lock è libero o meno: 0 se è libero, 1 se non lo è. Il problema qui si verifica a causa della non atomicità

delle istruzioni. Infatti, se implementiamo il `lock` in questo modo, staremmo creando una funzione di questo tipo:

```
int flag = 0;
void lock() {while (flag == 1); flag = 1;}
void unlock() {flag = 0;}
```

Con questa implementazione è possibile che un thread chiami `lock()`, legga `flag == 0`, e prima di poter porre `flag = 1` venga tolto dalla CPU. A questo punto un secondo thread può leggere anch'esso `flag == 0`, porre `flag = 1` ed entrare nella sezione critica; quando il primo thread torna in esecuzione pone anch'esso `flag = 1` ed entra. Avremmo quindi due thread dentro la sezione critica, il che viola completamente il principio di mutua esclusione.

Si potrebbe allora pensare di usare un'altra variabile condivisa, chiamata `turn`, che decida di chi è il turno sulla sezione critica. L'implementazione è semplice:

```
int turn = 0;
void lock(int i) {while(turn != i);}
void unlock(int i) {turn = 1 - i;} // Passo il turno a quello dopo di me
```

Il problema, però, è altrettanto semplice: dato che `turn` parte pari a 0, se T_1 vuole accedere per primo alla sezione critica non può farlo, ma deve aspettare sempre che acceda per primo T_0 (problema che riaccade poi ogni volta che un thread vuole accedere per due volte di fila alla sezione critica senza aspettare che l'altro gli ripassi il turno).

Dall'unione del `flag` e del `turn` nasce un altro algoritmo, che prende il nome di **algoritmo di Peterson**. Vediamo come viene implementato:

```
int flag[2] = {0,0};
int turn;
void lock(int i){
    flag[i] = 1; // Voglio entrare
    turn = 1-i; // Cedo però la precedenza all'altro thread
    while(flag[1-i] == 1 && turn == 1-i){} // Entro se l'altro non vuole entrare
        oppure se è il mio turno
}
void unlock(int i) {flag[i] = 0;}
```

In questo algoritmo viene assegnato a ogni thread un proprio flag, e la variabile `turn` decide sempre di chi è il turno. Non appena `lock(int i)` viene chiamata, poniamo `flag[i] = 1` per indicare la volontà di entrare, ma poniamo allo stesso tempo `turn = 1-i` per lasciare la precedenza all'altro thread. A questo punto entriamo in attesa: o aspettiamo che l'altro thread

ponga `turn = i`, o aspettiamo che chiami `unlock` e ponga `flag[1-i] = 0`. Una volta che ciò succede possiamo finalmente entrare nella sezione critica.

Questo algoritmo, teoricamente, funziona: infatti, anche nel caso limite in cui due thread chiamino contemporaneamente `lock()`, l'ultimo che modificherà la variabile `turn` sarà sempre quello che aspetterà, senza eccezioni. Il problema, però, si verifica per colpa di un costrutto hardware che abbiamo visto poco tempo fa, ossia lo **store buffer**. Quando un thread modifica `flag[i]`, questa modifica non arriva subito nella cache comune, ma ha un po' di ritardo. A causa di questa architettura, è possibile che due thread diversi leggano l'uno il `flag` dell'altro pari a 0, rendendo immediatamente la condizione del while falsa e dando immediatamente accesso ad entrambi alla sezione critica.

3.2.2 Implementazioni hardware

È quindi purtroppo evidente come un'implementazione puramente software non sia a tutti gli effetti possibile, e sia invece necessario un aiuto riguardo hardware. Questo ci permette di avere:

- **Atomicità**, effettuando le operazioni di lettura e scrittura in un unico passaggio
- **Visibilità**, garantendo che una scrittura atomica sia vista dagli altri core prima delle operazioni che la seguono, e che una RMW lavori sempre sul valore più recente
- **Ordine**, evitando problemi riguardanti la lettura di variabili in ordine diverso da quello scritto nel codice (problema già visto quando abbiamo parlato di memory barrier)

Definizione 3.2 – RMW

Una **RMW** (atomic Read-Modify-Write) è un'istruzione che legge un valore, ne calcola uno nuovo e lo scrive, **tutto in un unico passaggio**.

Esistono diverse istruzioni RMW:

- **TAS**
- **CAS**
- **FAA**

TAS

Definizione 3.3 – TAS

Una **TAS** (Test And Set) è un'istruzione atomica che scrive un nuovo valore e ne ritorna quello vecchio.

```c
int TAS(int *p, int new_value){
    int old = *p;
    *p = new_value;
    return old;
}
```

Una TAS è perfetta per verificare se una lock sia occupato o meno e, nel caso, occuparlo. Per fare ciò basta chiamare `TAS(&flag, 1)` e verificare il valore restituito dalla TAS (ossia il vecchio valore di `flag`). Se è 0 allora abbiamo trovato che la lock era libero e lo abbiamo occupato, altrimenti lo abbiamo trovato occupato e ci tocca aspettare (non abbiamo modificato nulla, dato che abbiamo semplicemente sovrascritto 1 su 1). Tutto ciò viene implementato in questo modo:

```c
#include <stdatomic.h> // Libreria per implementare le istruzioni atomiche

typedef struct { atomic_int flag; } spinlock_t; // 0 = Libero, 1 = Occupato
static inline int TAS(atomic_int *p, int v) {
    return atomic_exchange(p, v); // TAS in C
}

void spin_init(spinlock_t *l) { atomic_init(&l->flag, 0); } // Poniamo inizialmente
    flag = 0

void spin_lock(spinlock_t *l) {
    while (TAS(&l->flag, 1) == 1){}
}

void spin_unlock(spinlock_t *l) {
    atomic_store_explicit(&l->flag, 0, memory_order_release); // Ritorna flag = 0
}
```

N.B. Ci sono molte cose nuove in questo codice. Vediamole una per una:

- Una **struct** è un tipo di dato composto che raggruppa sotto un unico nome più variabili, dette campi.
- La freccia `->` serve per accedere a un campo di una struct tramite un puntatore. Siccome `->` ha precedenza più alta di `&`, `&l->flag` equivale a `&(l->flag)`, ossia a `&((*l).flag)`.
- `inline` serve invece a migliorare leggermente le prestazioni, eliminando il costo della chiamata: suggerisce al compilatore di sostituire ogni chiamata con il corpo della funzione. In poche parole, al posto di `TAS(&l->flag, 1)` il compilatore inserisce direttamente `atomic_exchange(&l->flag, 1)`.

Vediamo se questo lock rispetta i tre parametri che abbiamo citato all'inizio:

1) Correttezza: **rispettata**. Infatti, se diversi thread chiamano una TAS in contemporanea l'hardware li serializza e solo uno riceverà il vecchio valore 0, mentre gli altri avranno come valore 1, dunque solo uno potrà effettivamente lavorare nella sezione critica.
2) Equità: **non rispettata**. Infatti non essendoci alcun ordine il primo thread che chiama la TAS entra sempre se la sezione critica è libera. Tutto ciò è molto pericoloso, dato che può lasciare dei thread in starvation.
3) Performance: **non rispettata**. Infatti, chi è in attesa di entrare consuma CPU per colpa del `while` nella funzione `spin_lock` che ad ogni giro riscrive sempre 1 in `atomic_exchange`. Questo è ovviamente gravissimo nei processori con un solo core, mentre è già più accettabile nei processori multicore se la sezione critica è molto breve, ma rimane comunque un problema.

CAS

Definizione 3.4 – CAS

Una **CAS** (Compare And Swap) è un'istruzione atomica che riceve un indirizzo, un valore atteso e un nuovo valore, e se la variabile all'indirizzo specificato contiene il valore atteso vi sovrascrive il nuovo valore, altrimenti la lascia invariata. In output può o segnalare se la scrittura è avvenuta o restituire il valore letto.
N.B. A differenza della TAS, che scrive sempre, la CAS scrive quindi solo quando il confronto ha successo.

```
int CAS(int *p, int expected, int new_value){
    if (*p == expected){
        *p = new_value;
        return 1; // Valore modificato
    }
    return 0; // Nessuna modifica
}
```

Tramite una CAS possiamo ricreare una TAS ponendo `expected = 0` e `new_value = 1`, ma tramite una TAS non possiamo ricreare una CAS: questo significa che la CAS è *più generale*. Un lock tramite CAS viene implementato in questo modo:

```
#include <stdatomic.h>

typedef struct { atomic_int flag; } spinlock_t; // 0 = Libero, 1 = Occupato
static inline int CAS(atomic_int *p, int *expected, int new_value) {
    return atomic_compare_exchange_strong(p, expected, new_value); // CAS in C
}
```

```
7
8  void cas_lock(spinlock_t *l){
9      int expected;
10     do{expected = 0;}
11     while(!CAS(&l->flag, &expected, 1)); // Appena sovrascriviamo il valore usciamo
           dal do while
12 }
13 void cas_unlock(spinlock_t *l){
14     atomic_store_explicit(&l->flag, 0, memory_order_release); // Ritorna flag = 0
15 }
```

Tramite una CAS noi possiamo inoltre **aggiornare una variabile condivisa** senza usare alcuna lock:

```
1  int cas_increment(atomic_int *p){
2      int old, new;
3      do{
4          old = atomic_load(p); // Legge il valore
5          new = old + 1; // Lo incrementa
6      }while(!CAS(p, &old, new)); // Lo aggiorna solo se old non è cambiato
7      return new;
8  }
```

Abbiamo quindi trovato un modo di eseguire una RMW senza usare una lock. Gli algoritmi lock-free basati sulla CAS, però, quando lavorano su puntatori (ad esempio in una pila) soffrono di un nuovo problema: l'**ABA problem** (per un semplice contatore come quello appena visto, invece, non è un problema). Per capire questo problema, ipotizziamo di avere una pila globale con 3 elementi: $A \longrightarrow B \longrightarrow C$. Adesso, con una CAS in un thread T_1 eseguiamo un `pop()`, ma prima che venga eseguito il controllo del `while`, T_1 è buttato fuori dalla CPU, e passa in esecuzione un altro thread T_2 che manda 3 CAS: con la prima esegue un `pop()`, con la seconda esegue un altro `pop()`, e con la terza un `push()`, reinserendo A in cima. Prima che T_1 abbia effettuato il `pop()` la pila sarà quindi diventata $A \longrightarrow C$. Ma dato che lui aveva letto i valori `old` $= A$ e `new` $= B$ prima di tutte queste istruzioni di T_2, quando eseguirà la CAS questa avrà successo (in cima c'è di nuovo A), e rimetterà in cima il valore errato B, già rimosso, e non C.

Risolvere l'ABA problem però non è complesso: basta aggiungere alla chiamata della CAS un nuovo valore affianco all'indirizzo che ne specifica la versione, in modo da controllare ogni volta non solo `*p == expected`, ma anche che la versione della nostra struttura dati in questo caso sia rimasta identica.

La CAS, però, non risolve nessuno dei due problemi riguardanti l'equità e la performance della TAS. Per questo motivo viene introdotta una terza istruzione RMW: la **FAA**.

FAA

Definizione 3.5 – FAA

Una **FAA** (Fetch And Add) è un'istruzione atomica che somma un valore a una variabile e ne restituisce il valore vecchio, precedente alla somma.

```
int FAA(int *p, int d){
    int old = *p;
    *p = old + d;
    return old;
}
```

Tramite la FAA nasce un nuovo tipo di lock, che prende il nome di **ticket lock**. L'idea è semplice: ogni thread che controlla se la lock è libero riceve un ticket, che parte da 0. Chi quindi richiede l'accesso alla sezione critica quando questa è libera riceve un ticket uguale al turno corrente (all'inizio entrambi valgono 0) e può entrare subito, mentre gli altri che arrivano dovranno aspettare il loro turno, che è deciso proprio dall'ordine di arrivo. Tutto ciò viene implementato in questo modo:

```
#include <stdatomic.h>

typedef struct {
    atomic_uint ticket; // Prossimo ticket da dare
    atomic_uint turn;   // Ticket corrente
} ticket_lock_t;

static inline unsigned FAA(atomic_uint *p, unsigned d) {
    return atomic_fetch_add(p, d); // FAA in C
}

void ticket_lock(ticket_lock_t *l){
    unsigned my_ticket = FAA(&l -> ticket, 1); // Prendi il ticket (e aumenta il
        prossimo ticket da dare di 1)
    while (atomic_load(&l -> turn) != my_ticket){} // Finché non è il nostro turno
        aspettiamo
}
void ticket_unlock(ticket_lock_t *l){
    FAA(&l -> turn, 1); // Sarà ora il turno del prossimo ticket
}
```

Abbiamo quindi raggiunto finalmente l'equità tanto desiderata. Rimane, però, il problema della performance: tutti i thread in attesa, infatti, continuano a sprecare giri di CPU, e questo non ci

piace.

Risolvere i problemi di performance delle RMW

La prima idea che ci può venire in mente è semplicemente far uscire dalla CPU tramite un context switch tutti i thread in attesa. Questa tecnica prende il nome di **yield**, che in inglese significa proprio cedere, ossia cedere il posto. Questo sicuramente fa sprecare meno CPU rispetto allo spinning (diamo questo nome all'attesa di un thread che ricontrolla continuamente una condizione in un ciclo `while`, senza mai cedere la CPU), ma a causa dei continui context switch risulta comunque sconveniente.

Un'idea invece migliore è quella di mandare a "dormire" tutti i thread in attesa marcandoli come BLOCKED così che questi non possano essere schedulati. Diciamo in questo caso che il thread viene **parcheggiato**. Nasce così il **mutex**, e in particolare questa versione prende il nome di **wait-queue mutex**, e viene implementata in questo modo:

```
void park(void);            // Funzione che ci viene già fornita
void unpark(pid_t tid);     // Funzione che ci viene già fornita

typedef struct {
    int flag;          // Solito flag che vale 0 o 1
    int guard;         // Spinlock interno che protegge le variabili del mutex: per
        poter leggere/modificare il flag o la coda bisogna avere acquisito la guard
    queue_t waitqueue; // FIFO di tutti i thread in attesa
} mutex_t;

void mutex_lock(mutex_t *m){
    guard_acquire(m); // Acquisiamo la guard
    if (m -> flag == 0){ // flag = 0: possiamo entrare
        m -> flag = 1;
        guard_release(m); // Rilasciamo la guard che non ci serve più
    }
    else{ // flag = 1: non possiamo entrare
        queue_add(&m -> waitqueue, gettid()); // Ci aggiungiamo alla coda
        setpark();
        guard_release(m); // Rilasciamo la guard che non ci serve più
        park(); // Ci parcheggiamo
    }
}

void mutex_unlock(mutex_t *m){
    guard_acquire(m); // Acquisiamo la guard
    if (queue_empty(&m -> waitqueue)){ // Coda vuota: flag torna a 0
        m -> flag = 0;
```

```
28      }
29      else{ // Coda non vuota: flag rimane 1 e passo la sezione critica al prossimo
30          unpark(queue_remove(&m -> waitqueue)); // Togli dallo stato parcheggiato il
                primo elemento della coda
31      }
32      guard_release(m); // Rilasciamo la guard che non ci serve più
33  }
```

N.B. Nella dichiarazione, `park(void)` è diverso da `park()`: con il `void` come argomento viene specificato che la funzione non prende argomenti in ingresso, e il compilatore dà errore se gliene vengono passati, mentre senza (prima dello standard C23) il compilatore non controlla gli argomenti delle chiamate, ma passarne a una funzione definita senza parametri è comunque un comportamento indefinito.

Di tutto questo codice la parte più difficile da capire riguarda probabilmente la funzione `setpark()`. Questa serve per avvisare il sistema che il thread attuale *sta per essere parcheggiato*. Ma a che serve avvisare che sto per parcheggiarmi, quando tra pochissimo lo farò direttamente? Serve per risolvere un caso limite molto fastidioso. Ipotizziamo di avere un thread T_1 in esecuzione sulla sezione critica e un altro thread T_2 che ha appena fatto `mutex_lock` e sta per andare a parcheggiarsi perché la lock è occupato. Se T_1 finisce di lavorare e chiama `mutex_unlock` mentre T_2 si trova proprio tra `guard_release` e `park()` ma non si è ancora parcheggiato, la chiamata `unpark()` va a vuoto, perché T_2 ancora non ha chiamato `park()` per parcheggiarsi appunto. Dunque avremo in questo caso una sezione critica libera ma un thread che è in attesa che questa si liberi: un chiaro problema, che viene risolto da `setpark()`. Semplicemente, dice al sistema che lui si sta per parcheggiare, e che dunque se qualcuno lo vuole "svegliare" deve lasciare un messaggio, il quale verrà letto non appena viene eseguita la funzione `park()` e che farà svegliare subito il thread, senza che questo si parcheggi.
Tutto ciò funziona grazie alla guard: questa fa sì che mentre T_2 l'ha acquisita nessuno può fare nulla, ma deve piuttosto rimanere in attesa fino a quando non ritorna libera. In questo modo la chiamata di `setpark()` è sempre assicurata. Viene introdotto quindi con la guard un po' di spinning, ma le funzioni che la riguardano hanno così poche istruzioni da rendere questo spinning ininfluente.

Esiste anche un altro modo di realizzare una lock che dorme, usato dai sistemi Linux: il **futex**. Più precisamente, il futex non è una lock, ma un meccanismo fornito dal kernel con cui costruirne uno. Il lock che abbiamo appena visto, infatti, anche quando nessuno è in attesa deve comunque acquisire e rilasciare la guard, e deve gestire da sé la coda e il problema della sveglia persa. L'idea del futex è gestire la lock in user space con istruzioni atomiche, così che nel caso senza contesa basti una sola CAS, ed entrare nel kernel (con una syscall costosa) solo quando qualcuno deve effettivamente addormentarsi o essere svegliato. Per fare ciò il kernel ci fornisce due funzioni: **`futex_wait(addr, expected)`**, che addormenta il thread chiamante solo se `*addr == expected`, e **`futex_wake(addr, n)`**, che sveglia fino a n thread nella coda associata

all'indirizzo `addr`. Il controllo di `futex_wait` avviene atomicamente nel kernel, quindi nessuna sveglia può andare persa. Queste due funzioni, in realtà, in Linux sono unite in un'unica grande funzione, ossia `futex()`. Se vogliamo una `futex_wait` tramite questa funzione dobbiamo scrivere `futex(addr, FUTEX_WAIT, expected, NULL, NULL, 0)`, mentre se vogliamo una `futex_wake` dobbiamo scrivere `futex(addr, FUTEX_WAKE, n, NULL, NULL, 0)`. L'implementazione di un mutex_lock tramite futex è la seguente:

N.B. In questo codice la CAS ha un comportamento diverso da quella vista prima: `CAS(p, expected, new)` restituisce il valore che ha letto in `*p` (quindi ha avuto successo se questo è uguale a `expected`), e non 1 o 0 per indicare un fallimento come invece faceva la CAS vista prima.

```
void mutex_lock(int *flag){
    int c = CAS(flag, 0, 1); // Poniamo flag a 1 e verifichiamo se la sezione critica è vuota o no
    if (c == 0) // Sezione critica vuota, facciamo una CAS
        return;
    do{ // Sezione critica occupata, facciamo una futex
        if (c == 2 || CAS(flag, 1, 2) != 0){ // Finché la sezione critica è occupata segniamo flag a 2
            futex(flag, FUTEX_WAIT, 2, NULL, NULL, 0); // Dormiamo, ma solo se *flag vale ancora 2
        }
    } while((c = CAS(flag, 0, 2)) != 0); // Al risveglio riproviamo, lasciando lo stato a 2
}

void mutex_unlock(int *flag){
    if (atomic_dec(flag) != 1){ // Se il vecchio valore non era 1 ci sono persone che aspettano
        *flag = 0;
        futex(flag, FUTEX_WAKE, 1, NULL, NULL, 0); // Svegliamo un thread
    }
    // Se era 1 allora non c'era nessuno che aspettava
}
```

Abbiamo quindi 3 possibili stati in questa implementazione:

- `flag = 0` $\implies$ Sezione critica vuota
- `flag = 1` $\implies$ Sezione critica occupata, ma nessuno in attesa
- `flag = 2` $\implies$ Sezione critica occupata e qualcuno in attesa

Un'altra strategia ancora per implementare una lock che dorme è il cosiddetto **two-phase lock**. Questo lock unisce spinning e sleeping insieme: quando un thread trova la lock occupato,

piuttosto che eseguire spinning continuo come fanno le TAS o le CAS e piuttosto che andare subito a dormire come fanno i mutex, questo nel two-phase lock prova prima una fase di spinning a tempo limitato e, se questa non dovesse bastare per entrare nella sezione critica, si parcheggia finché non è il suo turno. Questo tempo limitato, che prende il nome di **spin budget**, spesso vale quanto un context switch (ossia pochi μs), così che, nel caso peggiore, si sprechi al più circa il doppio del tempo rispetto alla scelta migliore (che non si può conoscere in anticipo).

3.2.3 Tabella riepilogativa

Lock	**Corretto**	**Equo**	**Performance senza contesa**	**Performance con contesa**
Disabilitare gli interrupt	*Si* con CPU con solo un core	*N/A*, dato che con un solo core nessun altro thread può andare in esecuzione senza interrupt	*Ottima*	*N/A*, dato che se c'è contesa siamo in una CPU multicore e la lock non funziona
Flag singolo	*No*	–	–	–
Peterson	*Solo con SC* (consistenza sequenziale)	*Si*	*Ottima*	*Pessima*, per lo spinning
Spinlock TAS/CAS	*Si*	*No*	*Ottima*	*Pessima*, per lo spinning
Ticket lock FAA	*Si*	*Si*	*Ottima*	*Pessima*, per lo spinning
Yield lock	*Si*	*No*	*Ottima*	*Non ottimale*, per colpa dei continui context switch
Mutex con coda d'attesa	*Si*	*Si*	*Non perfetta*, per colpa della guard	*Ottima*
Mutex con futex	*Si*	*In parte*	*Ottima*	*Ottima*
Two-phase lock	*Si*	*In parte*	*Ottima*	*Ottima*

Tabella 3.1: Confronto tra tutte le implementazioni di lock

Il mutex con futex e il two-phase lock non sono equi per un motivo molto semplice: la lock non viene passato direttamente al thread in attesa. Infatti, quando il thread in esecuzione sulla

sezione critica rilascia la lock, `flag` torna a valere 0 e viene svegliato uno dei thread in attesa, ma se mentre questo si sveglia arriva un nuovo thread, questo può benissimo rubare il posto agli altri tramite una CAS, col pericolo di lasciare dei thread in starvation. Diciamo In Parte e non No per un motivo semplice: il kernel sveglia i thread più o meno nell'ordine in cui questi si sono addormentati. Dunque, dato che i sorpassi non avvengono così spesso, il pericolo di starvation per un thread con questi due lock è quasi nullo, ma sicuramente non possono essere definiti equi tanto quanto i lock che utilizzano una FIFO.

3.3 Condition variable

3.3.1 Fare aspettare un thread

Abbiamo risolto con i lock il problema della mutua esclusione. Adesso, però, vogliamo poter dire a un thread "**aspetta fino a quando X non succede**". Abbiamo già visto questo problema quando un thread padre deve aspettare la fine di un thread figlio, e lo avevamo risolto usando la funzione **`pthread_join`**, ma vogliamo ora una soluzione generale per ogni condizione X possibile da attendere.

La soluzione inutitiva è quella di usare una variabile che vale 0 finché X non succede e 1 quando questo finalmente accade, e usare nel thread in attesa un loop che gira fino a quando la variabile non vale 1. Questa soluzione altro non è che spinning, dunque il problema rimane lo stesso: il thread in attesa brucia core di CPU senza fare nulla. Rimane anche il problema della data race, dato che la scrittura di un valore rimane divisa nelle solite tre fasi RMW. Inoltre, se la condizione X viene aspettata da più thread, e si vuole garantire la mutua esclusione, con questa implementazione ciò rimane impossibile.

Per risovlere questo problema introduciamo invece un nuovo costrutto, ossia la **condition variable**.

Definizione 3.6 – Condition variable

Una **condition variable** (CV) è un costrutto che permette di addormentare un thread fino a quando una particolarre condizione non diventa vera, e permette a un altro thread di segnalare quando quella condizione è diventata true.

Proprietà 3.1 – Proprietà della CV

- Una CV è sempre usata insieme a un **mutex** per proteggere i dati condivisi
- Un thread addormentato rilascia il mutex e va a dormire tutto in un unico step atomico (così il messaggio da inviare quando si finisce di lavorare è sempre inviato)
- Un thread risvegliato riacquisce il mutex prima ancora di ricominciare a eseguire codice
- Una CV non ha memoria propria: la condizione è sempre espressa da variabili condivise

3.3.2 Struttura interna

Una CV viene implementata tramite una delle variabili di stato (come `done == 1` o `count > 0`), gestite da un mutex, e da una queue di thread addormentati. Per gestirla esistono due funzioni fondamentali:

- **`wait(cv,m)`**, che rilascia **atomicamente** `m` (la lock) e mette il chiamante a dormire e lo inserisce nell'insieme di CV (chiamato *coda d'attesa*) in attesa di un messaggio che gli dica di svegliarsi. Se questo passaggio non avviene atomaticamente, è possibile che un thread rimanga a dormire per sempre (se infatti rilascio la lock ma non vado a dormire, quando qualcuno vorrà svegliarmi non potrà farlo e rimarrò a dormire potenzialmente per sempre). Per effettuare questa operazione in maniera atomica usiamo la funzione `pthread_cond_wait()`, che rilascia `m`, inserire il thread nell'insieme e manda a dormire il thread, tutto in un unico passaggio indivisibile.
- **`signal(cv)`**, che sveglia un thread a caso nell'insieme e lo fa passare dallo stato BLOCKED allo stato READY. La lock rimane però in mano al signaler: il thread svegliato dovrà aspettare che questo la rilasci per poter operare. Per effettuare questa operazione usiamo la funzione `pthread_cond_signal()`.
- **`broadcast()`**, che sveglia tutti i thread addormentati contemporaneamente. Tra questi, il primo schedulato dalla CPU otterrà la lock e potrà lavorare prima degli altri.

Proprietà 3.2 – Regole di una CV corretta

- Qualsiasi operazione di `wait` o `signal` va fatta solo con la lock `m` **acquisita**
- Una volta svegliato controlla la condizione in un while
- Qualsiasi signal va amandato solo dopo aver modificato il proprio stato (non ha senso fare signal se ho ancora `done = 0`)

```
1 // Colui che chiama wait
2 pthread_mutex_lock(&m); // Prendo la lock
```

```
while(!condition){ // Solo se la mia condizione è soddisfatta vado avanti
    pthread_cond_wait(&cv, &m); // Torno a dormire e perdo la lock atomicamente
}
perform_action(); // Faccio le mie operazioni
pthread_mutex_unlock(&m); // Sblocco la lock
```

```
// Colui che chiama signal
pthread_mutex_lock(&m); // Prendo la lock
perform_action(); // Faccio le mie operazioni e modifico la condizione
pthread_cond_signal(&cv); // Signal
pthread_mutex_unlock(&m); // Sblocco
```

N.B. In questa implementazione signal e unlock non sono un'unica operazione atomica. Questo significa il thread che chiama signal può essere deschedulato con la lock ancora in mano. In questo caso, il thread che chiama wait richiede di nuovo la lock e si ritrova in un altro wait in attesa della lock (tutto ciò è implementato nella funzione `pthread_cond_wait()`).

Con questa implementazione abbiamo la certezza che colui che chiama `wait()` non vada mai avanti fino a quando la condizione non diventi vera, indipendentemente dall'ordine di esecuzione dei due processi sullo scheduler.

3.3.3 Problema del produttore e consumatore

Vediamo ora un problema classico risolvibile con una CV, ossia il problema del produttore e del consumatore.

Ipotizziamo di avere un produttore che genera degli itemi e li inserisce in un buffer, che per semplicità può valere contenere solo un oggetto. Il produttore può decidere di inserire un elemento nel buffer solo se questo è vuoto (vale 0), mentre il consumatore può decidere di prendere un elemento dal buffero solo se questo è pieno (vale 1).

L'implementazione più semplice è utilizzando una CV, con un if:

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;

void *producer(void *arg){
    for (int i = 0; i < loops; i++){
        pthread_mutex_lock(&m);
        if (count == 1){ // Se il buffer è pieno aspetto che venga svuotato
            pthread_cond_wait(&cv, &m);
        }
        put(i); // Riempo il buffer
```

```
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m)
    }
}
```

```
void *consumer(void *arg){
    for (int i = 0; i < loops; i++){
        pthread_mutex_lock(&m);
        if (count == 0){ // Se il buffer è vuoto aspetto che venga riempito
            pthread_cond_wait(&cv, &m);
        }
        int val = get(); // Svuoto il buffer
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m)
    }
}
```

Tutto ciò con un solo produttore e un solo consumatore funziona alla perfezione, anche con l'**if**. Si rompe, però, con l'introduzione di un secondo consumatore. Se infatti abbiamo due consumatori C_1 e C_2 e un produttore P, se C_1 va a dormire perché il buffer è vuoto, una volta che il buffer verrà riempito è possibile che C_1 venga svegliato ma che C_2 gli rubi l'elemento nel buffer. A questo punto C_1 sveglio prova a eseguire una `get()` su un buffer vuoto e restituisce un errore. Tutto ciò accade perché la condizione sul buffer viene eseguita una sola volta per colpa dell'if. Per questo motivo è **fondamentale usare un while e non un if**.

Oltre all'if, vi è anche un altro problema in questo codice. È possibile, infatti, che tutti i consumatori e produttori, che si trovano sulla stessa CV, si ritrovino a dormire nello stesso momento, con nessuno che li svegli. Infatti, se C_1 e C_2 vanno a dormire subito, P potrà svegliarne solo uno dei 2, e dato che per svegliare uno dei due consumatori P deve riempire il buffer, una volta svegliato C_1 ad esempio P andrà a dormire anche lui. A questo punto C_1 prenderà l'oggetto, farà una signal, ma a questo punto non è detto che sveglierà P, ma è possibile che svegli C_2. Se questo accade, C_2 andrà anche lui a dormire perché il buffer è vuoto e ci ritroveremo con tutti e 3 i processi addormentati. Tutto ciò accade per un motivo: **abbiamo un solo CV per tutti i processi**. Per risolvere questo problema quindi non ci basta che **aggiungere una seconda CV**: una per i produttori, e una per i processori. In questo modo un produttore sveglierà sempre un consumatore e viceversa, evitando problemi. Il codice finale sarà quindi il seguente:

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv_empty = PTHREAD_COND_INITIALIZER; // Consumatori
pthread_cond_t cv_full = PTHREAD_COND_INITIALIZER; // Produttori

void *producer(void *arg){
```

```
    for (int i = 0; i < loops; i++){
        pthread_mutex_lock(&m);
        while (count == 1){ // Se il buffer è pieno aspetto che venga svuotato
            pthread_cond_wait(&cv_full, &m);
        }
        put(i); // Riempo il buffer
        pthread_cond_signal(&cv_empty);
        pthread_mutex_unlock(&m)
    }
}
```

```
void *consumer(void *arg){
    for (int i = 0; i < loops; i++){
        pthread_mutex_lock(&m);
        if (count == 0){ // Se il buffer è vuoto aspetto che venga riempito
            pthread_cond_wait(&cv_empty, &m);
        }
        int val = get(); // Svuoto il buffer
        pthread_cond_signal(&cv_full);
        pthread_mutex_unlock(&m)
    }
}
```

Tutto ciò si può anche generalizzare a un buffer con N slot e non solo 1. L'idea di base non cambia, ma avremo semplicemente come condizione `count = N` e non `count = 1`, e avremo un buffer implementato come coda circolare per evitare starvation di item.

3.3.4 Due modi per implementare il signal

Esistono due modi per implementare un signal:

- **Semantica Hoare** (1974), dove il thread che chiama `signal` consegna subito la lock all'thread appena svegliato
- **Semantica Mesa** (1980), dove il thread che chiama `signal` si tiene la lock fino a quando non ha finito

I sistemi reali usano la *semantica Mesa*, che è anche quella che abbiamo visto in precedenza anche noi. Questo perché con la semantica Hoare si introduce un context switch ad ogni `signal`, non proprio l'ideale.

3.3.5 Quando il `signal` non è più sufficiente

Ci sono situazioni in cui il `signal` da solo non basta. Ad esempio, se abbiamo una CV con thread che attendono condizioni diverse. Potremmo avere due thread A e B: A aspetta 50 byte liberi, mentre B 100. In questo caso solo A va svegliato quando si hanno 50 byte liberi: se B viene svegliato tornerà a dormire e il segnato andrà perso.

Potremmo pensare di usare più CV per ogni condizione diversa, seguendo lo stesso ragionamento visto prima nel problema del produttore consumatore. La soluzione ottimale però è un'altra, ossia quella di usare la funzione **`broadcast()`**, che ricordiamo sveglia tutti i thread addormentati. In questo modo, utilizzando comunque solo una CV sono sicuro al 100% di svegliare il thread che mi interessava realmente svegliare. In questo modo, però, rallentiamo leggeremente l'operazione perché, svegliando tutti, introduciamo una lock contention. In poche parole, tutti i thread svegli vogliono acquisire la lock, ma dato che solo uno alla volta potrà acquisirla, ci metteremo un po' prima di trovare il thread desiderato.

3.3.6 Thread Pool

Quando abbiamo diversi lavori (o task) da eseguire, piuttosto che eseguire un'operazione costosa e creare N thread da 0, possiamo creare una sola volta N thread "worker", che prendono i lavori da una coda tra loro condivisa, creando così un cosiddetta **thread pool**. L'implementazione è semplice:

```
typedef struct { void (*func)(void*); void *arg; } Task; // Creiamo una task da far
    eseguire a un worker

Task queue[MAX_TASKS]; // Array circolare con le task da eseguire

int head = 0, tail = 0, count = 0; // Aggiungiamo in coda e preleviamo dalla testa

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; // Mutex (lock)
pthread_cond_t cv = PTHREAD_COND_INITIALIZER; // CV

void *worker(void *arg) {
    while (1) {
        pthread_mutex_lock(&m); // Prendo la lock
        while (count == 0){ // Vado a dormire
            pthread_cond_wait(&cv, &m);
        }
        Task t = queue[head]; // Prendo la task da fare
        head = (head + 1) % MAX_TASKS; // Modifico l'indice della testa: il prossimo
            worker deve prendere la task dopo
        count--;
```

```
19          pthread_mutex_unlock(&m); // Lascio la lock
20          t.func(t.arg); // Eseguiamo la funzione fuori dal lock: dobbiamo prima
                lasciarlo altrimenti ce lo teniamo per tutto il tempo della funzione
21      }
22  }
23
24  void submit(void (*f)(void*), void *a) {
25      pthread_mutex_lock(&m); // Prendo la lock
26      queue[tail] = (Task){f, a}; // Aggiungo una nuova task (assumendo count <
            MAX_TASKS)
27      tail = (tail + 1) % MAX_TASKS; // Modifico l'indice della coda
28      count++; // Aumento il numero di task da fare
29      pthread_cond_signal(&cv); // Sveglio un worker che ora può lavorare
30      pthread_mutex_unlock(&m); // Lascio la lock
31  }
```

Se guardiamo bene notiamo una cosa molto interessante: questo thread pool altro non è che un problema produttore consumatore! Infatti, il produttore dà le task ai consumatori con la funzione `submit()`, e i worker (ossia i consumatori) se hanno task disponibili le eseguono, altrimenti dormono finché il produttore non gli dà nuove task da eseguire.

3.4 Semaforo

3.4.1 Unire lock e CV

Fino ad ora abbiamo visto due diversi oggetti utilizzabili per gestire i thread: i *lock*, che ci garantiscono la mutua esclusione, e le *CV*, che ci garantiscono l'ordine tra i thread. Introduciamo ora un nuovo costrutto, il **semaforo**, che ci permette di unire questi due costrutti in uno solo.

La prima implementazione di semaforo consisteva in una scatola di ticket dalla quale ogni thread prende o ripone un ticket. Per prendere un ticket, però, la scatola non deve essere vuota, altrimenti il thread va a dormire finché un altro thread non ripone un ticket nella box. Usiamo quindi due operazioni:

- **`wait`**, per provare a predere un ticket (Proberen)
- **`post`**, per riporre un ticket e svegliare un thread (Verhogen)

N.B. Tra parentesi ci sono i nomi delle funzioni date da Dijkstra nei primi semafori, che altri non è che l'ideatore di questo concetto di semaforo appunto.

In base al numero di ticket **iniziali**, implementiamo cose diverse:

- Con **1 ticket** implementiamo una **mutex**
- Con **0 ticket** implementiamo una **CV**

- Con **N ticket** implementiamo un **thread pool**

3.4.2 Cos'è un semaforo?

Definizione 3.7 – Semaforo

Un **semaforo** è costrutto che unisce un intero ≥ 0 e una coda di attesa.

Abbiamo nell'implementazione base le due funzioni `wait` e `post` già accennate in precedenza, implementate in questo modo:

```
sem_wait(S): // Provo a prendere il ticket
    while (S == 0):
        sleep(); // Dormiamo finché qualcuno non ci sveglia
    S = S - 1 // Diminuiamo S
```

```
sem_post(S): // Lascio un biglietto e sveglio qualcuno
    S = S + 1 // Aumentiamo S
    if queue not empty: // Se c'è qualcuno da svegliare lo sveglio
        wake();
```

N.B. Entrambe le operazioni sono atomiche.

Dunque abbiamo sempre $S = \text{init} + \#\text{post chiamate} - \#\text{wait completate} \geq 0$. La sveglia di un thread nella coda di attesa è implementata come una normale CV.

Con la liberia standard abbiamo, oltre a `sem_wait` e `sem_post`, altre funzioni importanti:

```
#include <semaphore.h>

sem_t s;
int sem_init(sem_t *sem, int pshared, unsigned value); // pshared = 0: thread tutti
    dello stesso processo; value = valore iniziale di s

int sem_getvalue(sem_t *sem, int *sval); // Mostra il valore di s. Solo per debugging
int sem_destroy(sem_t *sem); // Distrugge il semaforo
```

3.4.3 `S = 0`

Con **`S = 0`** implementiamo una **CV**, ordinando quindi i diversi thread come vogliamo dando un ticket al thread che vogliamo che parta prima rispetto a un altro. Ad esempio potremmo

implementare un join in questo modo eseguendo `post()` sul thread figlio quando questo ha finito e chiamare `wait()` sul padre per attendere che il figlio termini le proprie operazioni.

La differenza tra questa implementazione e un CV è una sola ma molto importante: con `post` inseriamo il ticket in modo permanente nella scatola. In questo modo riusciamo a *tenere traccia dello storico* dei `post`: se ci sono 10 thread che fanno `post` lo noto perchè ci sono 10 ticket nella scatola. Questo non accade con `signal` dato che queste non lasciano alcuna traccia se non vi è nessun thread da svegliare.

3.4.4 Produttore e consumatore con i semafori

Possiamo risolvere il problema del produttore e consumatore utilizzando due semafori al posto di due CV. Avremo un semaforo `EMPTY`, inizializzato a N (numero di slot diponibili nel buffer), e un semaforo `FULL`, inizializzato a 0. Su `empty` aspetteranno quindi i produttori, mentre su `full` i consumatori. Con questa impleemntazione, in qualsiasi momento avremo sempre `empty + full = N`.

```
#define MAX 5 // Usiamo un buffer con 5 elementi
int buffer[MAX], fill = 0, use = 0;
sem_t empty, full, mutex;

void init(void){
    sem_init(&empty, 0, MAX); // Tutti gli slot per produrre sono disponibili
    sem_init(&full, 0, 0); // Non consumare prendere nulla
    sem_init(&mutex, 0, 1); // Lock implementata tramite semaforo (N iniziale = 1)
}

void put(int v){
    buffer[fill] = v; // fill è l'indice che ci dice dove inserire i nuovi prodotti
    fill = (fill + 1 ) % MAX;
}

int get(void){
    int v = buffer[use]; // use è l'indice che ci dice dove consumare
    use = (use + 1) % MAX;
    return v;
}
```

```
void *producer(void *arg){
    for(int i = 0; i < 100; i++){
        sem_wait(&empty); // Controlliamo se possiamo produrre
        sem_wait(&mutex); // Entriamo nella sezione critica
        put(i);
```

```
        sem_post(&mutex); // Usciamo dalla sezione critica
        sem_post(&full); // Lasciamo un nuovo item
    }
    return NULL;
}

void *consumer(void *arg){
    for (int i = 0; i < 100; i++){
        sem_wait(&full); // Controlliamo se possiamo consumare
        sem_wait(&mutex); // Entriamo nella sezione critica
        int v = get();
        sem_post(&mutex); // Usciamo dalla sezione critica
        sem_post(&empty); // Consumiamo un item
    }
    return NULL;
}
```

N.B. È importante l'ordine dei due wait: non possiamo assolutamente prendere prima la mutex e poi fare un `sem_wait()` su `full` o `empty`, altrimenti è possibile che un thread prenda la mutex ma vada a dormire subito dopo, tenendosi la mutex con sé e bloccando tutto.

3.4.5 Filosofi a cena

Vediamo ora un problema risolvibile con i semafori, ossia il problema dei filosofi a cena.

Ipotizziamo di avere N filosofi a cena e N coppie di bastoncini. Ogni filosofo, per mangiare, deve avere sia il bastoncino alla propria sinistra sia quello alla propria destra. Inoltre un filosofo, per mangiare, deve prima pensare, e nessun filosofo deve andare in starvation e dobbiamo evitare deadlock, ossia stalli, dove nessuno può mangiare perché nessuno può prendere due bastoncini contemporaneamente.

Una prima idea potrebbe essere quella di far pensare ogni filosofo e poi, una volta finito, di fargli fare una wait sul bastoncino alla sua sinsitra e alla sua destra e, dopo aver mangiato, di fargli chiamare una post sui due bastoncini. Questa idea, che può sembrare funzionante, si rompe in un caso specifico: quando tutti i filosofi hanno preso il bastoncino alla loro sinistra (o alla loro destra). In questo caso sono tutti in attesa e arriviamo proprio in un deadlock.

Questo problema di deadlock si risolve molto facilmente: se tutti i filosofi partono col prendere il bastoncino alla loro sinistra, basta imporre all'ultimo di prendere quello alla propria destra, o viceversa. Rompiamo così il ciclo: se tutti i filosofi prendono il bastoncino alla propria sinistra, il primo filosofo potrà prenderli entrambi perché l'ultimo punterà prima a quello a destra e dopo quello a sinsitra, eliminando quindi il deadlock. L'impelementazione è semplice:

```c
#define N 5

sem_t chops[N]; // Un semaforo per ogni bastoncino, inizializzati a 1 per riprodurre
    un classico mutex

void *philosopher(void *arg){
    int i = *(int *) arg; // i-esimo filosofo
    while (1){ // I filosofi mangiano all'infinito
        think();
        if (i == N - 1){ // Se sono l'ultimo filosofo
            sem_wait(&chop[(i+1) % N]); // Prendo il bastoncino prima a destra (chop
                [0])
            sem_wait(&chop[i]); // Poi a sinistra
        }
        else{ // Se sono un filosofo normale
            sem_wait(&chop[i]); // Prima a sinistra
            sem_wait(&chop[(i+1) % N]) // Poi a destra
        }
        eat();
        sem_post(&chop[i]); // Poso il bastoncino a sinistra
        sem_post(&chop[(i+1) % N]) // Poso il bastoncino a destra

    }
}
```

3.4.6 Implementare il semaforo

Per implementare un semaforo personalmente possiamo usare i due concetti che già conosciamo: le mutex e le CV. Creiamo in questo modo un semaforo che prende il nome di **zemaphore**:

```c
typedeef struct {int value; pthread_mutex_t lock; pthread_cond_t cond;} Zem_t;

void Zem_wait(Zem_t *s){
    // Lock
    pthread_mutex_lock(&s -> lock);
    // CV
    while(s-> value == 0){
        pthread_cond_wait(&s->cond; &s->lock);
    }
    s->value--;
    // Unlock
    pthread_mutex_unlock(&s->lock);
```

```
13 }
14
15 void Zem_post(Zem_t *s) {
16     // Lock
17     pthread_mutex_lock(&s->lock);
18     s->value++;
19     // CV
20     pthread_cond_signal(&s->cond);
21     // Unlock
22     pthread_mutex_unlock(&s->lock);
23 }
```